

云原生与微服务架构实践

王天青 2024-07

星环信息科技（上海）有限公司

www.transwarp.io

07 - 2024



日期	课程内容	作业布置
2024/7/1	1. 云原生与微服务架构实践	
2024/7/2	2. Docker 入门及实践	
2024/7/3	3. Docker 原理	第一次作业
2024/7/4	4. Kubernetes 入门及实践	
2024/7/5	5. Kubernetes 原理	
2024/7/6	6. Spring Boot 及 Spring Cloud 在 Kubernetes 上 up and running	第二次作业
2024/7/7	7. 基于 Kubernetes 实现 DevOps 流程	
2024/7/8	8. 课程总结	大作业

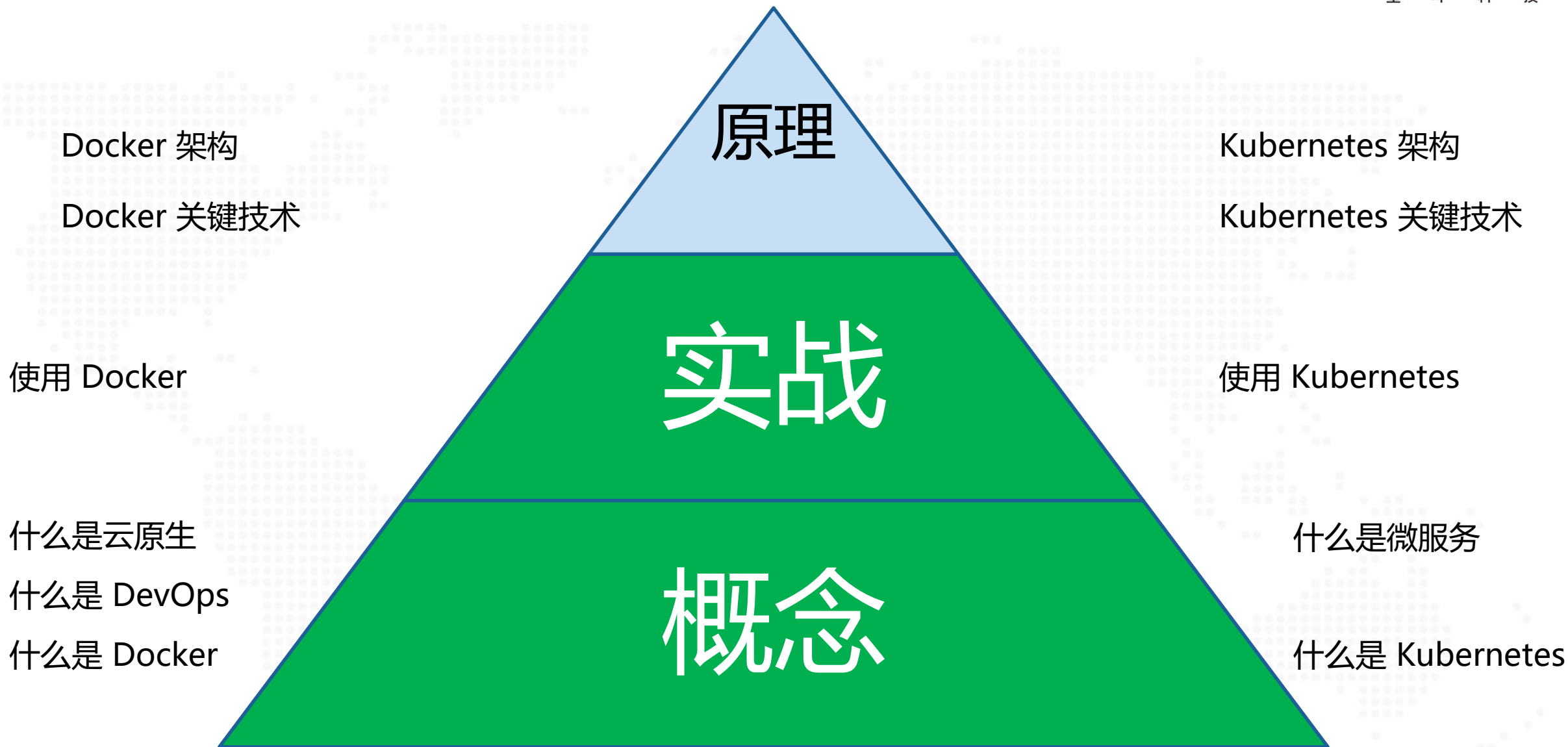
群聊: 2024 - 基于云原生技术的软件开发



该二维码7天内(7月7日前)有效, 重新
进入将更新

原力注入





课后自学

操作系统基础：
《操作系统：设计与实现》
《鸟哥的 Linux 私房菜》

网络基础：
《计算机网络》
《TCP/IP详解卷一：协议》

Kubernetes 深入：
《Kubernetes in Action》

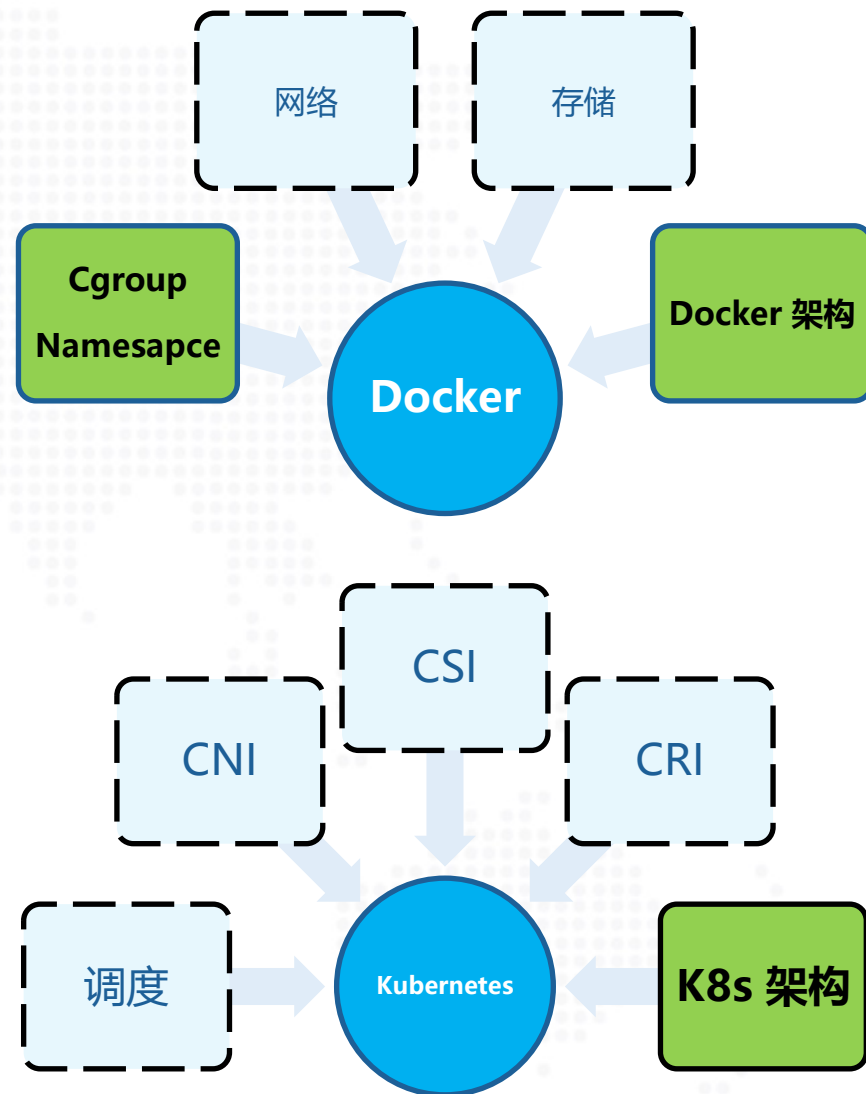
Kubernetes 源码

课程内容

概念

实战

原理



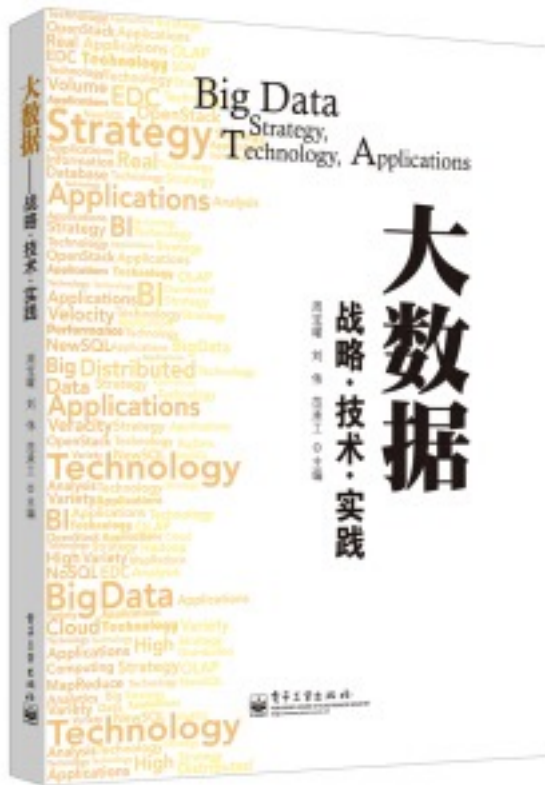
建议

1. 三人为一组
2. Team work (大作业)



About Me

- 南京大学计算机科学与技术系硕士。
- 资深Java 程序员，2003年开始从事J2EE开发，从软件开发做到架构设计。
- 08年加入EMC中国研究院，最高担任云平台主任研究员，长期从事云计算创新技术解决方案设计和实现。
- 2015年9月加入麻袋理财，任首席架构师
- 2016年12月加入 DaoCloud，任首席架构师，负责**微服务**和 **DevOps** 相关产品研发与咨询工作
- 2019年6月任产品副总裁，负责 DaoCloud Digital Platform 的设计和研发
- 2020年8月份加入**星环科技**，任星环数据云负责人，负责 TDC 研发工作
- 在工作期间，分别在中国和美国提交了20+专利申请，截止到2023年1月，其中有20+个专利被美国专利局授权，13个专利被中国专利局授权。



作者之一

——刘强东

京东商城董事局主席兼首席执行官

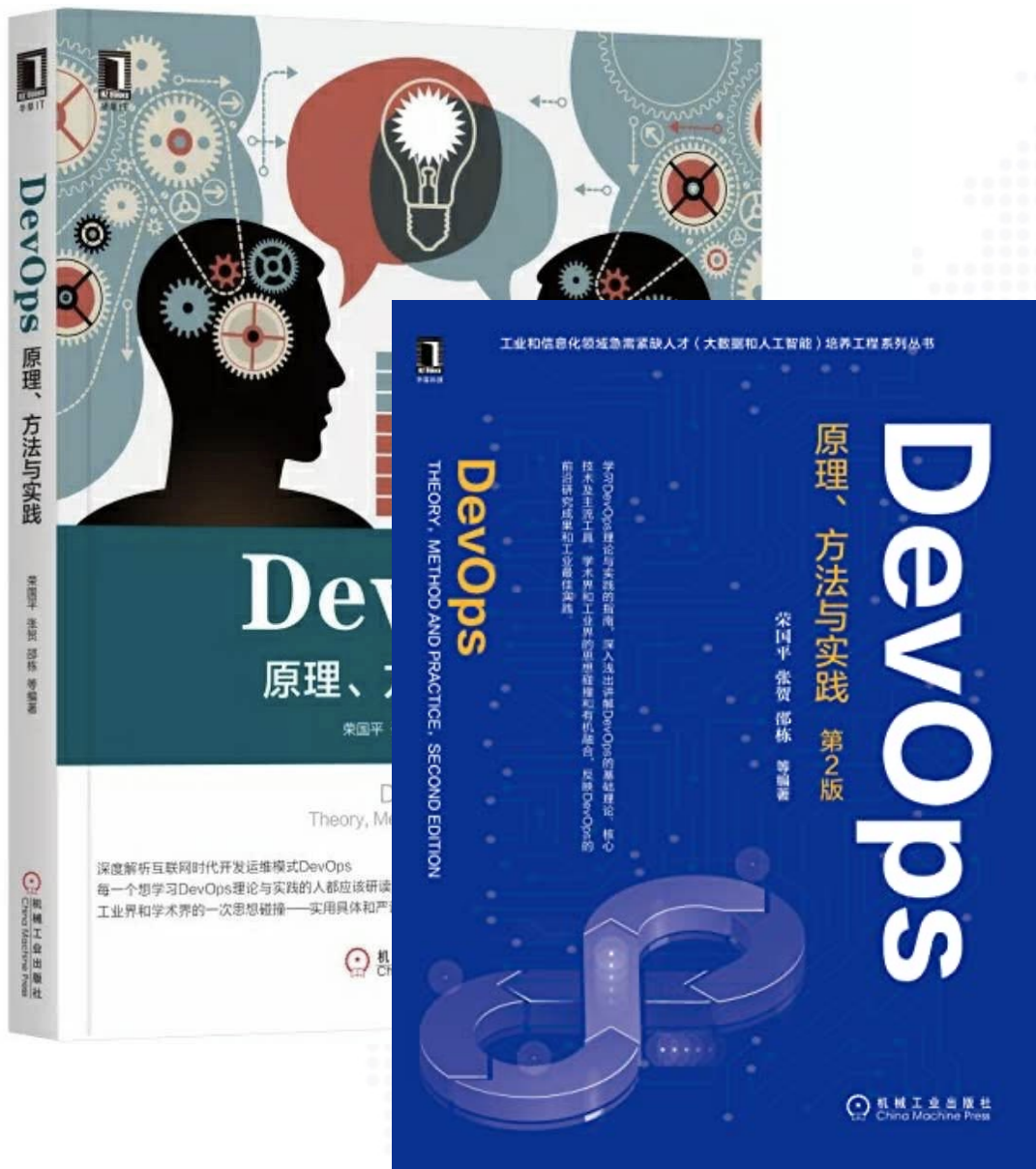
《大数据：战略·技术·实践》从实际技术解决方案出发，提出了大数据技术四层架构，即：基础设施层、管理层、分析层、应用层。在此基础上，全面剖析了当前大数据领域中的主流技术，并配以行业应用实例和一线研发人员的独到见解。力求使读者能够通过阅读此书，全面了解当前大数据技术动态和发展趋势，并可针对自己面临的大数据问题找到可行的解决方案。

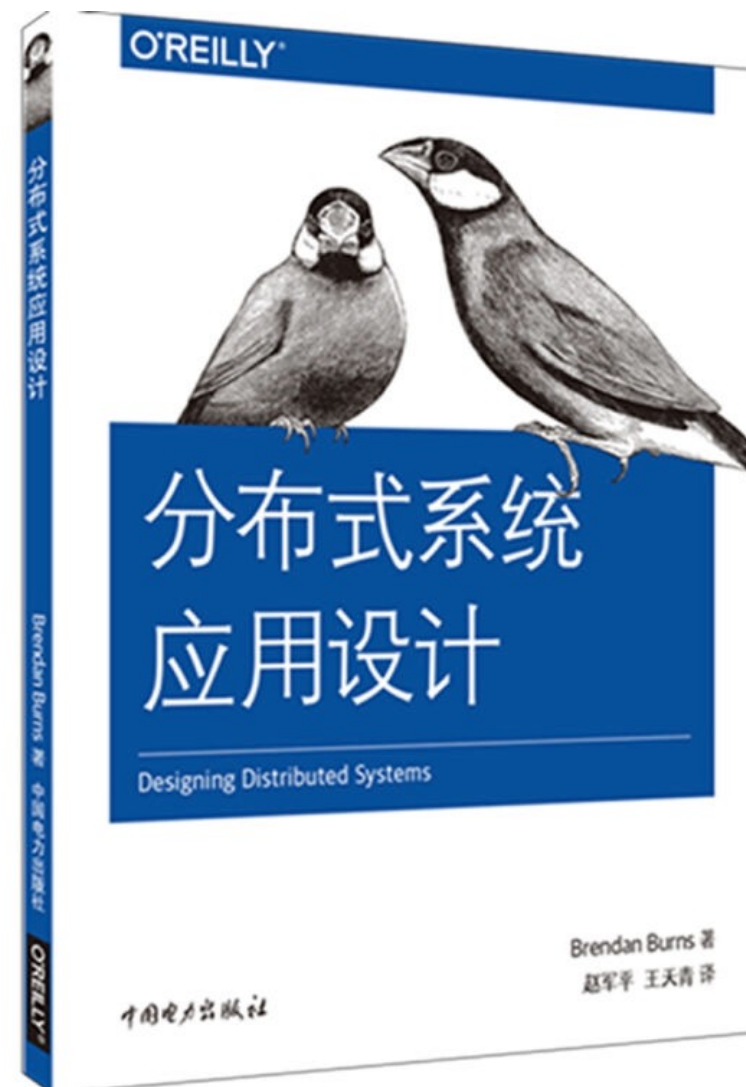
《大数据：战略·技术·实践》是一本不错的技术普及读物。可作为软件开发、数据存储处理工程师、数据分析师等工程技术人员的参考书，亦可作为高等学校相关专业课程的教材或参考书。

大数据时代已经来临，但在国内能系统阐述大数据的书却不多。此次编委会把EMC中国研究院和业界的大数据专业人才汇聚起来，撰写这本关于大数据技术、管理、分析和应用的书，非常及时。EMC是全球领先的存储及管理技术和产品提供商，VMware是全球领先的虚拟化技术和产品提供商，也是开源Cloud Foundry 云平台的贡献者和开源Redis存储系统的赞助者，编写组成员在这两家公司的经验，将会给读者带来他们在大数据技术和实际应用中深刻和独到的见解。

DevOps：原理、方法与实践

本书由南京大学软件学院三位资深教师联合行业一线专家编写而成，系统全面地介绍DevOps——这一互联网时代新型软件开发模式的原理、方法和实践。内容详实、结构清晰、表述浅显易懂，非常适合在校学生学习使用，也可以作为产业界 DevOps 初学者学习参考。





Edge Cloud Operations: A Systems Approach

Peterson, Baker, Bavier, Williams, Davie

The authors provide a detailed overview of cloud operations in this new era of distributed clouds. To understand how it all works...you need to read this book.

– Martin Casado, General Partner, a16z

Synopsis

Everyone uses the cloud to either access or deliver services, and those services are moving to the edge, where the real world meets the virtual. And while the hyperscaler cloud providers are eager to manage those edge clouds as an extension of their data centers, they do not have a monopoly on the know-how to do so. This book lays out a roadmap that a small team of engineers followed over the course of a year to stand up, operationalize, and then continue to operate an edge cloud, the Aether platform.

Features

- Emphasizes underlying concepts, key principles and design choices.
- Describes the main open source components used and the rationale for their selection.
- Walks through the steps from initial hardware configuration to lifecycle management including continuous integration and continuous deployment (CI/CD).
- Includes sample code used in configuration and operations of a real edge cloud running across a dozen enterprises.

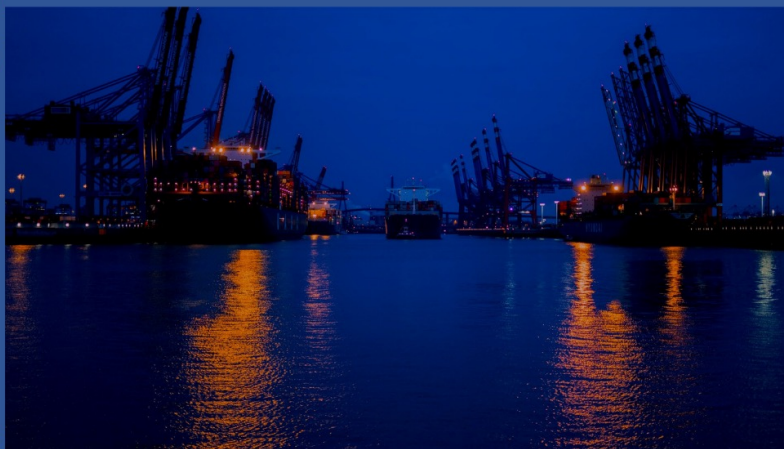
About the Authors

Larry Peterson is the Robert E. Kahn Professor of Computer Science, Emeritus, at Princeton University, and former CTO of the Open Networking Foundation (ONF). **Scott Baker, Andy Bavier**, and **Zack Williams** are Cloud Software Development Engineers at Intel working on the Aether platform. **Bruce Davie** is a former CTO for the AsiaPacific region at VMware, an ACM Fellow, and author of many books and RFCs.



[HTTPS://SYSTEMSAPPROACH.ORG](https://systemsapproach.org)

Edge Cloud Operations A Systems Approach



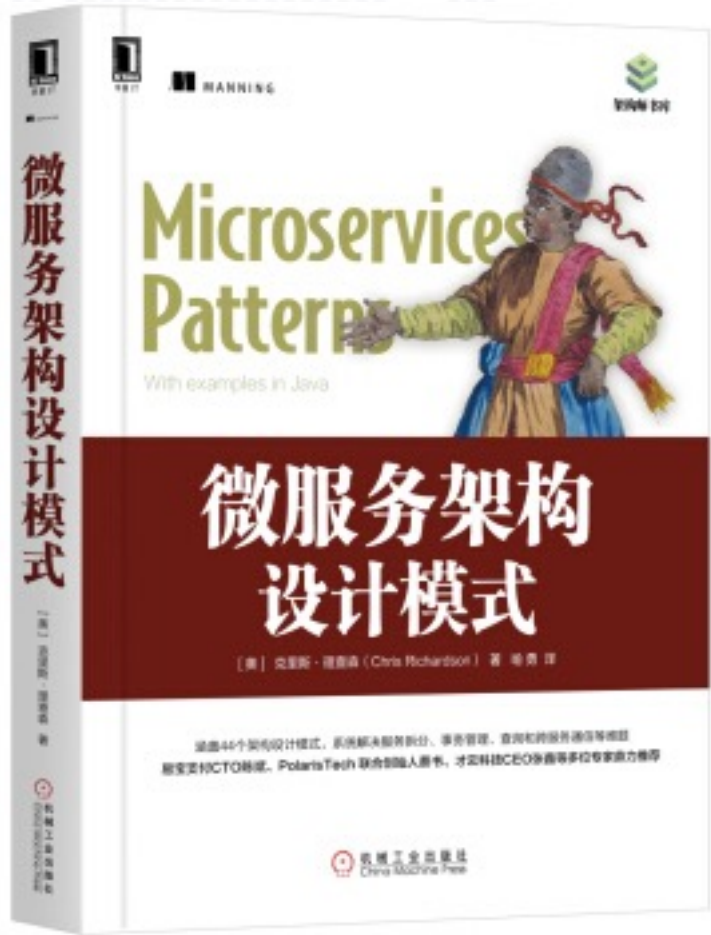
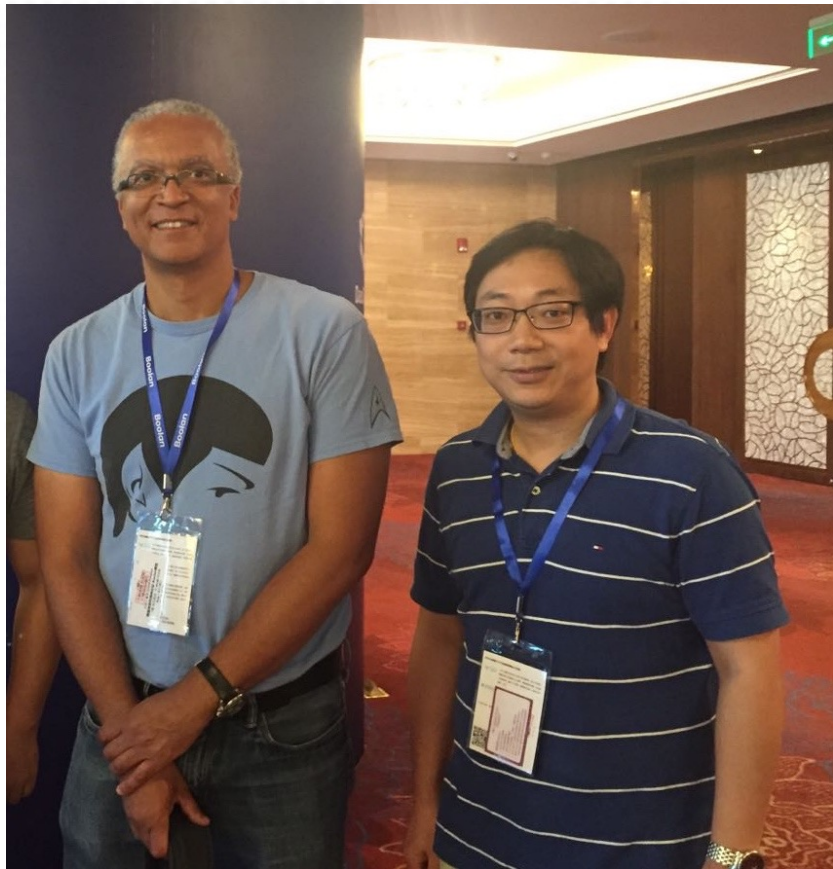
Peterson | Baker | Bavier | Williams | Davie



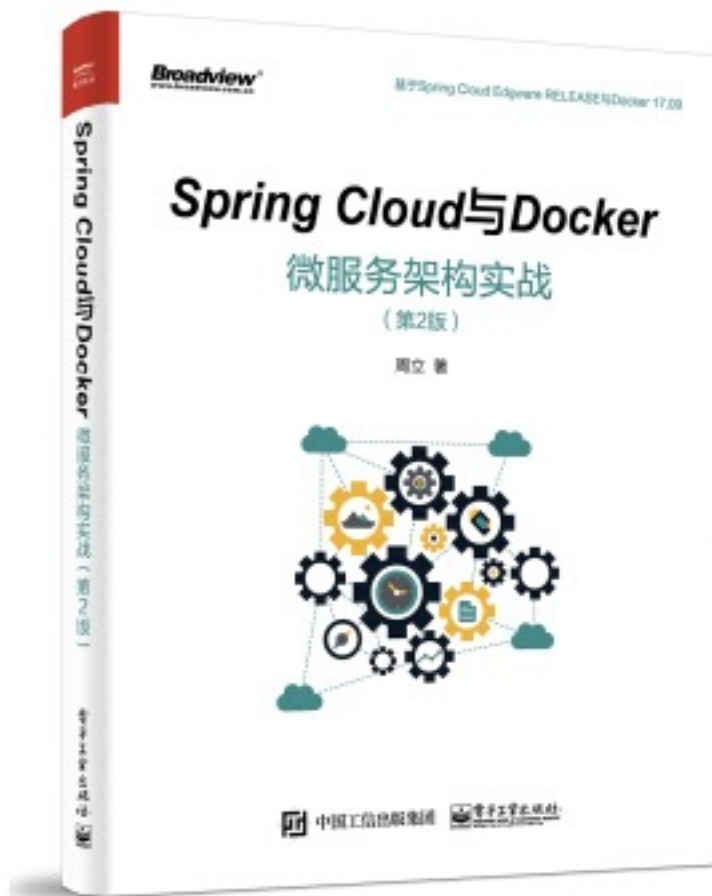
[HTTPS://SYSTEMSAPPROACH.ORG](https://systemsapproach.org)



Chris Richardson (微服务元老之一)



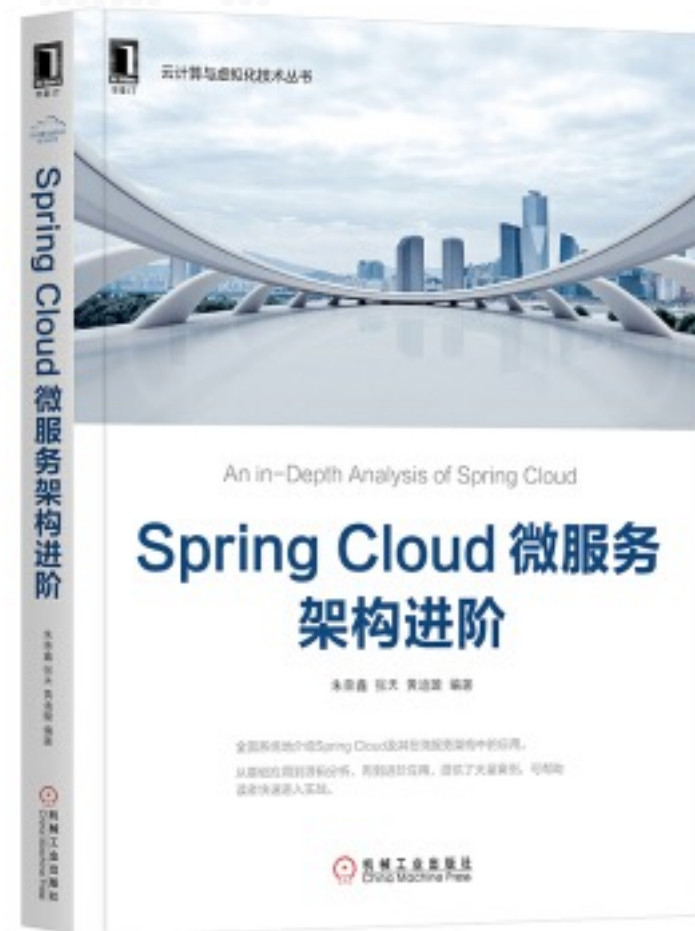
国内几本有名的 Spring Cloud 书籍



Spring Cloud与Docker微服务架构实战

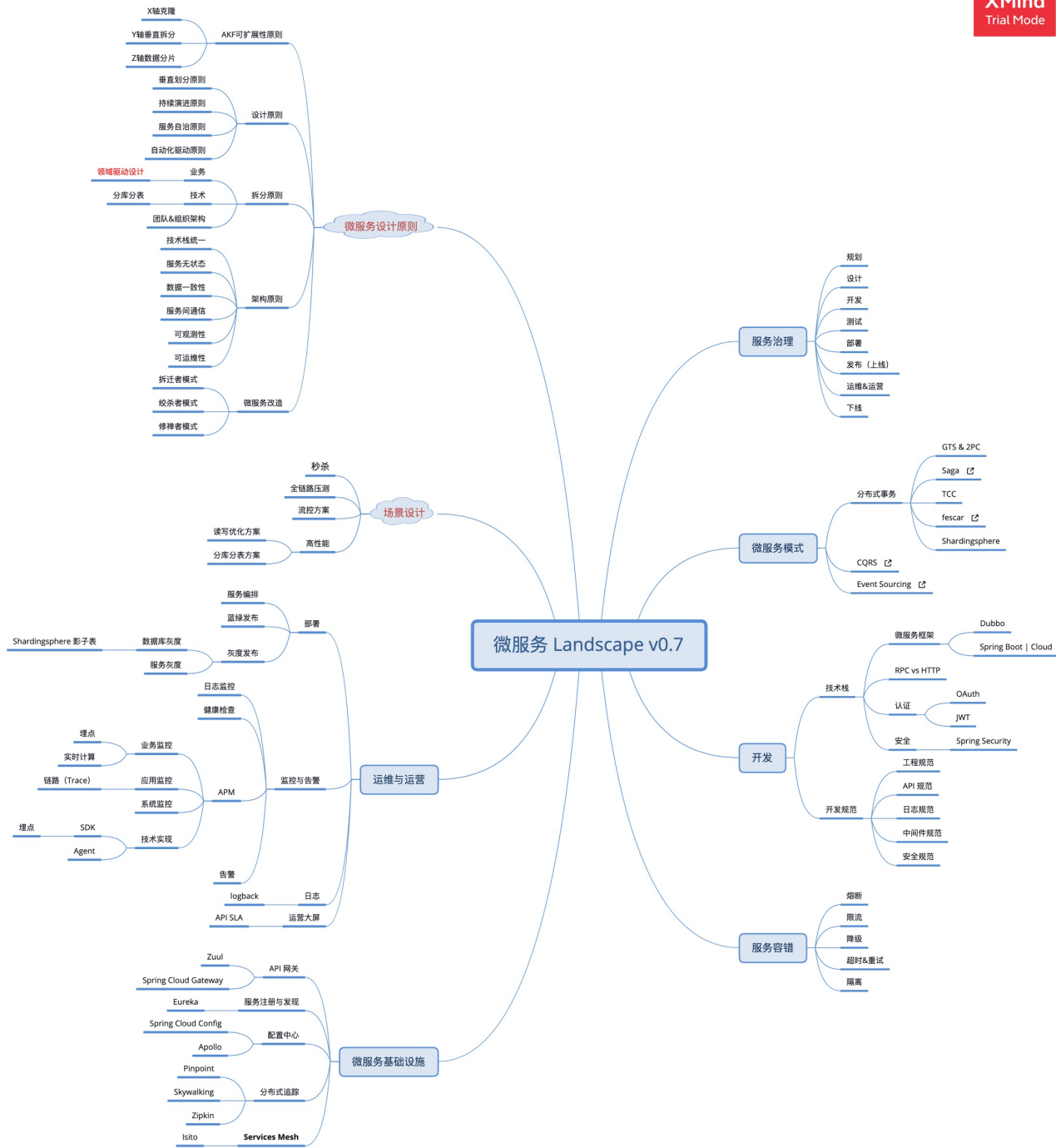


Spring Cloud微服务实战

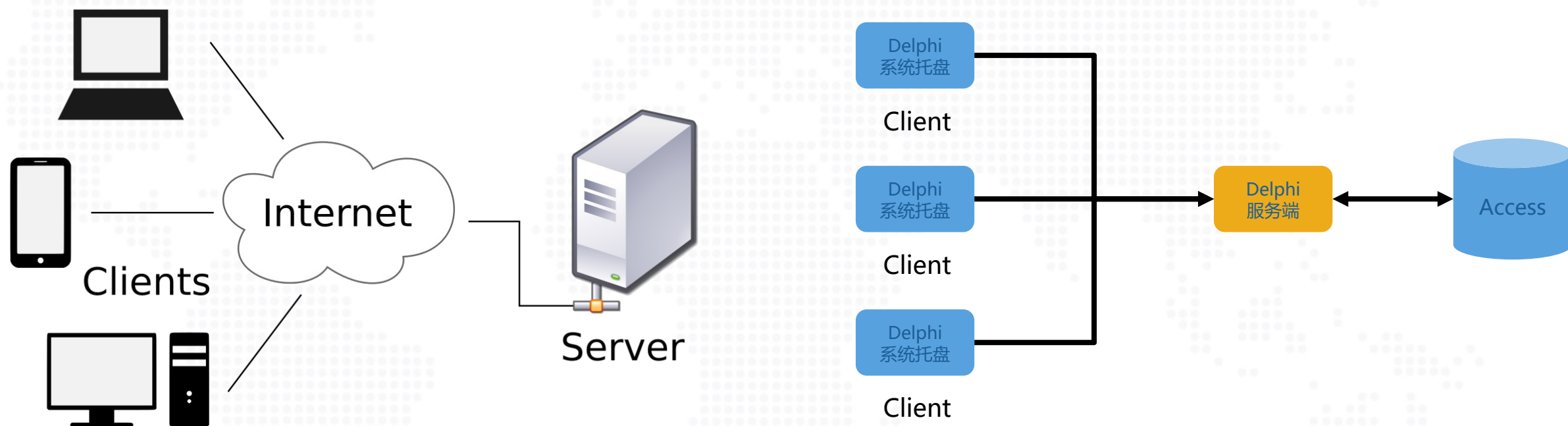


Spring Cloud微服务架构进阶

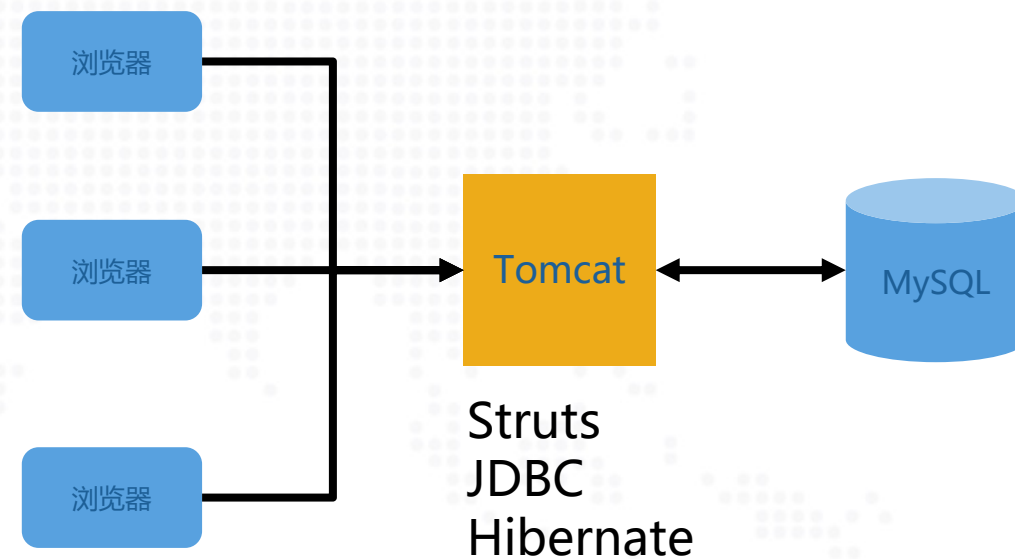
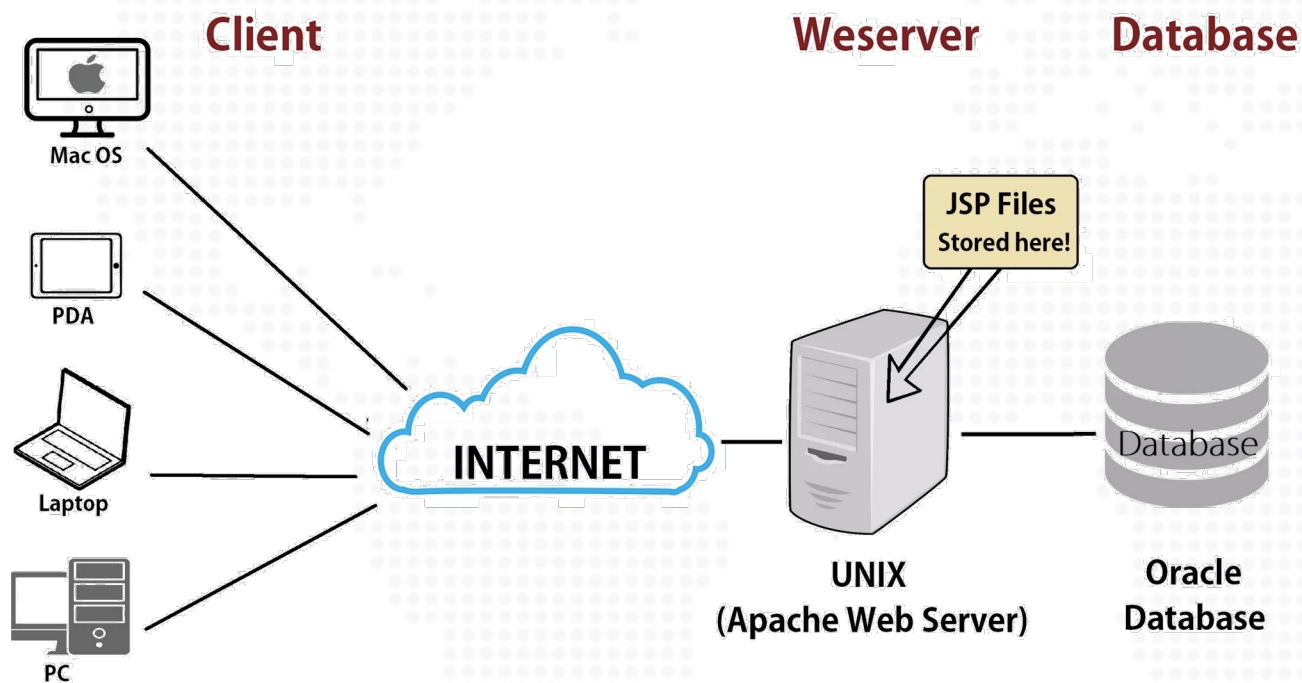
1. 架构历史
2. 云原生简介
3. 微服务架构核心思想
4. 12 范式与 DevOps
5. 微服务原则
6. 微服务拆分原则
7. 微服务改造策略
8. 微服务模式
9. 微服务基础设施及生态
10. 微服务治理
11. 微服务实践案例



架构历史

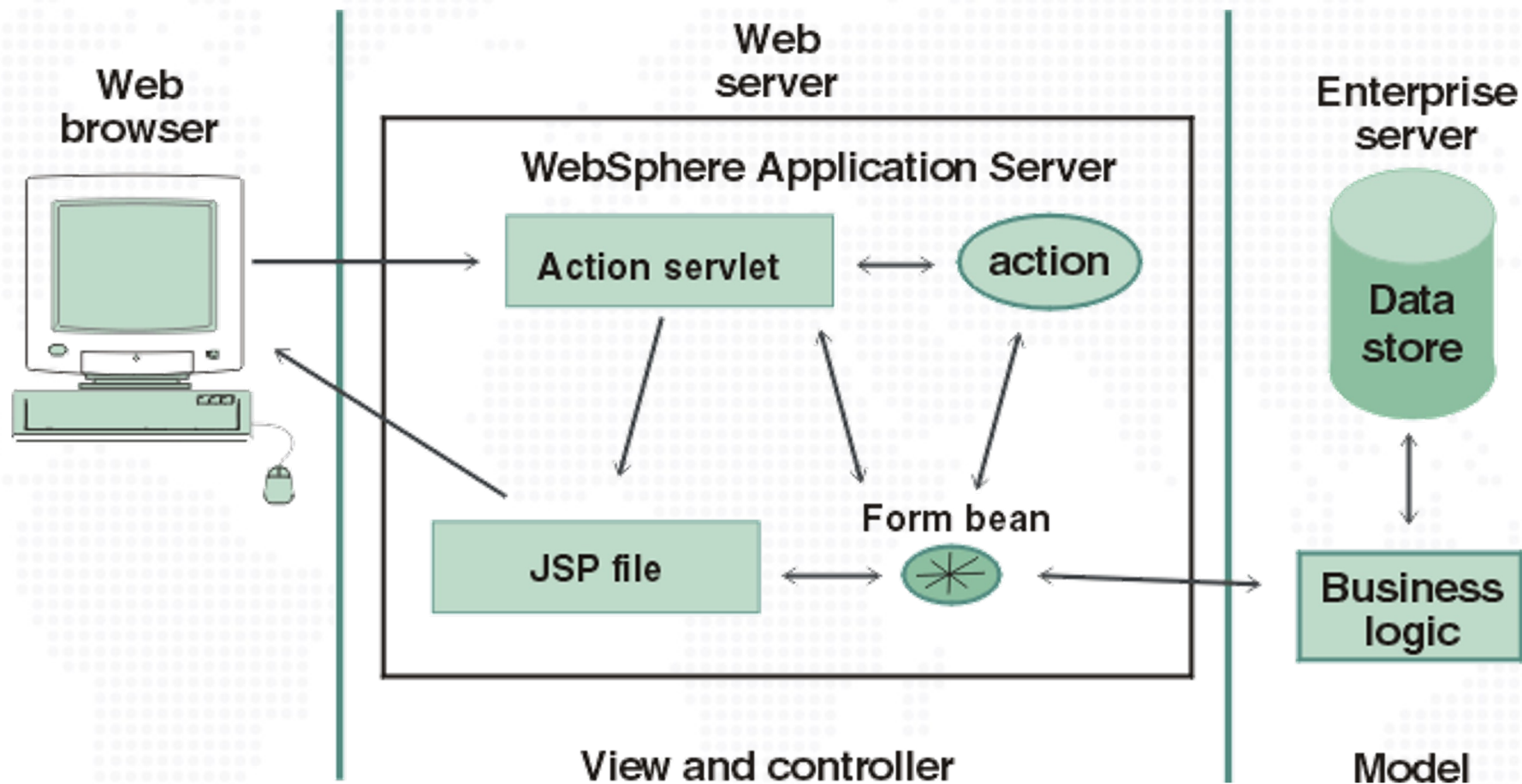


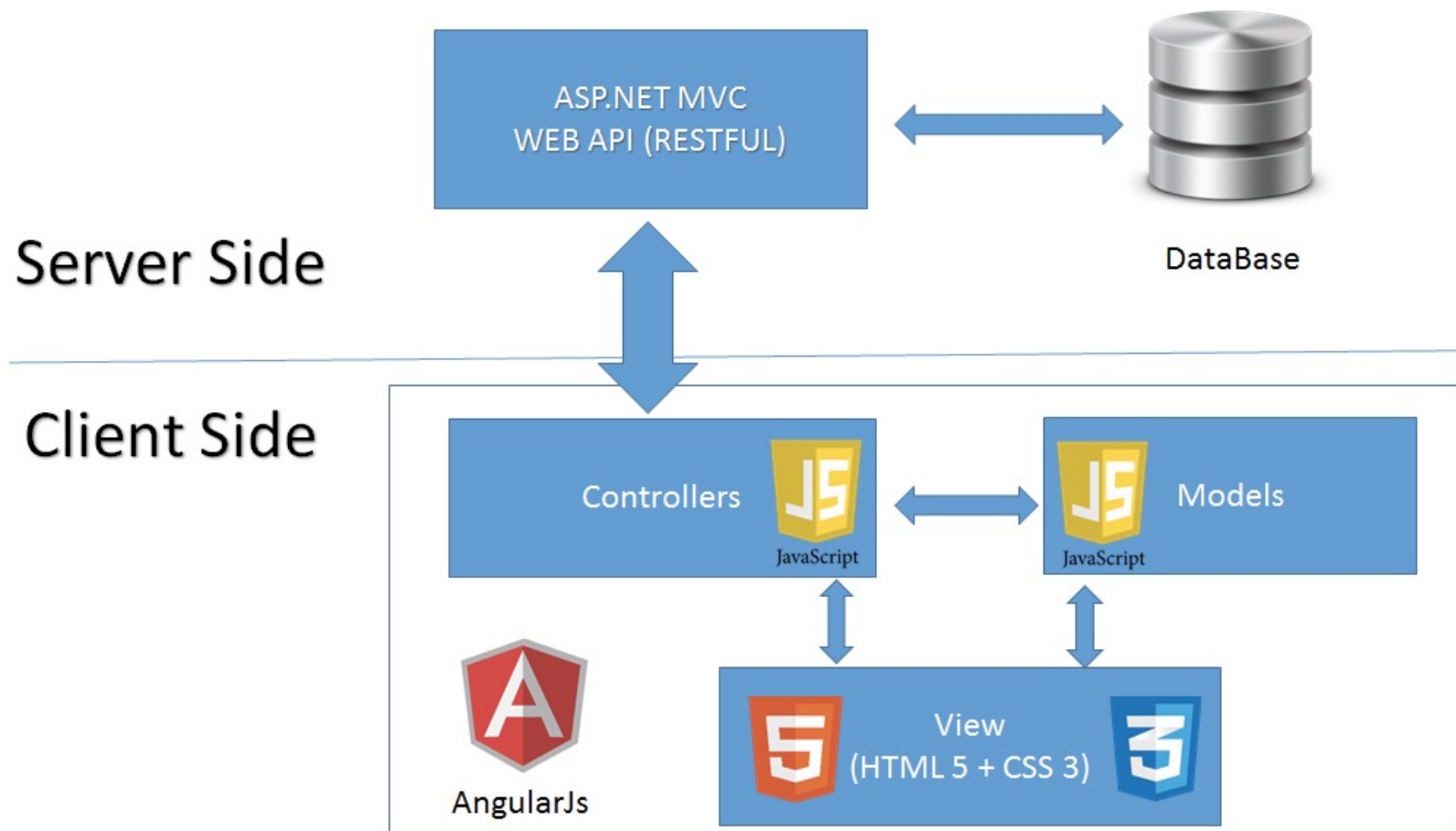
富士通南大: SPIF



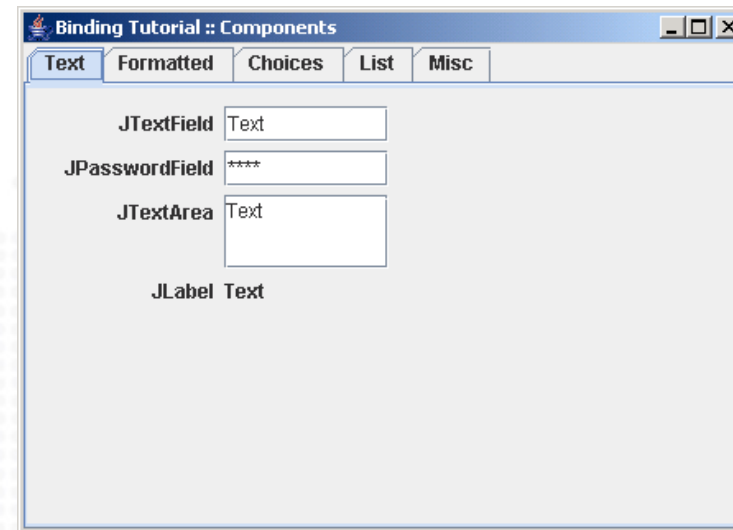
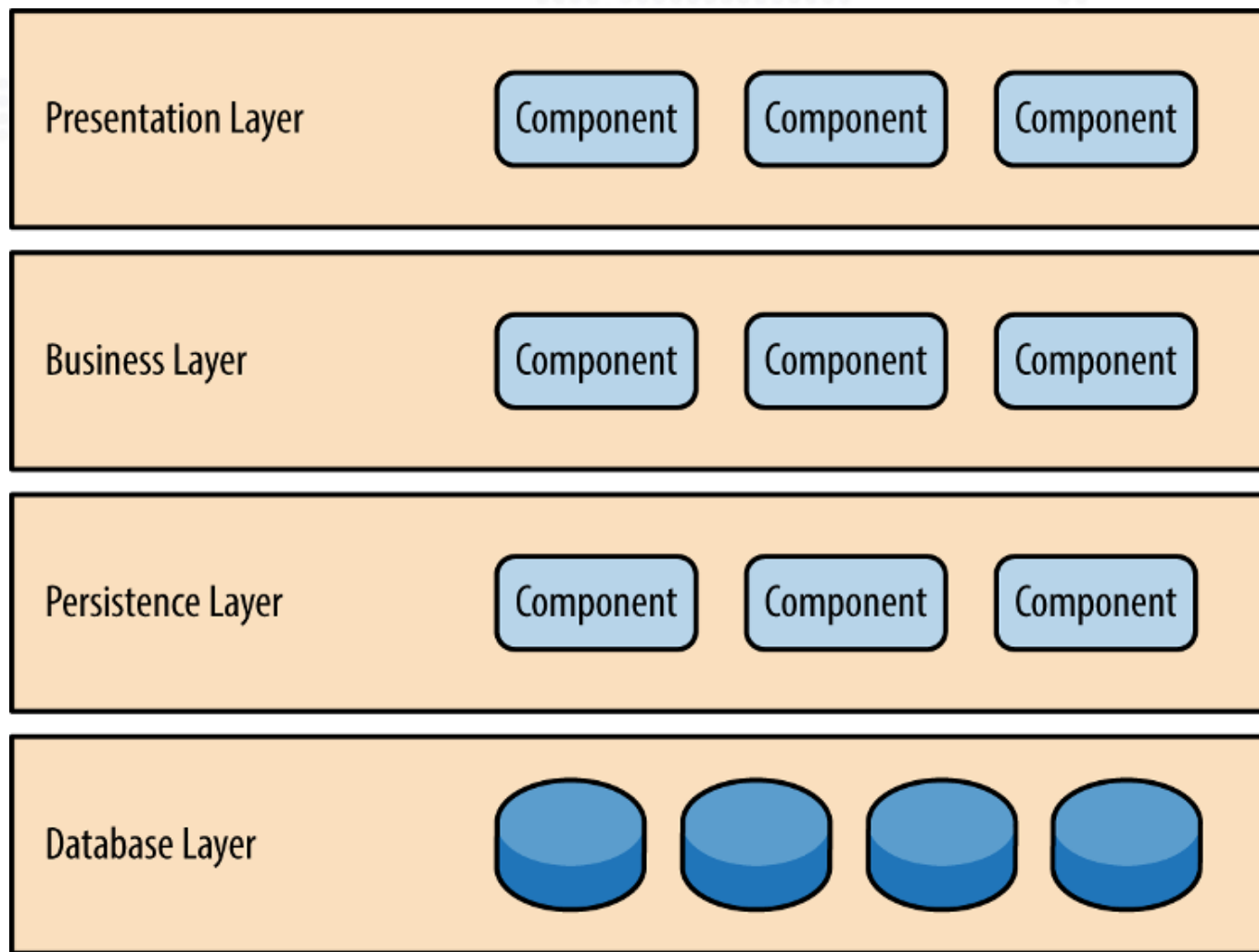
典型: LAMP 架构

服务端 MVC 架构



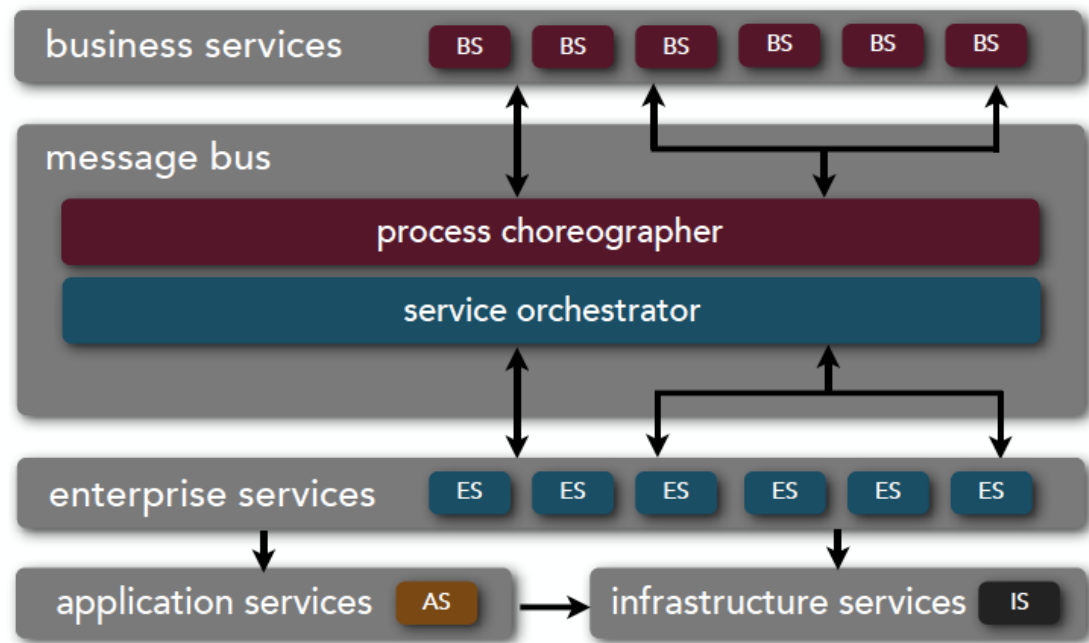


服务端 – N层架构

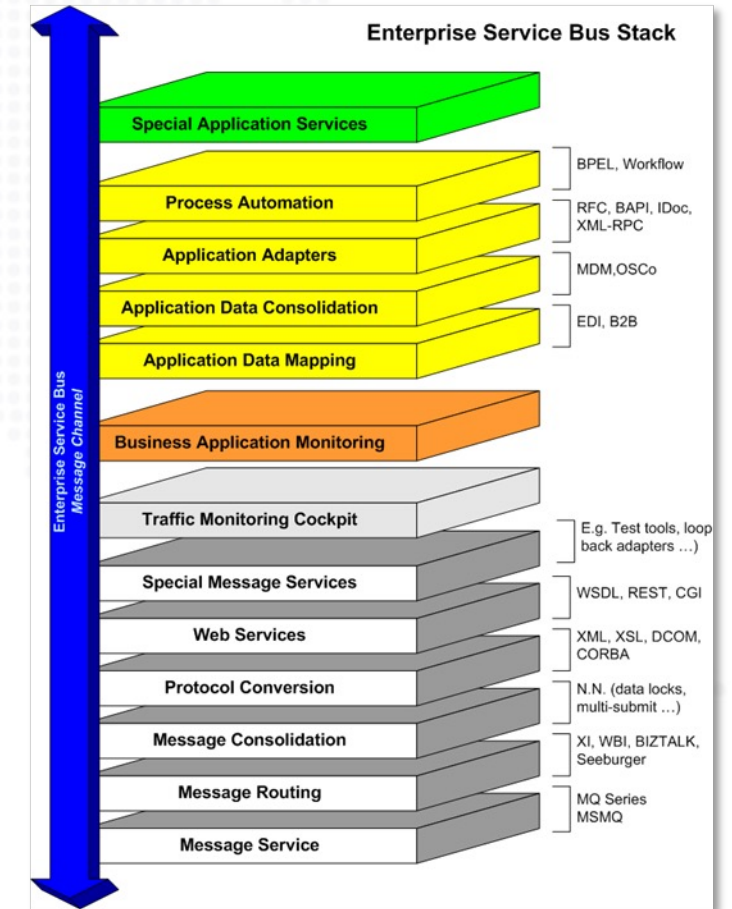
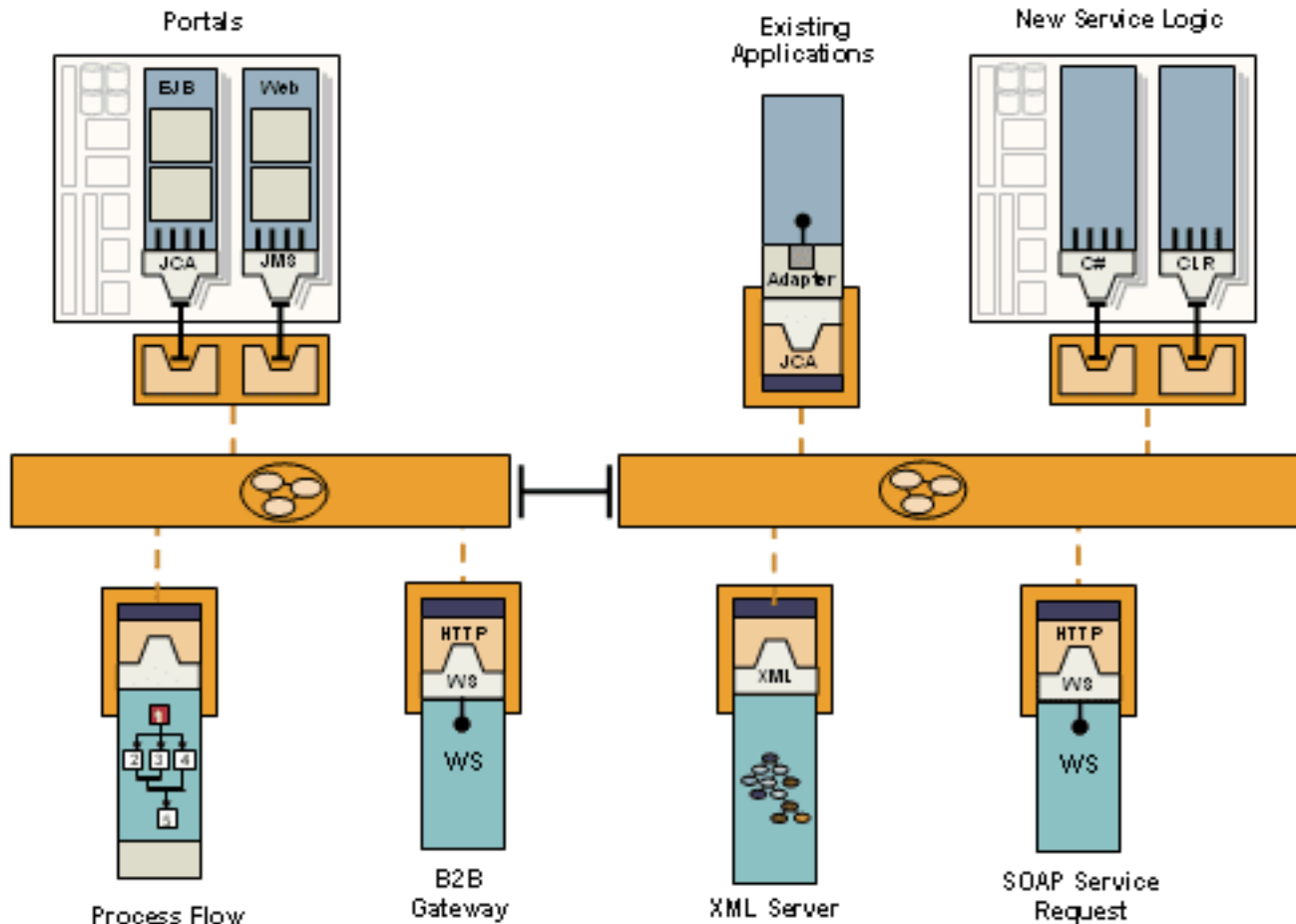


Java Swing

- **展示层 (presentation):** 用户界面，负责视觉和用户互动
- **业务层 (business):** 实现业务逻辑
- **持久层 (persistence):** 提供数据访问，SQL语句就放在这一层
- **数据库 (database):** 保存数据



- 服务封装
- 服务松耦合(Loosely coupled): 服务之间的关系最小化, 只是互相知道。
- 服务契约: 服务按照服务描述文档所定义的服务契约行事。
- 服务抽象: 除了服务契约中所描述的内容, 服务将对外部隐藏逻辑。
- 服务的重用性: 将逻辑分布在不同的服务中, 以提高服务的重用性。
- 服务的可组合性: 一组服务可以协调工作并组合起来形成一个组合服务。
- 服务自治: 服务对所封装的逻辑具有控制权
- 服务无状态: 服务将一个活动所需保存的资讯最小化。
- 服务的可被发现性: 服务需要对外部提供描述资讯, 这样可以通过现有的发现机制发现并访问这些服务。



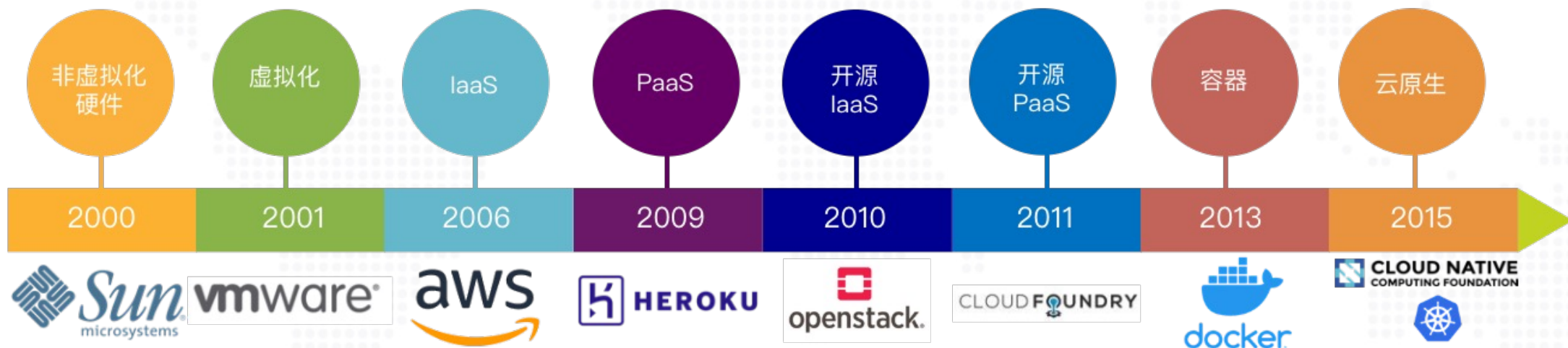
云原生简介

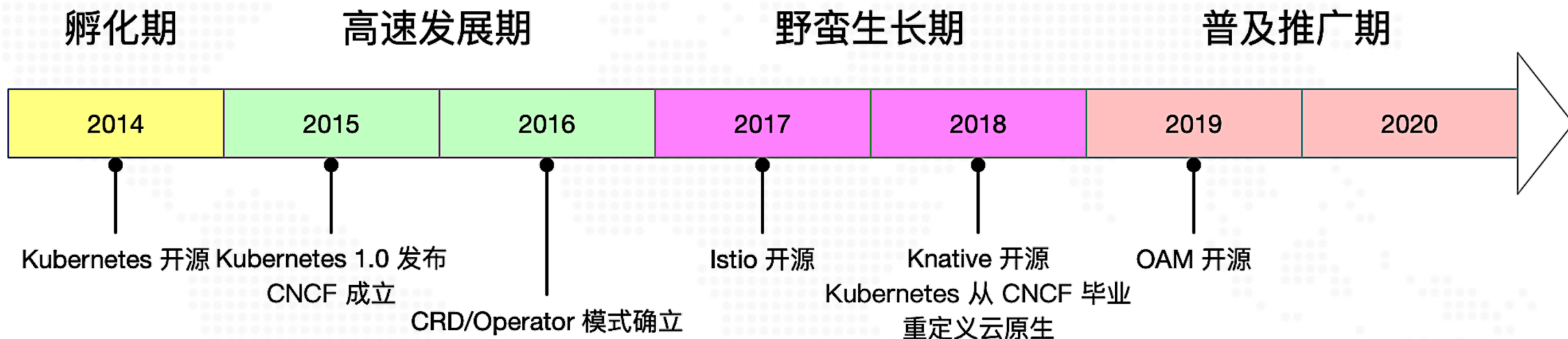




kubernetes

- 云原生计算使用开源软件栈：
 - 根据微服务将应用进行细分，
 - 将每一部份打包放入对应的容器
 - 动态统筹管理容器，实现资源利用最大化

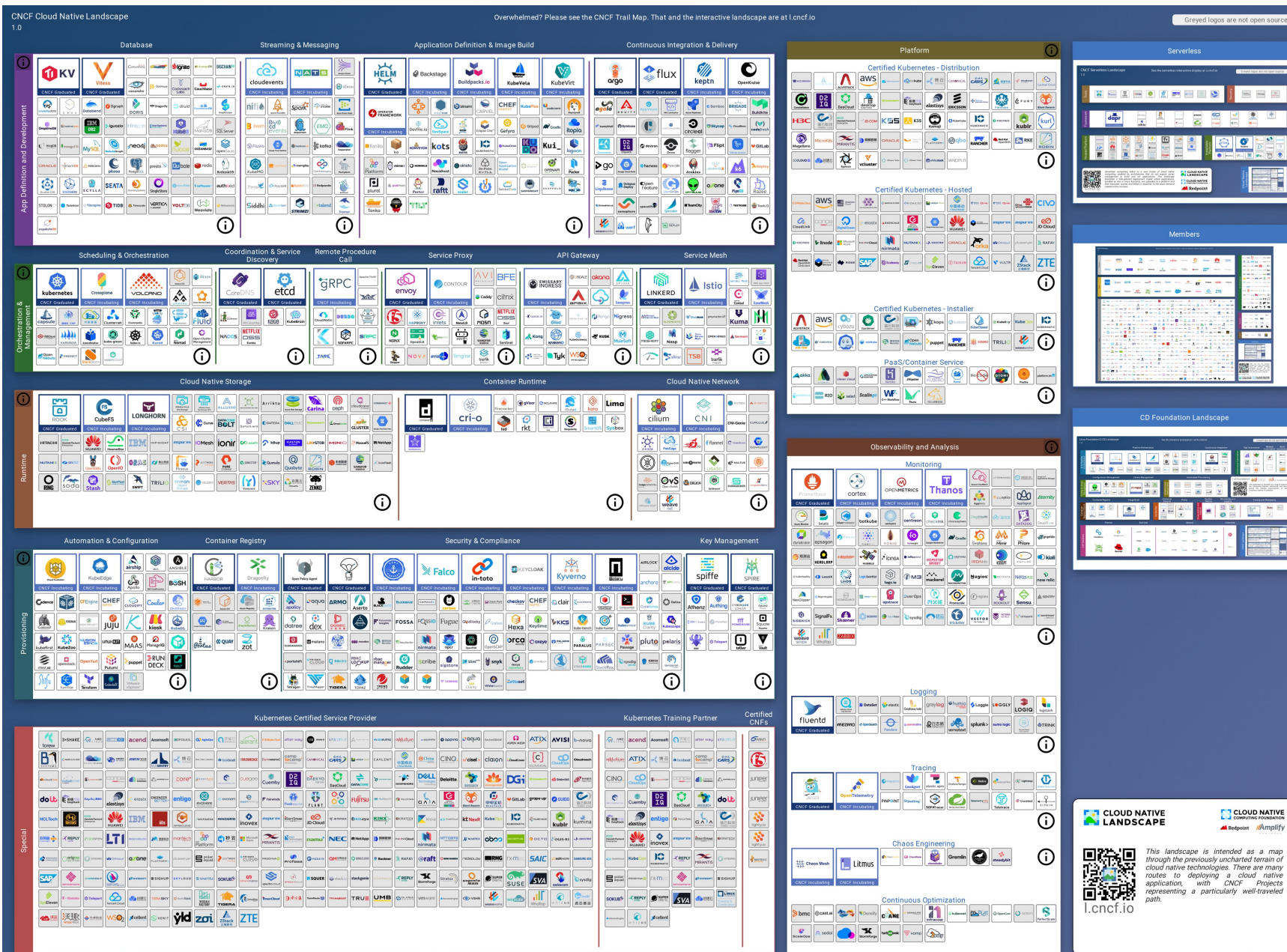


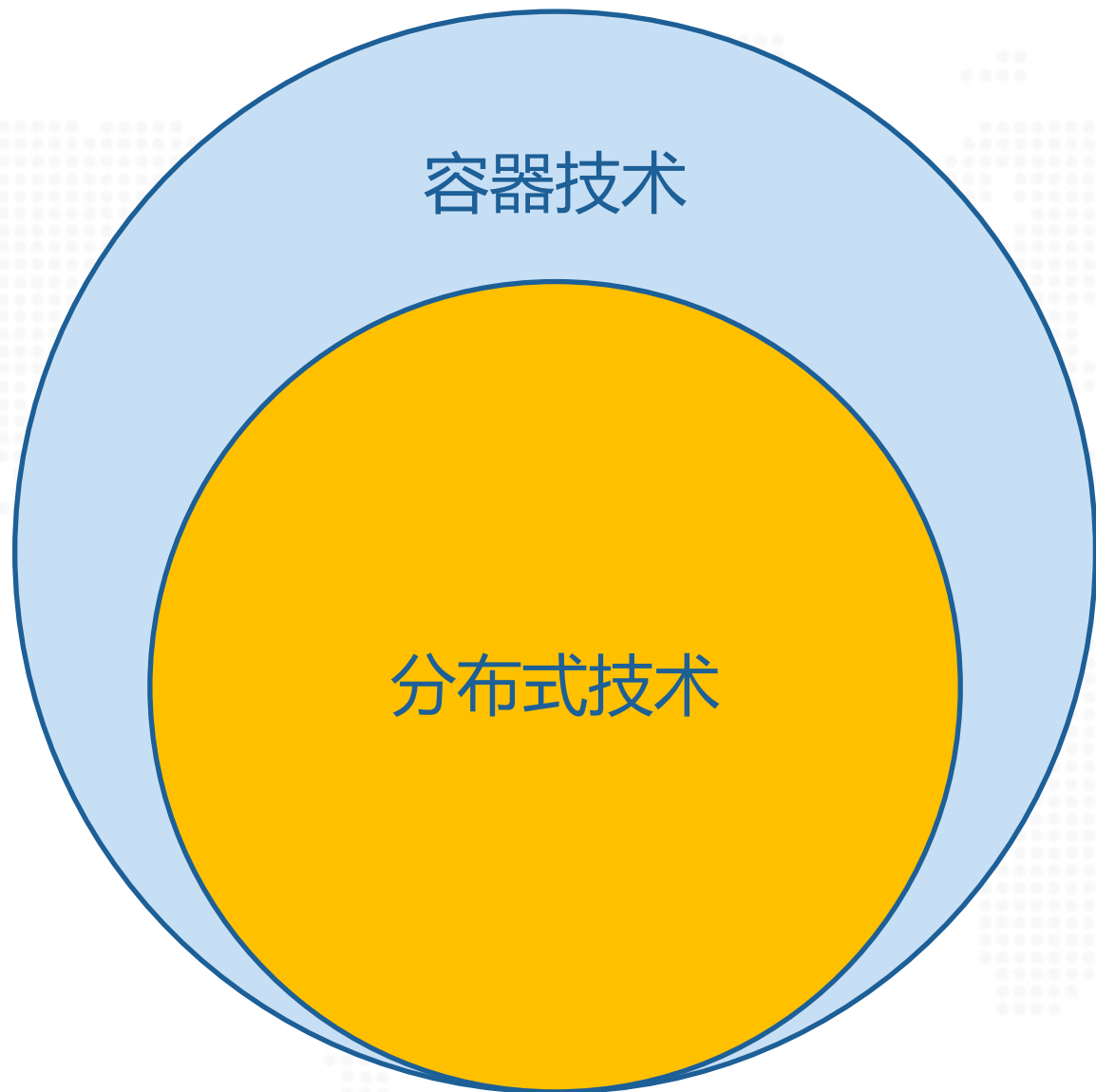


2018年6月，CNCF正式对外公布了更新之后的**云原生**的定义 v1.0 版本：云原生技术有利于各组织在公有云、私有云和混合云等新型动态环境中，构建和运行可弹性扩展的应用。云原生的代表技术包括「**容器、服务网格、微服务、不可变基础设施和声明式 API**」。这些技术能够构建容错性好、易于管理和便于观察的松耦合系统。结合可靠的自动化手段，云原生技术使工程师能够轻松地对系统作出频繁和可预测的重大变更。

云原生应用专为云模型而开发。小的专用功能团队快速将这些应用构建和部署到可提供轻松的横向扩展和硬件解耦的平台，为企业提供更高的敏捷性、弹性和云间的可移植性。企业需要一个用于构建和运行云原生应用和服务的平台，来自动执行并集成 DevOps、持续交付、微服务和容器等概念。

CNCF Landsacpe





云原生 (Cloud Native) 是一种基于微服务的软件架构思想以及基于DevOps进行软件开发实践的一组方法论，其实现了以分布式和容器技术为核心在异构云平台环境下利用云计算交付模型优势来快速构建、部署和运行企业应用。



应用上云之路 2019 年版

应用的视角

采用 DevOps

加速应用交付

采用容器技术

标准化和高效

实现可观测性

运行可度量

全局的视角

建设数字平台

推动业务创新

微服务化改造

可伸缩性

实现可调度性

高性能与运行可靠性

采用云原生安全

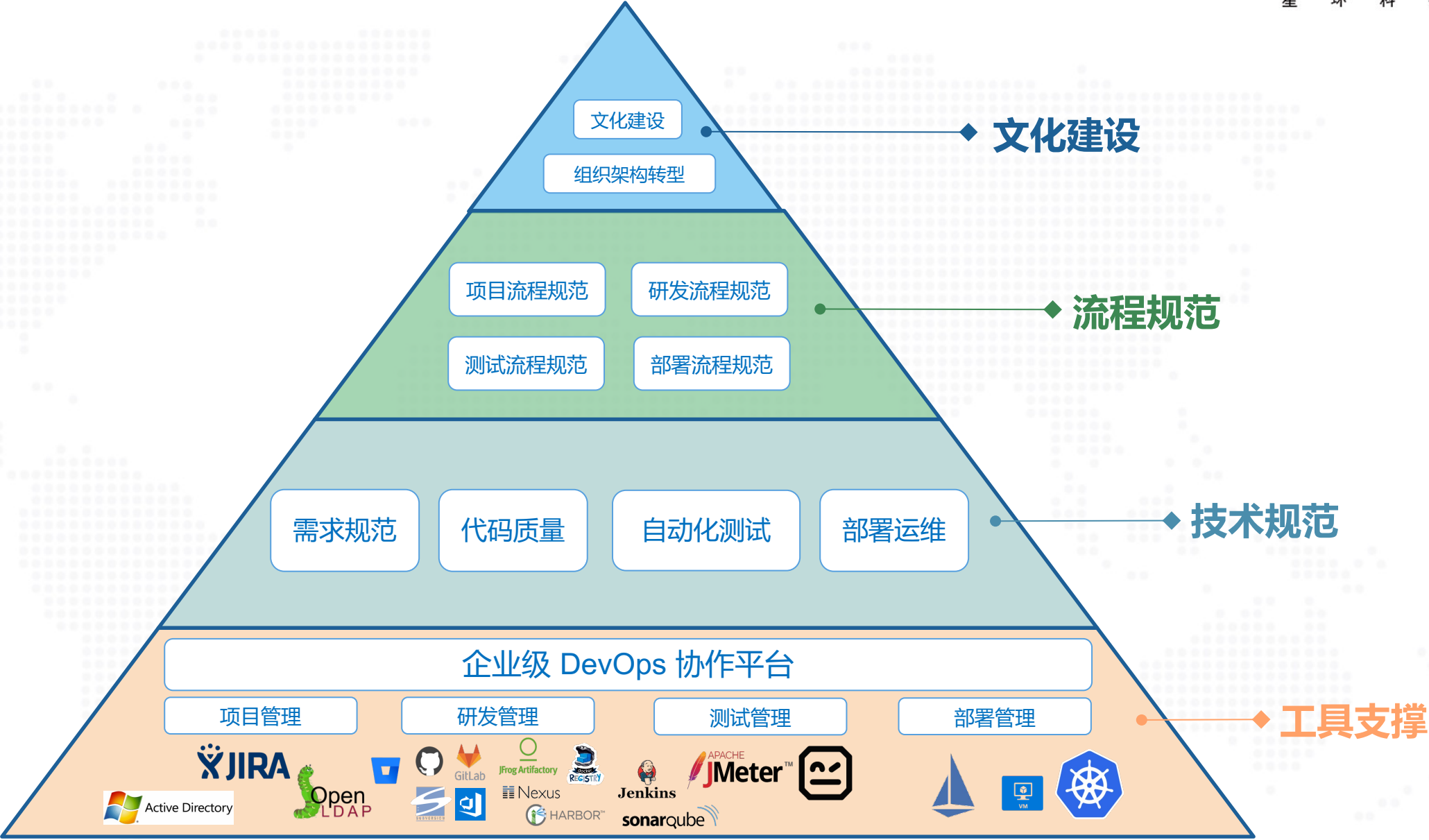
提升安全性

自动运维

提升运维效率

应用运营

提升业务效率



微服务架构核心思想

■ 2005年

Peter Rodgers 博士于 2005 年度云端运算博览会提出的微 Web 服务 (Micro-Web-Service) 开始

■ 2014年

Martin Fowler 与 James Lewis 共同提出了微服务的概念，定义了微服务

- 一组小的服务
- 独立的进程
- 轻量级通信
- 基于业务能力
- 独立部署
- 无集中式管理
- 参考： <https://www.martinfowler.com/articles/microservices.html>

分而治之



Martin Fowler

- Loosely coupled service oriented architecture with bounded context
- 基于有界上下文的，松散耦合的面向服务的架构



Adrian Cockcroft

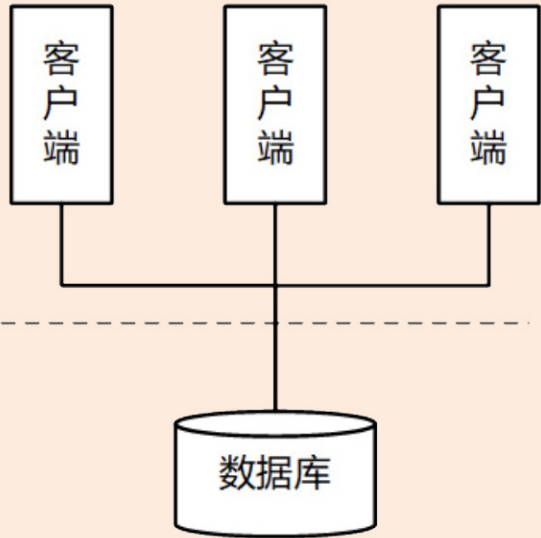
利

- 强模块化边界
- 可独立部署
- 技术多样性

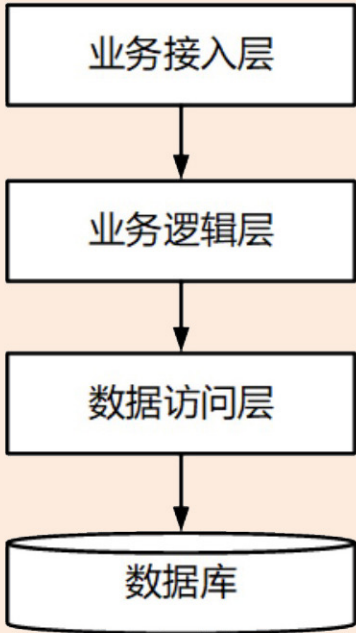
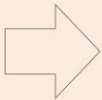
弊

- 分布式系统复杂性
- 最终一致性
- 运维复杂性
- 测试复杂性

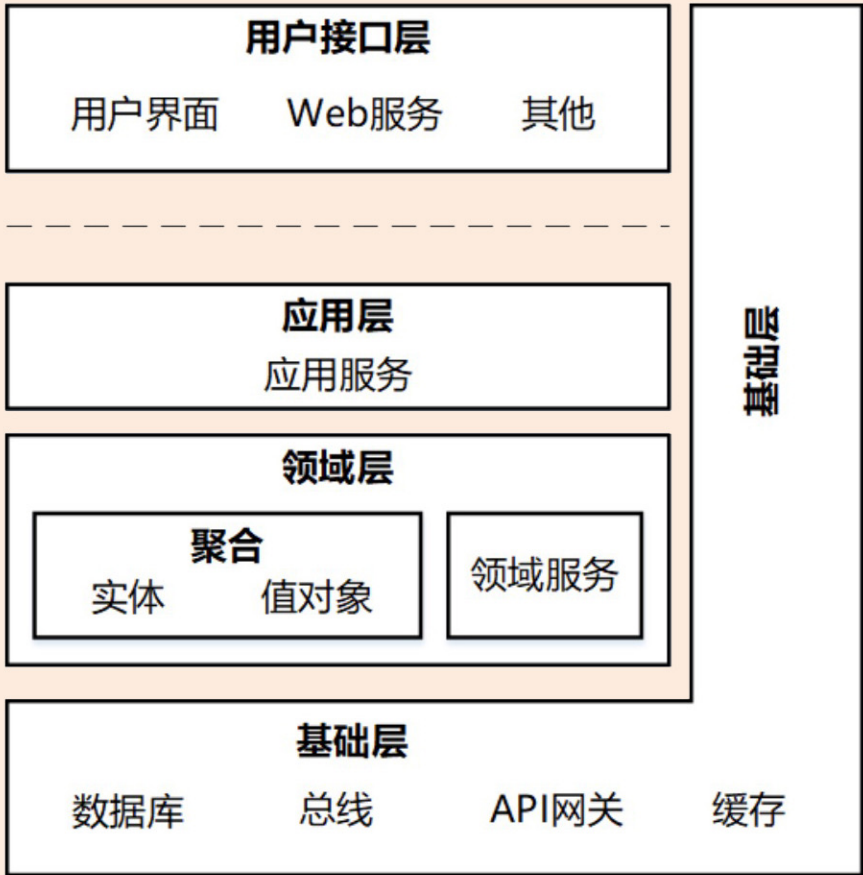
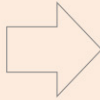
软件架构模式的演进



两层架构
(单机架构)

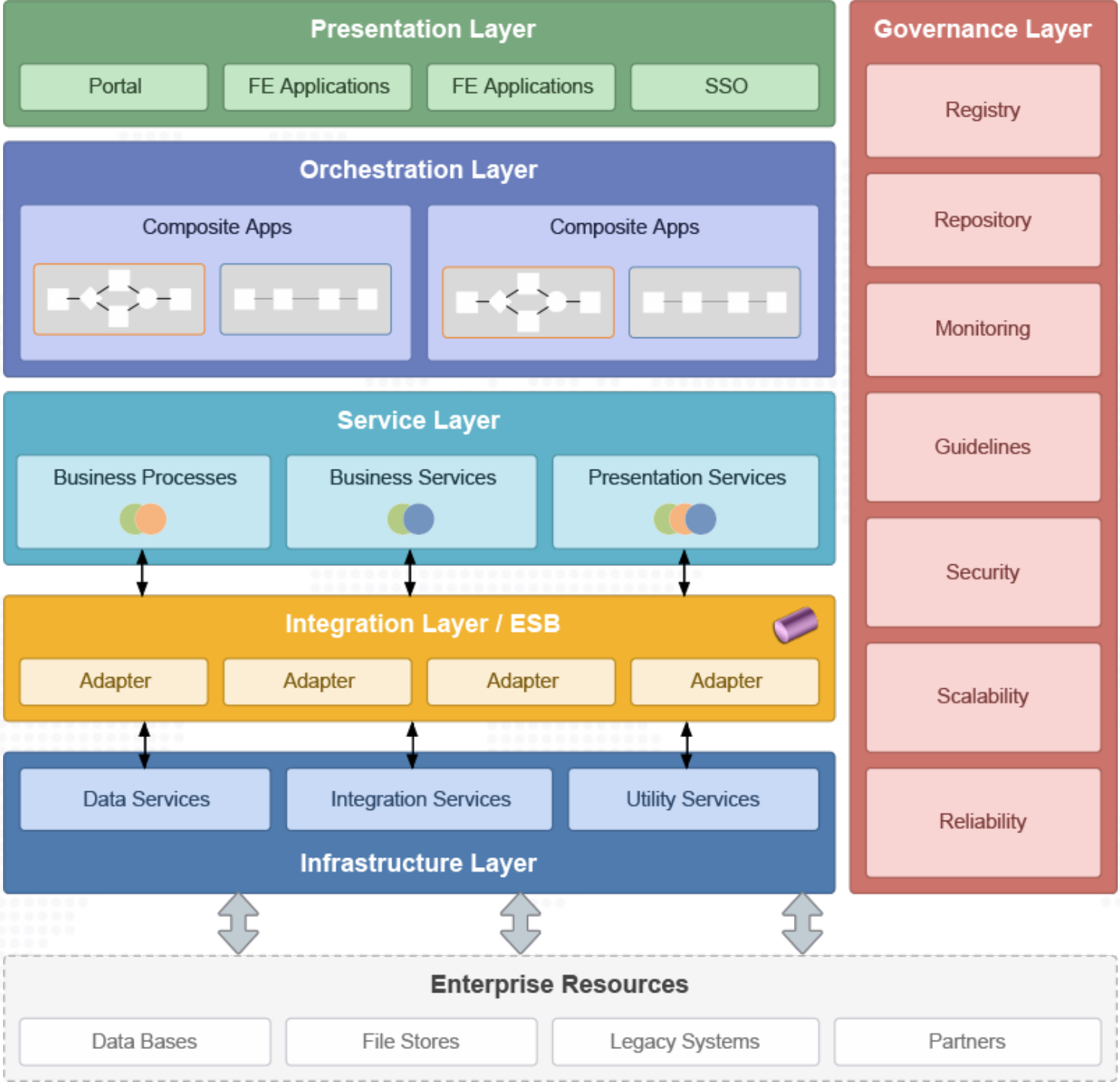


三层架构
(集中式架构)



DDD分层架构
(分布式微服务架构)

SOA 架构



	优势	劣势
单体	• 人所众知: 传统工具、应用和脚本都是这种结构 • IDE友好: Eclipse、IntelliJ等开发工具多 • 便于共享: 单个归档文件包含所有功能, 便于共享 • 易于测试: 单体应用部署后, 服务或特性即可展现, 没有额外的依赖, 测试可以立刻开始 • 容易部署: 只需将单个归档文件复制到单个目录下	• 不够灵活: 任何细修改需要将整个应用重新构建/部署, 这降低了团队的灵活性和功能交付频率 • 妨碍持续交付: 单体应用比较大时, 构建时间比较长, 不利于频繁部署, 阻碍持续交付。 • 受技术栈限制: 必须使用同一语言/工具、存储及消息, 无法根据具体的场景做出其它选择 • 技术债务: “不坏不修 (Not broken, don't fix) ”, 系统设计/代码难以修改, 偶合性高。
SOA	• 服务重用性: 通过编排你基本服务以用于不同的场景 • 易维护性: 单个服务的规模变小, 维护相对容易 • 高可靠性: 使用消息机制及异步机制, 提高了可靠性 • 高扩展和可用性: 分布式系统的特性 • 软件质量提升: 单个服务的复杂度降低 • 平台无关: 可以集成不同的系统 • 提升效率: 服务重用、降低复杂性, 提升了开发效率	• 过分使用ESB: 使得系统集成过于复杂 • 使用基于SOAP协议的WS: 使得通信的额外开销很大 • 使用形式化的方式管理: 增加了服务管理的复杂度 • 需要使用可靠的ESB: 初始投资比较高
微服务	• 简单: 单个服务简单, 只关注于一个业务功能。 • 团队独立性: 每个微服务可以由不同的团队独立开发。 • 松耦合: 微服务是松散耦合的。 • 平台无关性: 微服务可以用不同的语言/工具开发。 • 通信协议轻量级: 使用REST或者RPC进行服务间通信	• 运维成本过高 • 分布式系统的复杂性 • 异步, 消息与并行方式使得系统的开发门槛增加 • 分布式系统的复杂性也会让系统的测试变得复杂

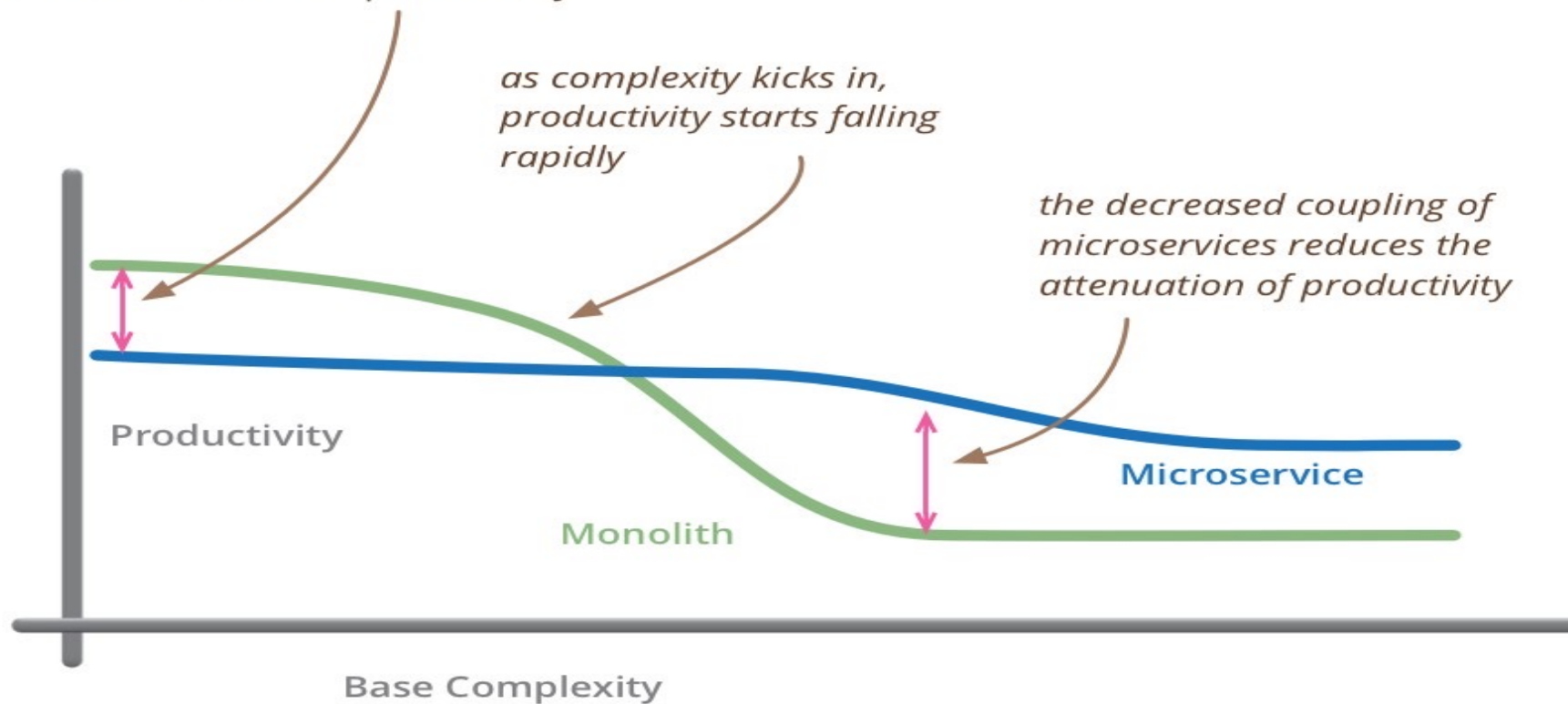
什么时候才用微服务架构？

产品初期应该优先使用单体架构。面对一个新的领域，对业务的理解是一个循序渐进的过程，而微服务架构与业务架构紧密相关；另外一方面，微服务架构对团队能力和协作要求都相对较高，同时微服务架构对 DevOps 和敏捷基础架构也有较高的的要求，而初期往往资源 – 人和机器，都有限，因此很难用好微服务架构。

当**业务复杂度**、**用户量**和**业务量**增长到一定时候，微服务架构的投入产出比才会明显。在服务拆分之前，应该保证 DevOps 和敏捷的基础架构已经就绪。同时团队规模和业务复杂度已经到达了使用微服务架构的要求。

微服务的 ROI

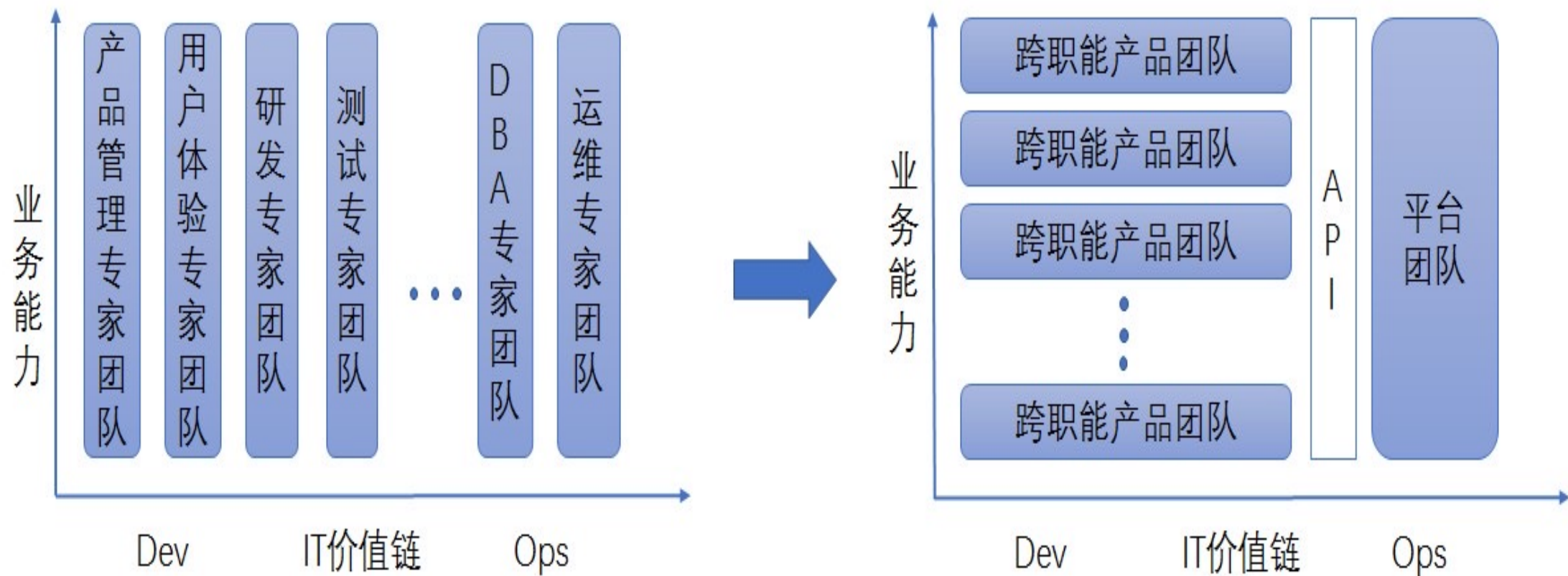
for less-complex systems, the extra baggage required to manage microservices reduces productivity



but remember the skill of the team will outweigh any monolith/microservice choice

Organizations which design systems ... are constrained to produce designs which are copies of the communication structures of these organizations.

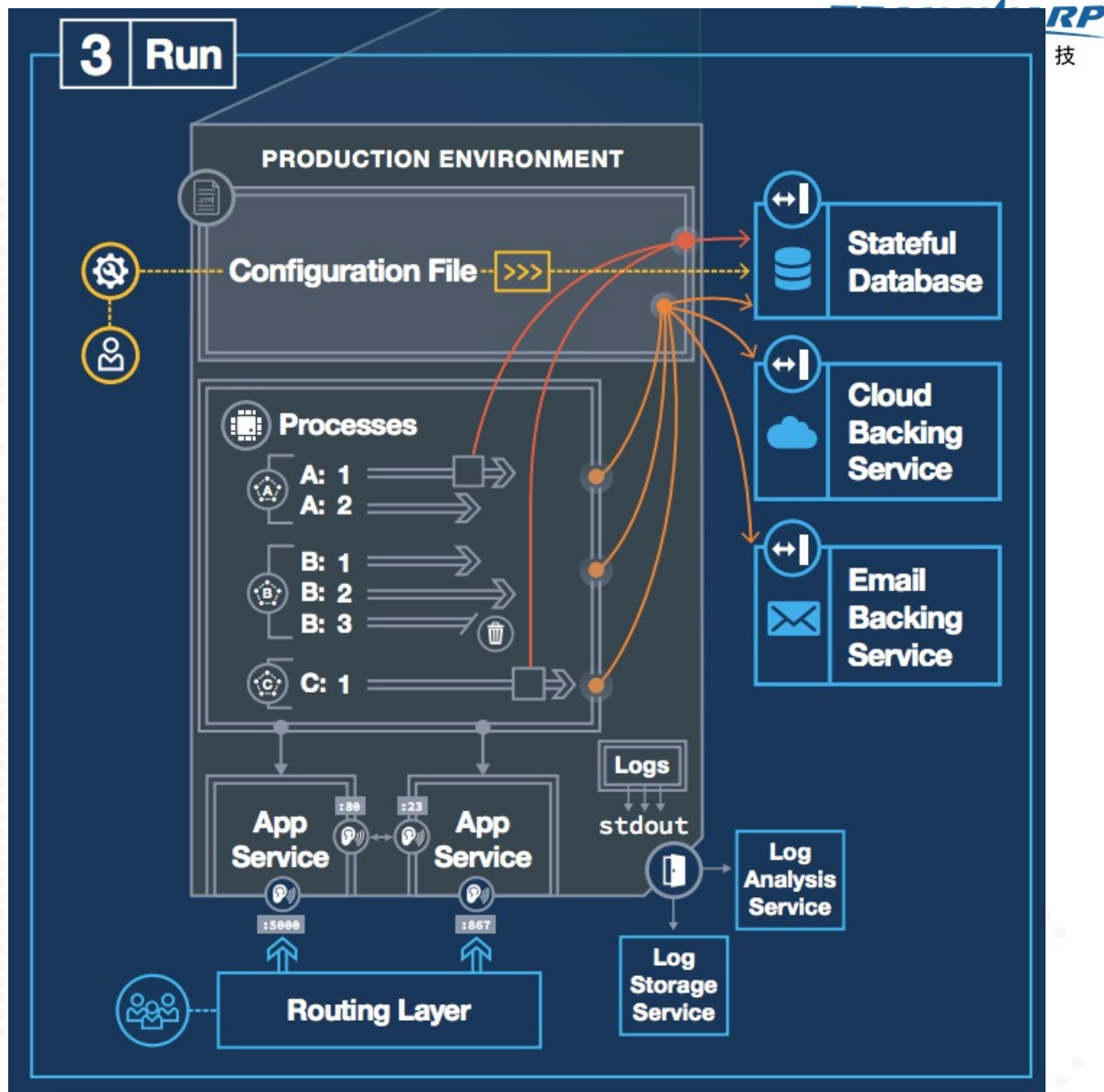
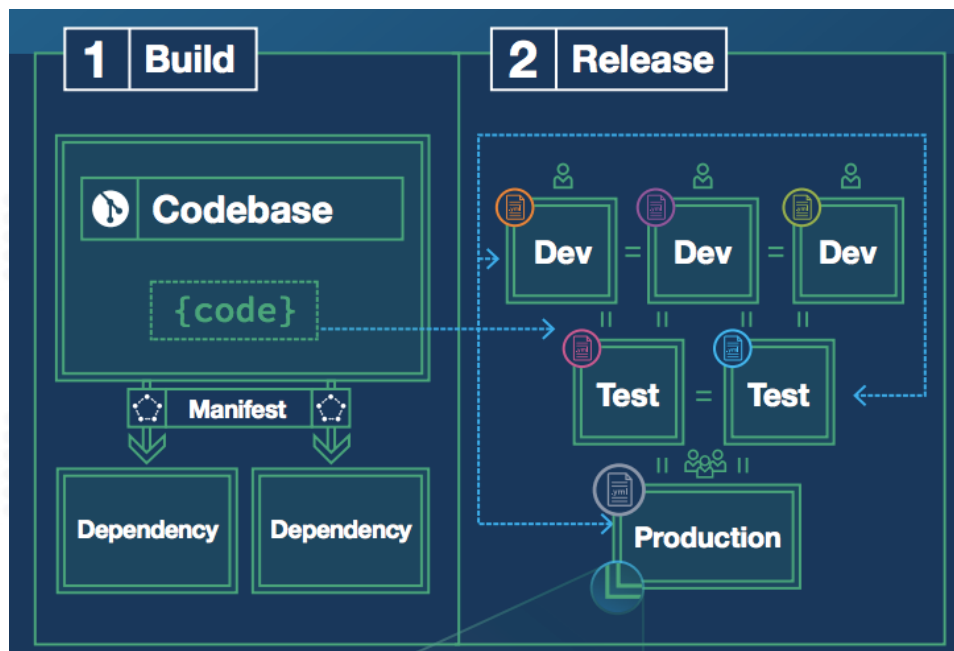
设计系统的组织其产生的设计和架构---等价于---组织架构

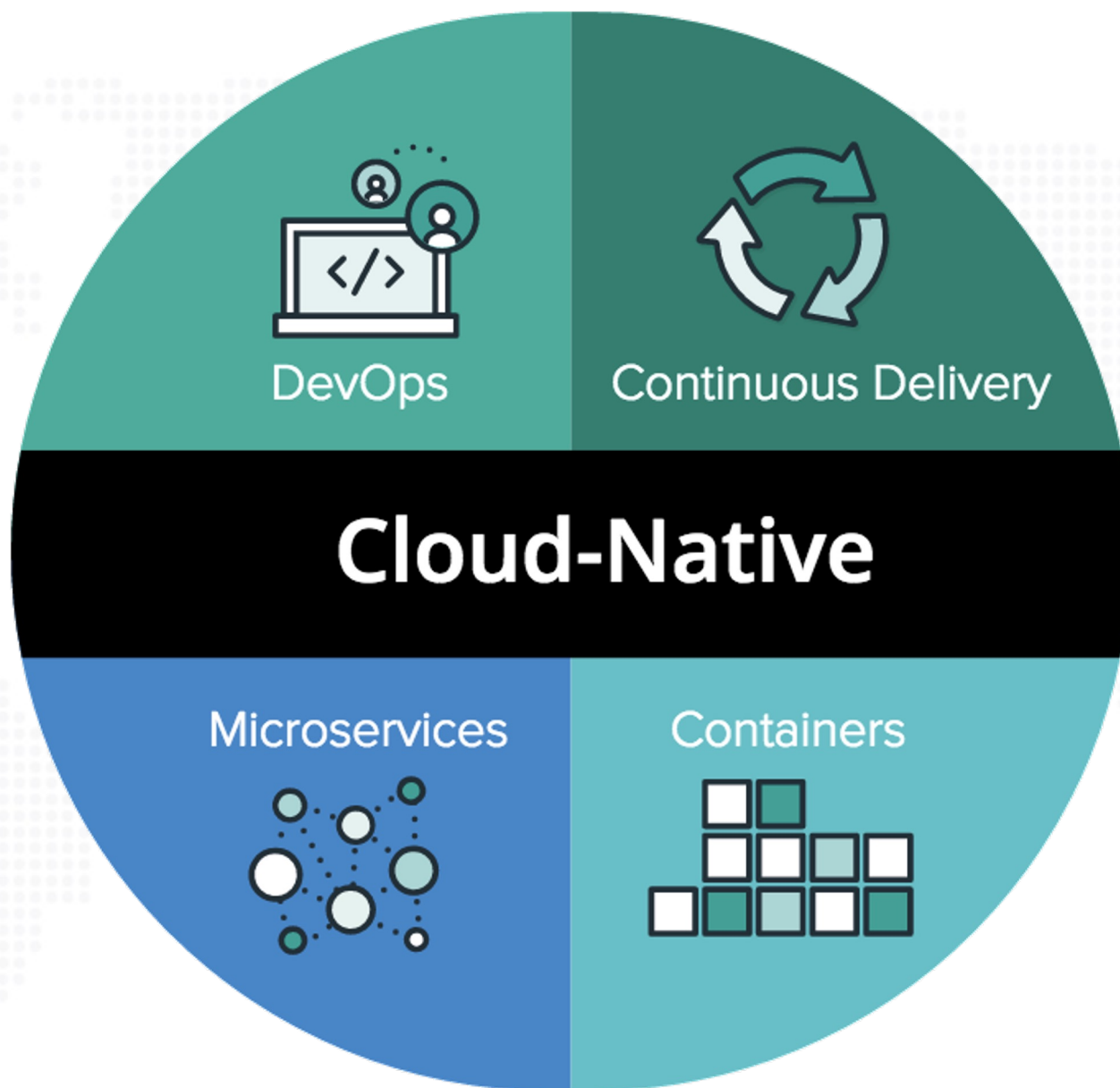


12 范式与 DevOps

十二范式 (The Twelve-factor App)

1. **基准代码**：一份基准代码，多份部署。基准代码和应用之间总是保持一一对应的关系。所有部署的基准代码相同，但每份部署可以使用其不同的版本。
2. **依赖**：显式声明依赖关系。应用程序一定通过依赖清单，确切地声明所有依赖项。
3. **配置**：在环境中存储配置。将应用的配置存储于环境变量中。环境变量可以非常方便地在不同的部署间做修改，却不动一行代码。
4. **后端服务**：把后端服务当作附加资源。应用不会区别对待本地或第三方服务。对应用程序而言，两种都是附加资源。
5. **构建，发布，运行**：严格区分构建，发布，运行这三个步骤。
6. **进程**：以一个或多个无状态进程运行应用。应用的进程必须无状态且无共享。
7. **端口绑定**：通过端口绑定提供服务。应用完全自我加载而不依赖任何网络服务器就可以创建一个面向网络的服务。
8. **并发**：通过进程模型进行扩展。开发人员可以运用这个模型去设计应用架构，将不同工作分配给不同的进程类型。
9. **易处理**：快速启动和优雅终止可最大化健壮性。应用的进程是可支配的，意思是说它们可以瞬间开启或停止。
10. **开发环境与线上环境等价**：尽可能保持开发、预发布、线上环境相同。应用想要做到持续部署就必须缩小本地与线上差异。
11. **日志**：把日志当作事件流。应用本身考虑存储自己的输出流。不应该试图去写或者管理日志文件。
12. **管理进程**：后台管理任务当作一次性进程运行。一次性管理进程应该和正常的常驻进程使用同样的环境。





DEVOPS

BEFORE

Not my problem

Separate tools, varied incentives, opaque process



AFTER

Shared responsibility

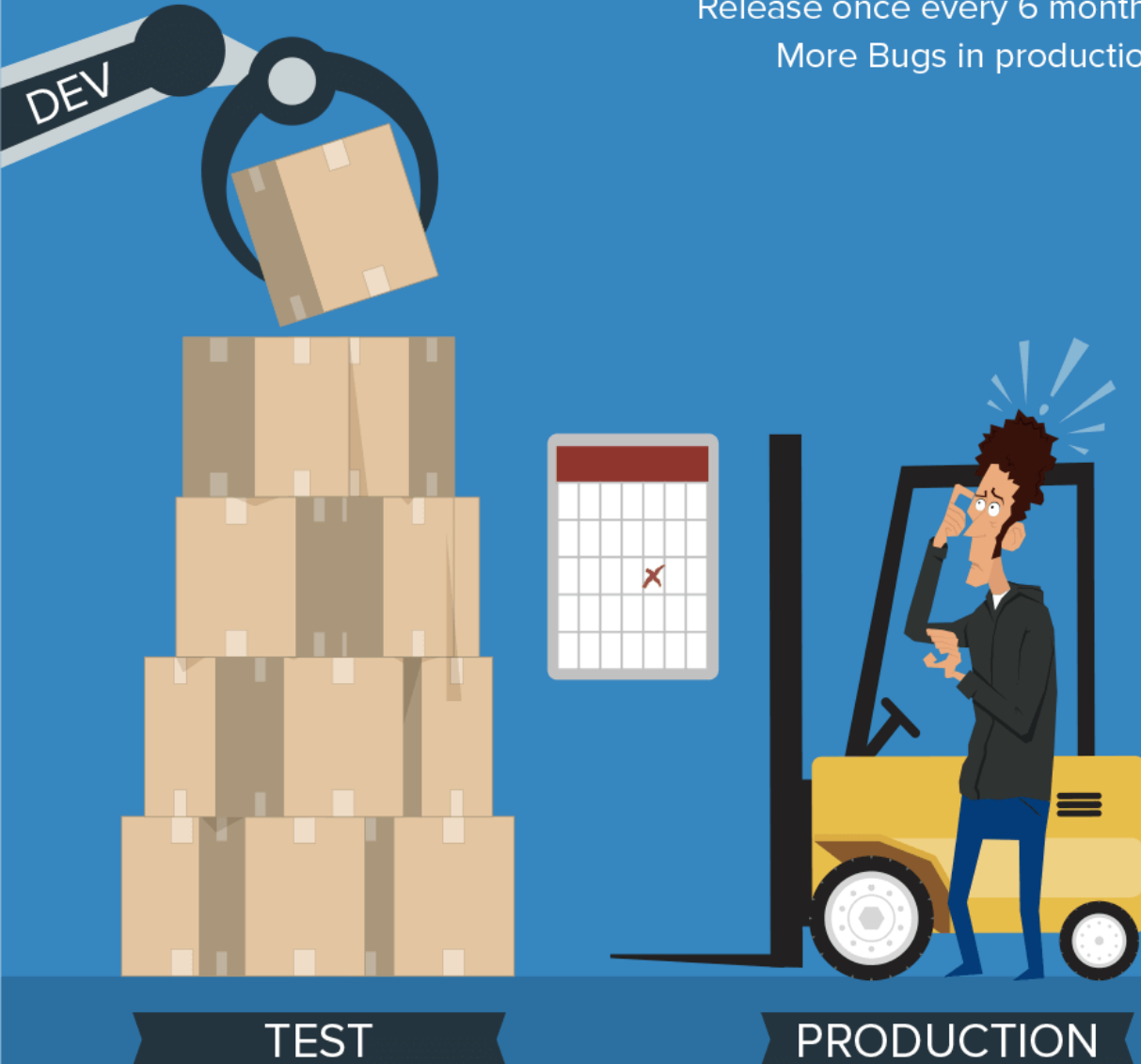
Common incentives, tools, process and culture



CONTINUOUS DELIVERY

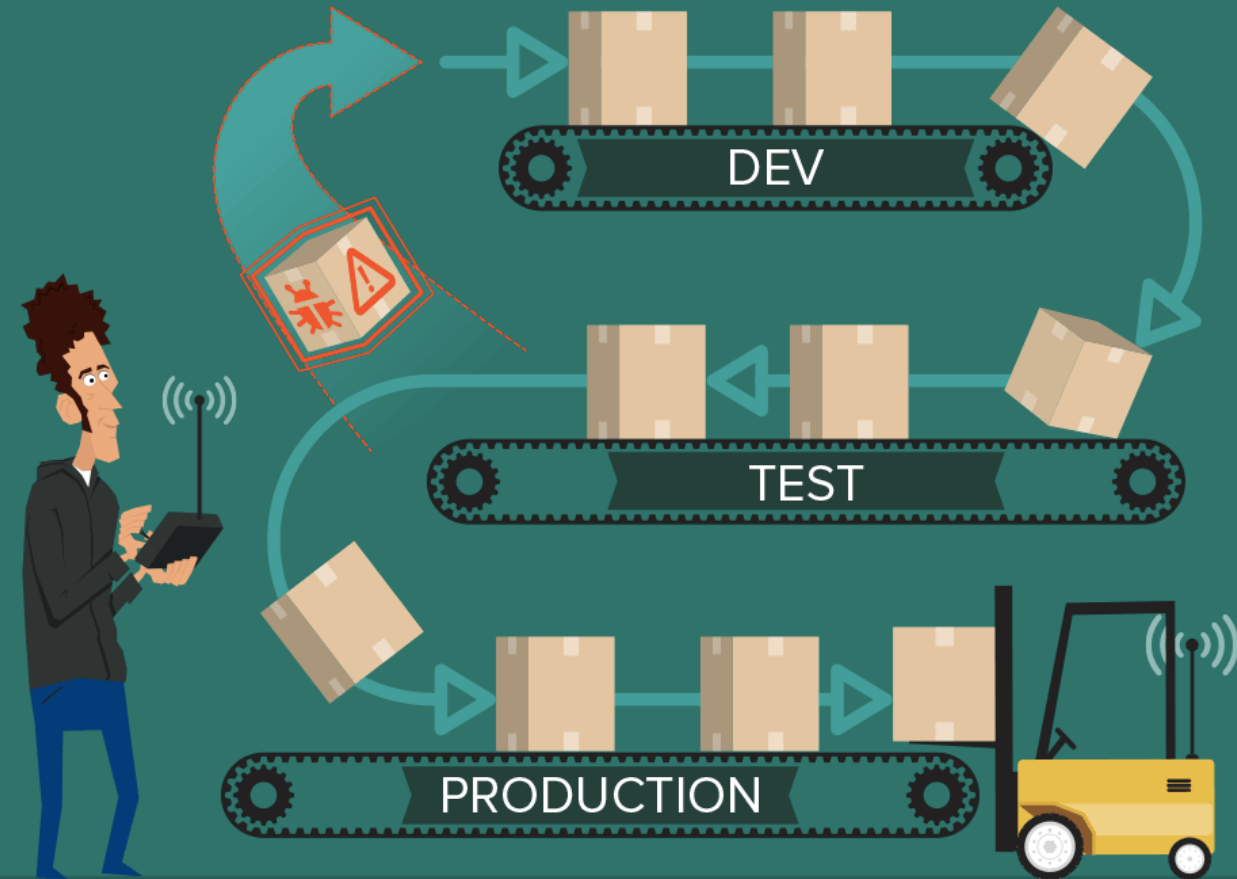
BEFORE

Release once every 6 months
More Bugs in production



AFTER

Release once a week
Higher Quality of Code



MICROSERVICES

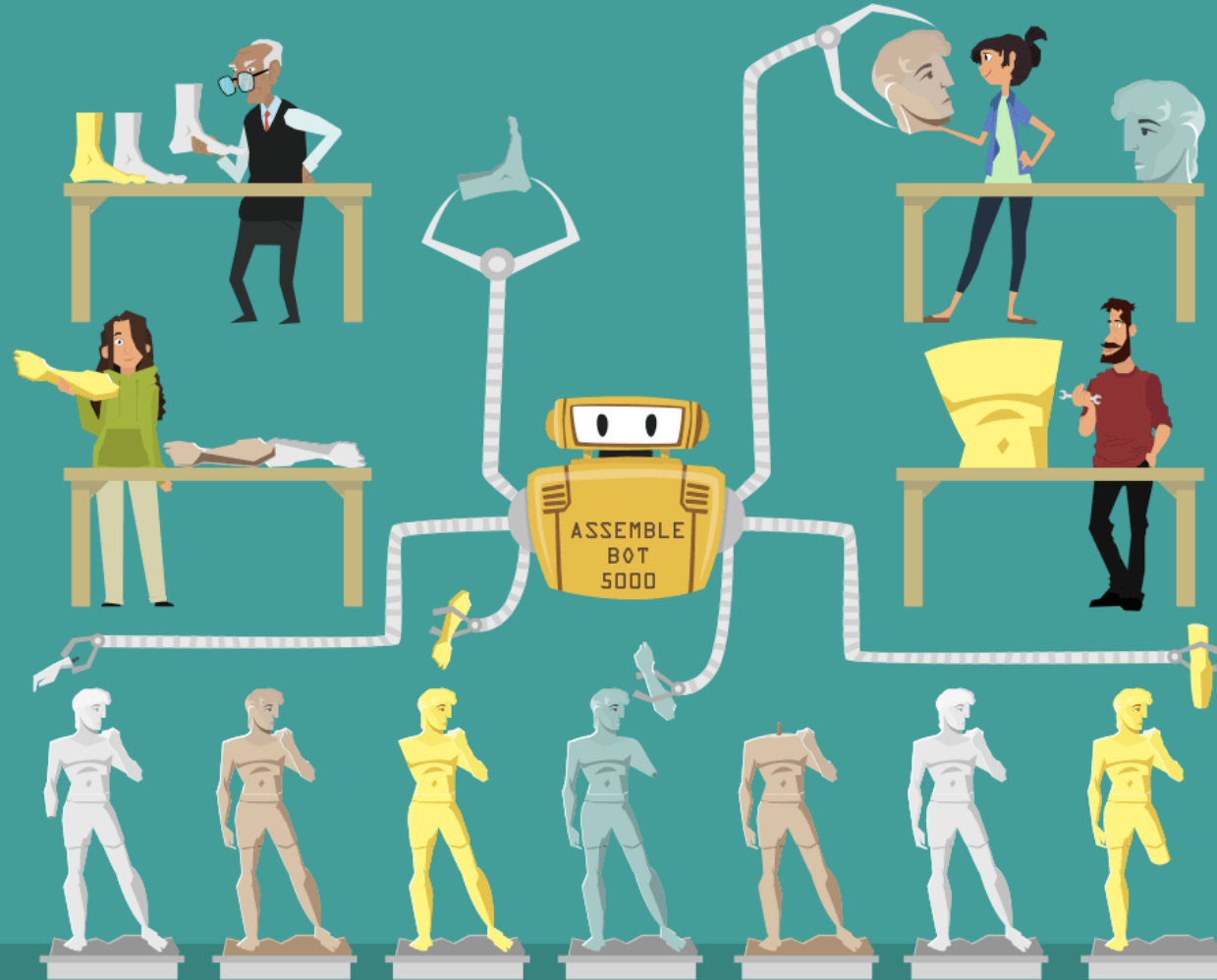
BEFORE

Tightly coupled components
Slow deploy cycles waiting on integrated tests and teams



AFTER

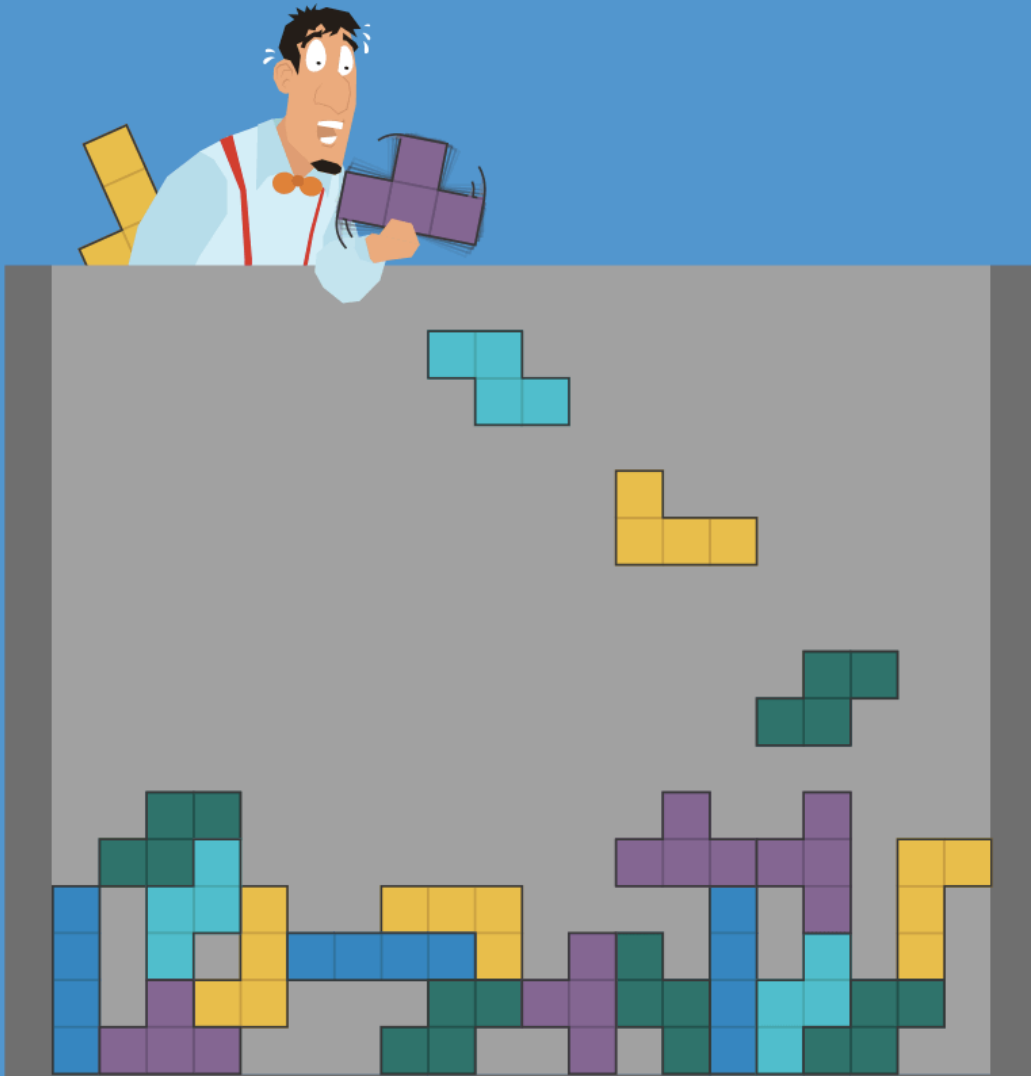
Loosely coupled components
Automated deploy without waiting on individual components



CONTAINERS

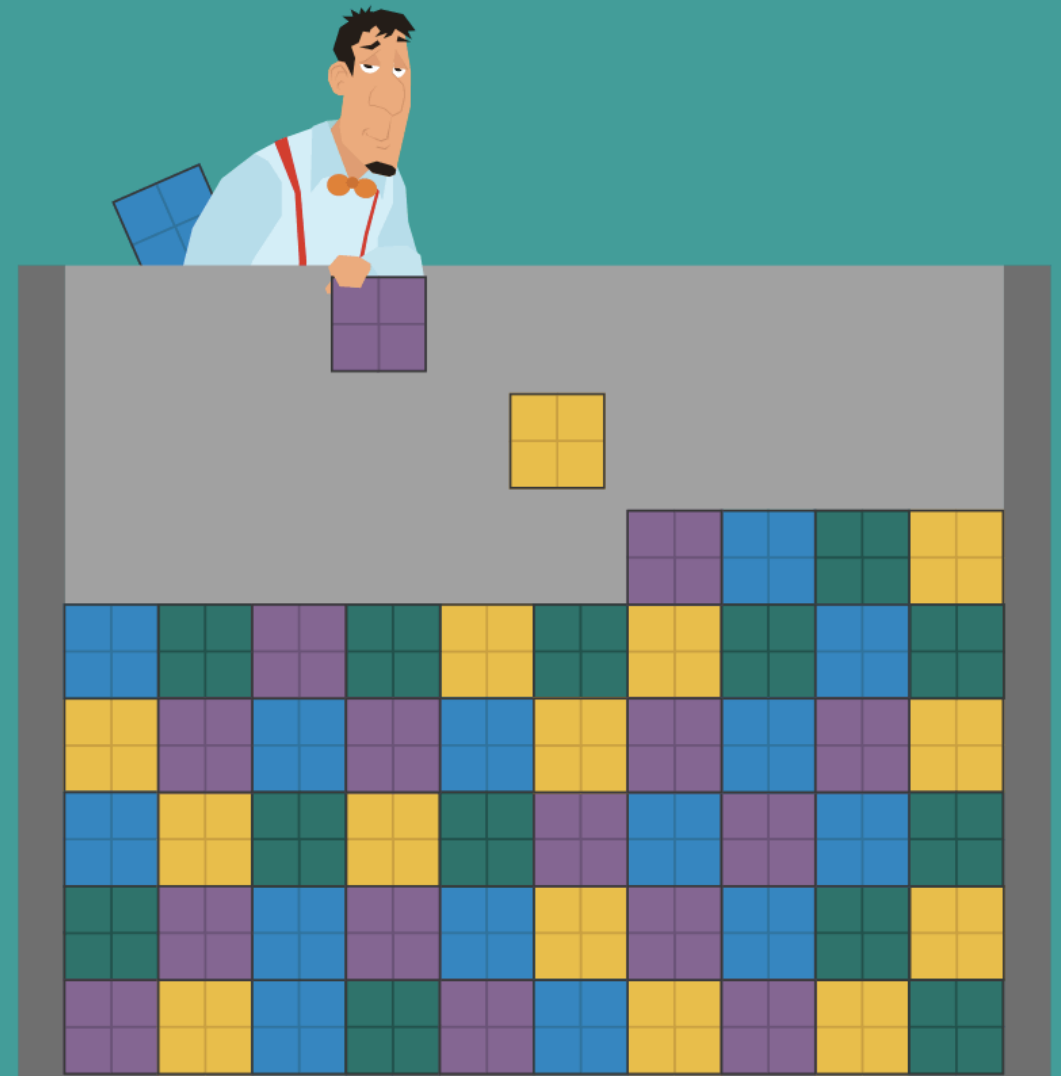
BEFORE

Make sure software matches with hardware
Build snowflakes for each environment



AFTER

Build once, run anywhere
A shared construct between software and hardware



微服务原则

全局视角

站在企业整个技术发展的角度进行设计

业务驱动

根据业务需求设计领域模型，推导业务感知的架构设计

微服务设计

根据领域模式设计微服务系统，并抽象微服务平台组件

技术服务业务

根据业务需要，合理选择技术栈和解决方案

Two Pizza Rule = Better Productivity



另外一方面，微服务代码规模需要控制，需要根据持续交付速度的要求进行调整，包括持续集成（构建，单元测试，代码扫描），自动化测试，持续部署等





01 垂直划分原则

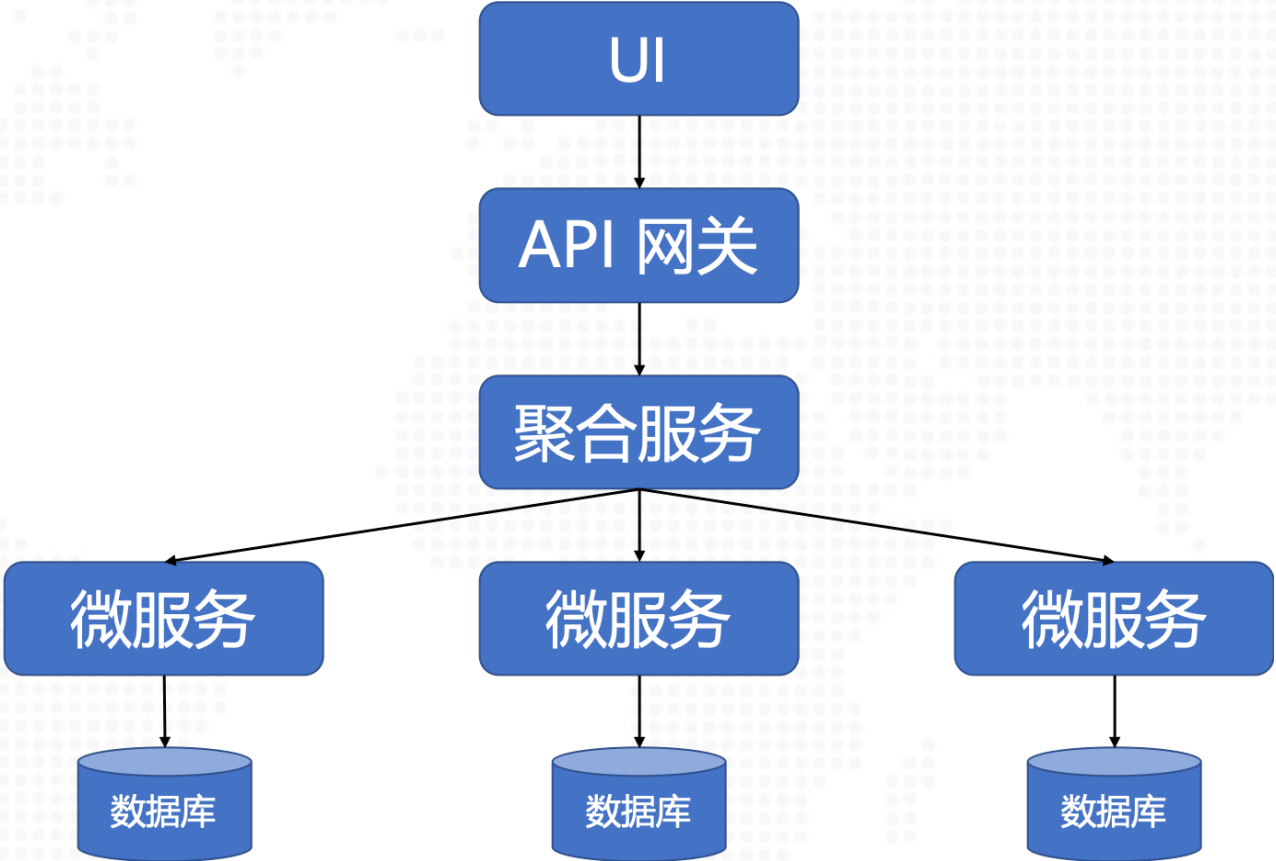
02 持续演进原则

03 服务自治原则

04 自动化驱动原则



垂直划分原则



服务数量快速增长带来架构复杂度急剧升高，开发、测试、运维等环节很难快速适应，会导致故障率大幅增加，可用性降低，非必要情况，应逐步划分，持续演进，避免服务数量的爆炸性增长，这等同于灰度发布的效果，先拿出几个不太重要的功能拆分出一个服务做试验，如果出现故障，则可以减少故障的影响范围。

另外，除了业务服务数量的增加，还需要准备持续交付的工具、微服务框架等，加强可观测性。

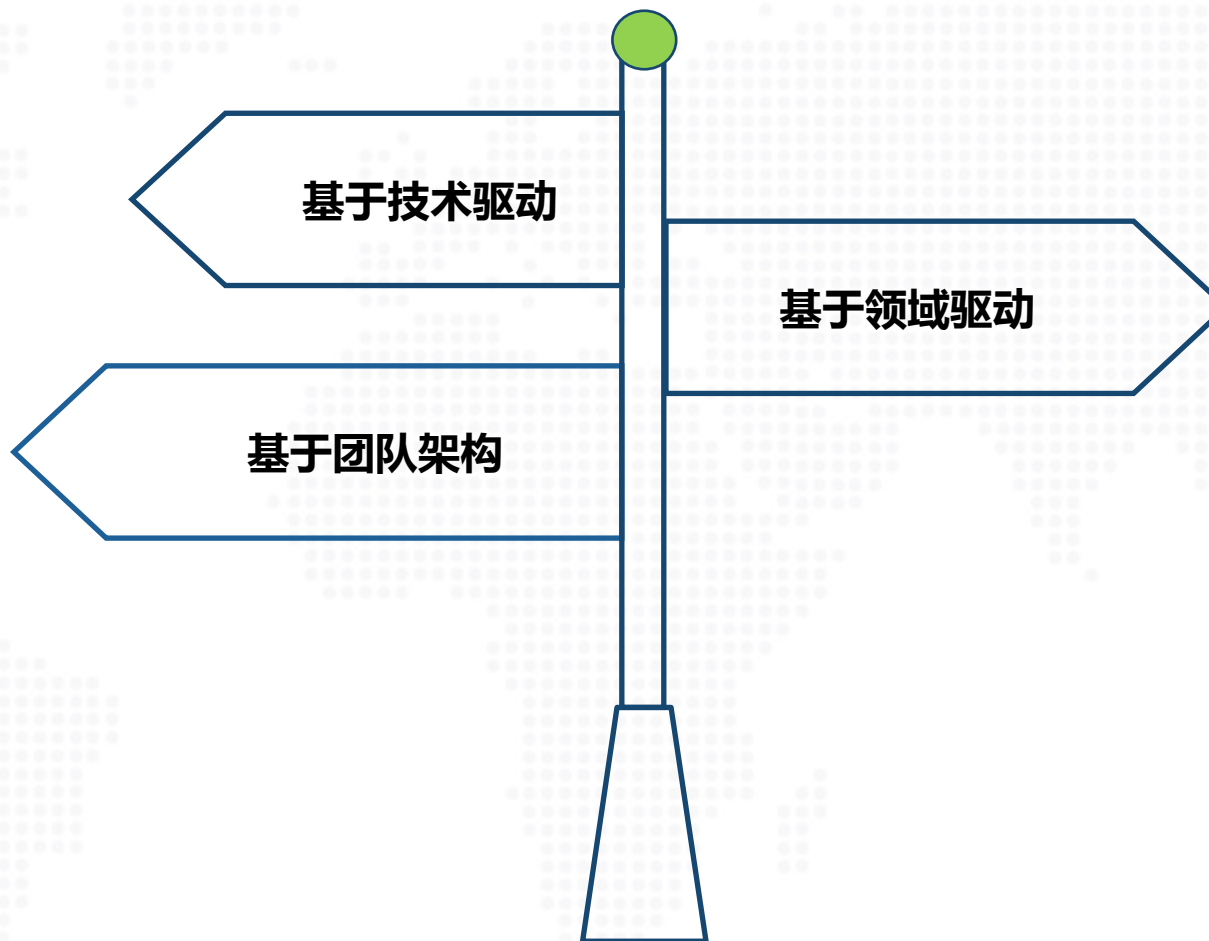
尽量消除对其他服务的强依赖，这样可以降低沟通成本，提升服务稳定性。服务通过标准的接口隔离，隐藏内部实现细节。这使得服务可以独立开发、测试、部署、运行，以服务为单位持续交付。直接访问对方的数据库会造成一定的耦合性，应该尽量避免。

部署与运维的成本会随着服务的增多呈指数级增长，每个服务都需要部署、监控、日志分析等运维工作，成本会显著提升。在服务划分之前，应该首先构建自动化的工具及环境。开发人员应该以自动化为驱动力，简化服务在创建、开发、测试、部署、运维上的重复性工作，通过工具实现更可靠的操作。避免微服务数量增多带来的开发、管理复杂度问题。自动化可以从多个方面节省时间、提升效率，它可以快速跟踪整个交付过程并实时向所有参与者报告这个过程，赋予参与者责任感和成就感，如研发过程中，推行持续集成的文化就特别重要，而持续集成所依赖的工具就是一种自动化的体现。

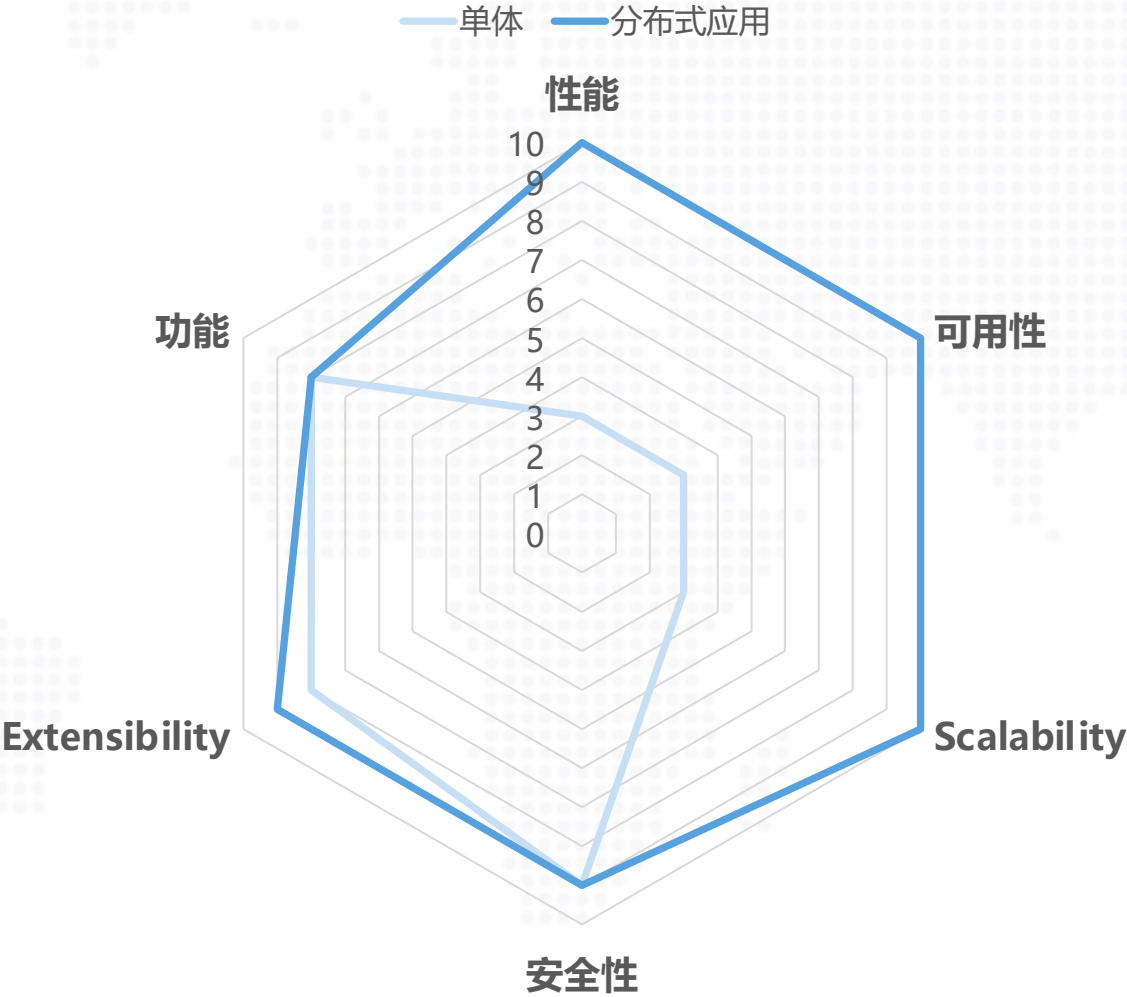
很多互联网公司都遵循「**一切皆自动化**」的原则，特别是存在跨地域的研发模式时，使用自动化工具将是至关重要的。

微服务拆分原则

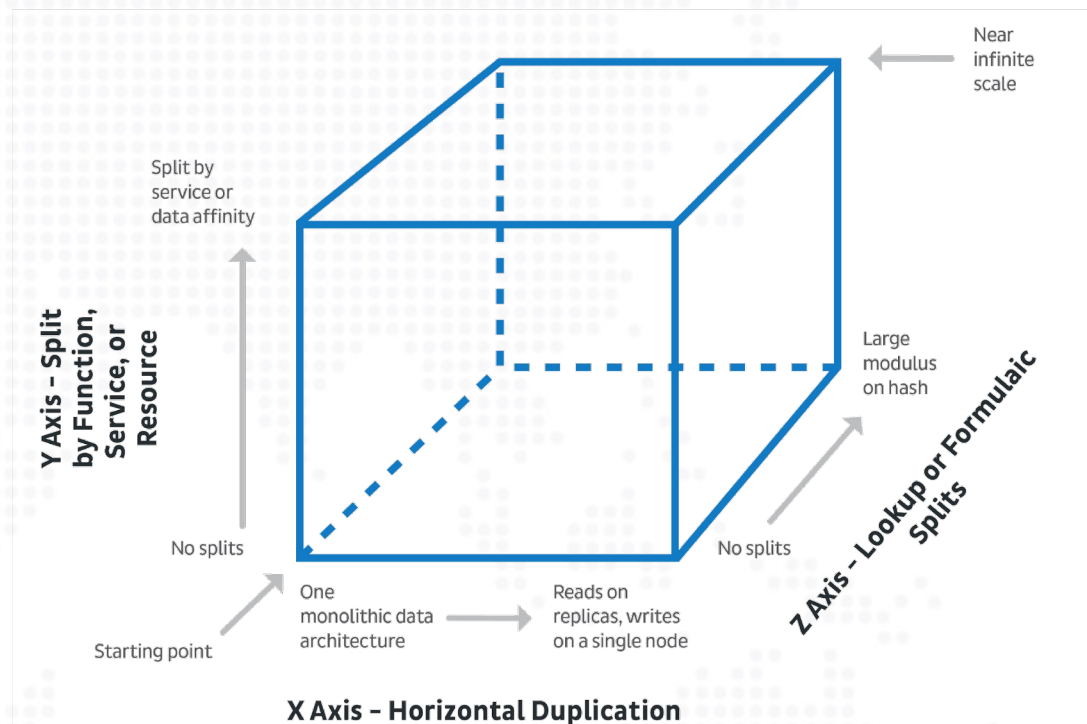
微服务拆分原则



单体 vs. 分布式



- AKF扩展立方体，《架构即未来》一书中提出的可扩展模型。理论上按照这三个扩展模式，可以将一个单体系统，进行无限扩展。
- X 轴：是水平复制。比如讲单体系统多运行几个实例，做个集群加负载均衡的模式。
- Z 轴：是数据分区，比如按照用户请求的地区进行数据分区，北京、上海、四川等多建几个集群。
- Y 轴：是微服务的拆分模式，就是基于不同的业务进行拆分。



无状态化

定时任务

本地存储

本地缓存

去触发器、 存储过程

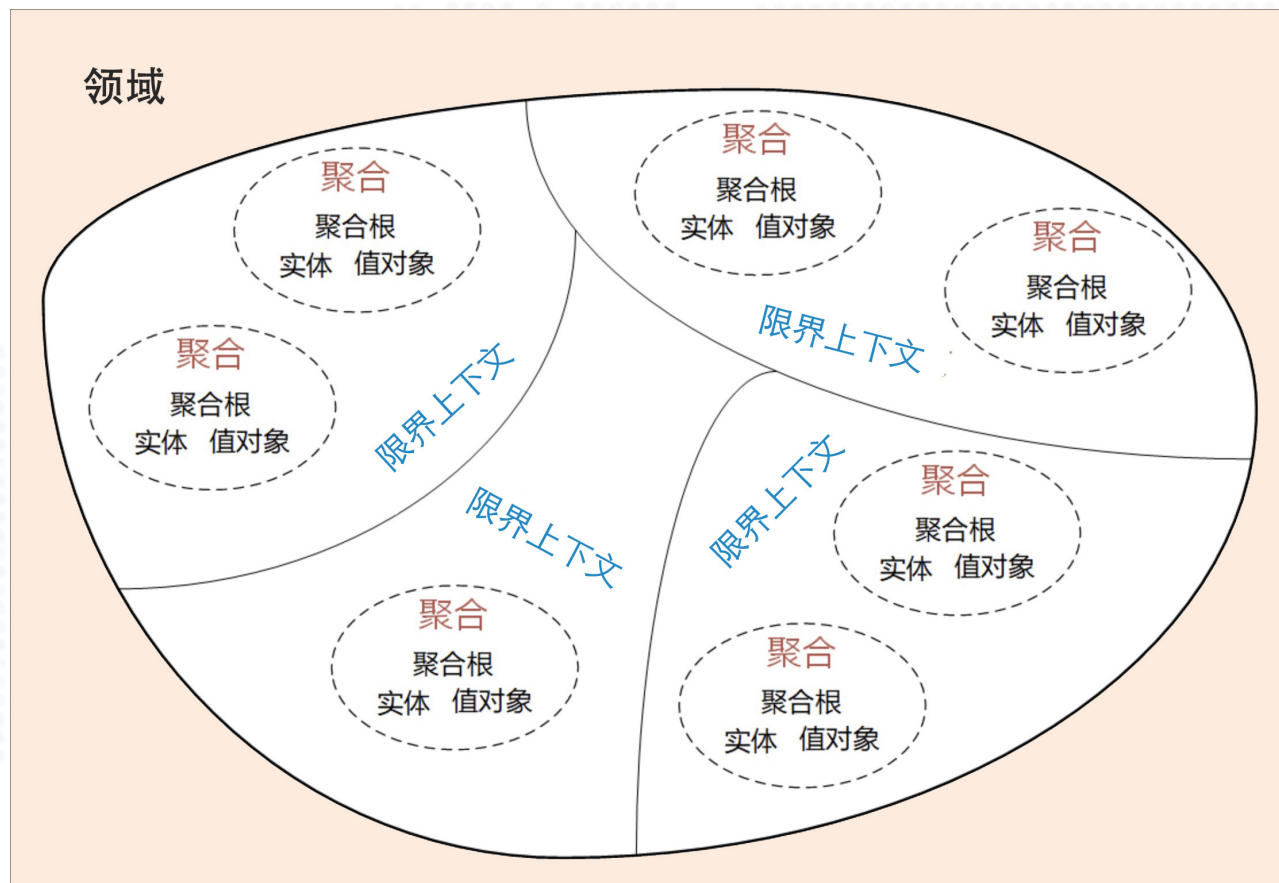
整体的伸缩受到数据库的限制，因为触发器、存储过程难以扩展。

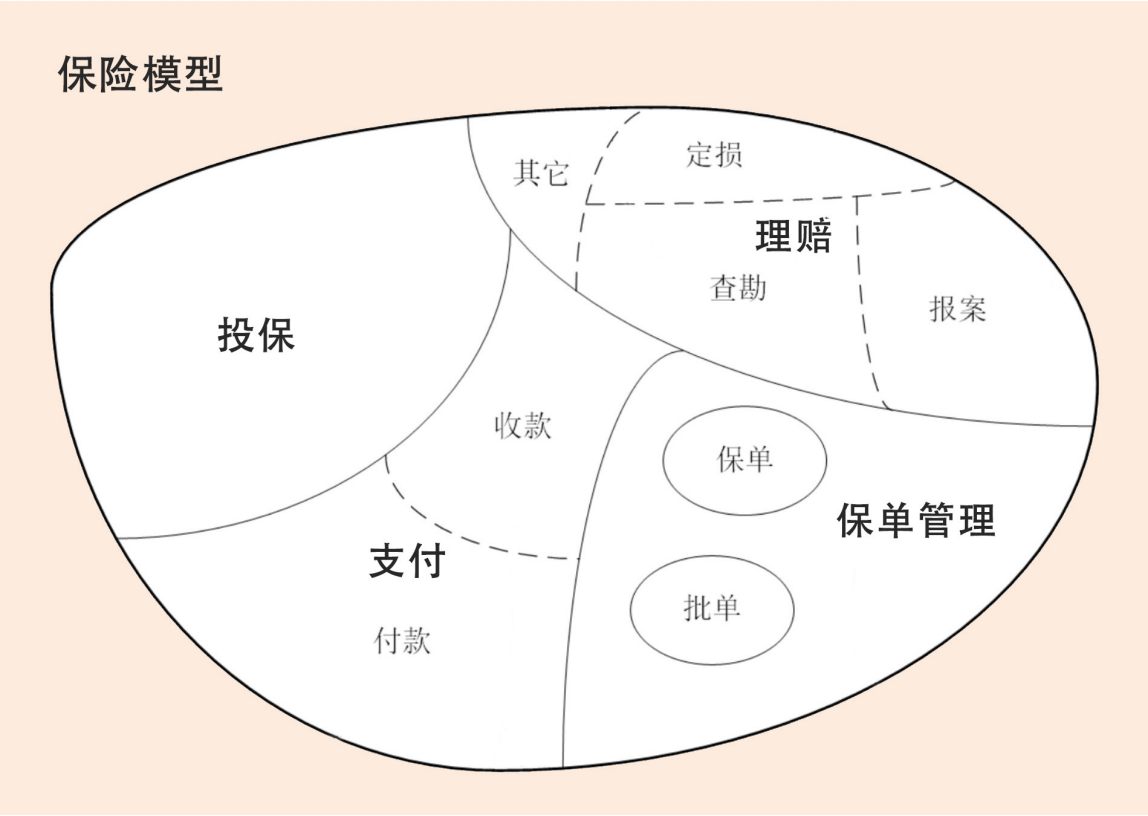
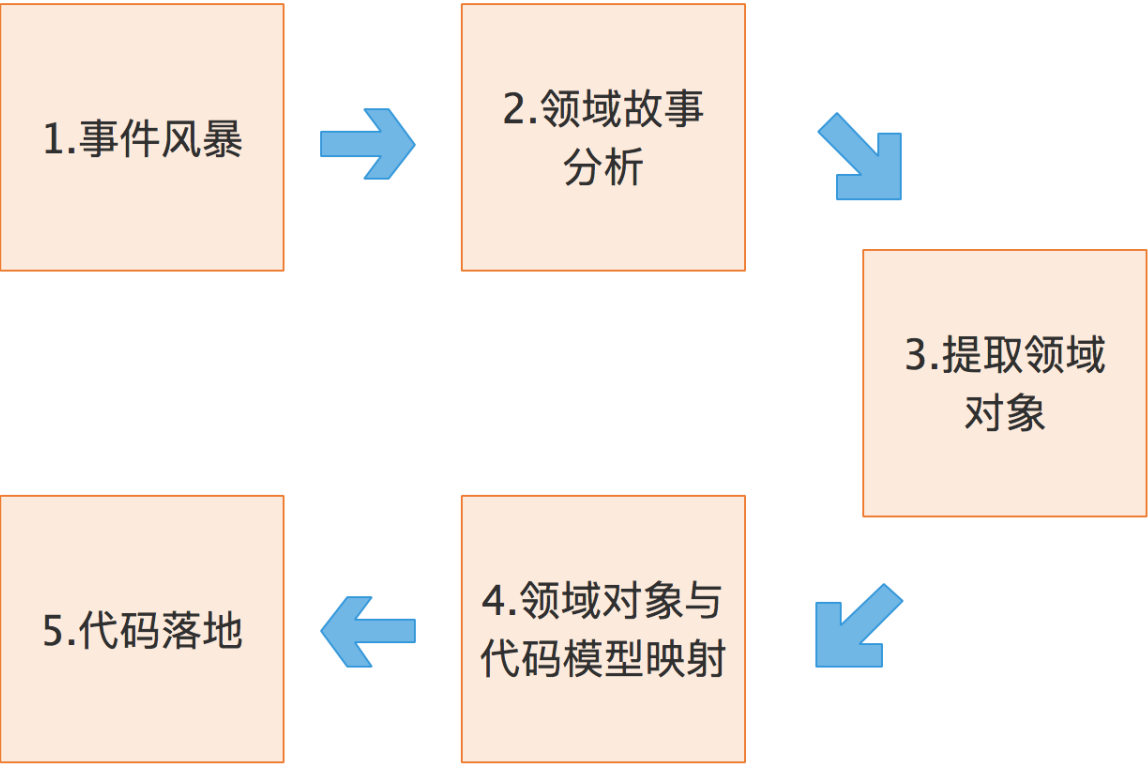
当存在水平分表的时候，可能无法满足需求。

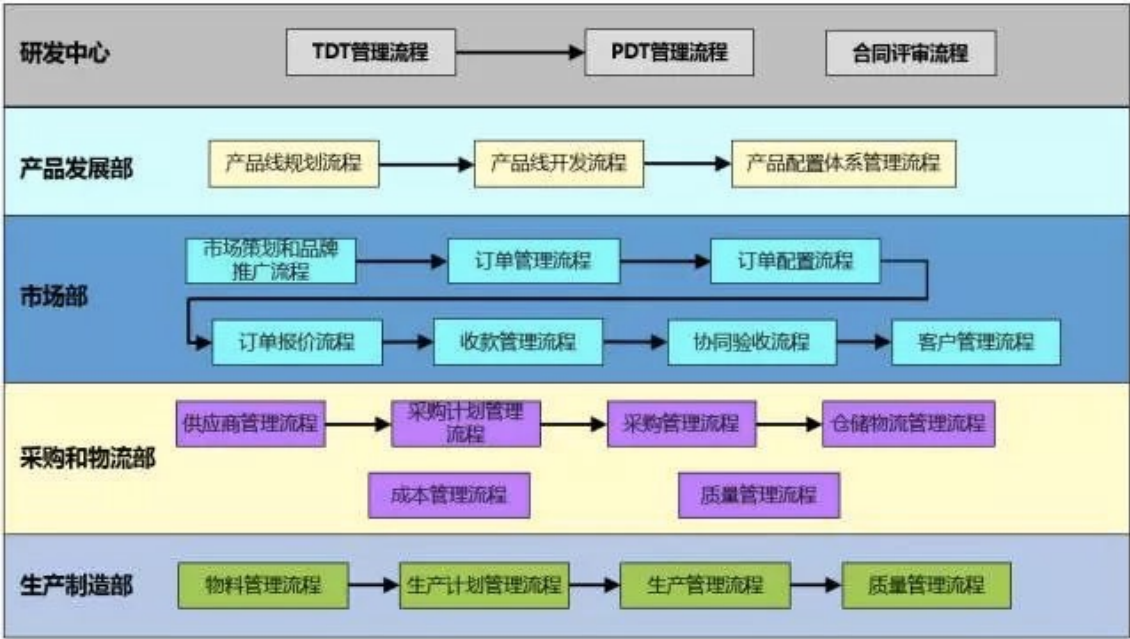
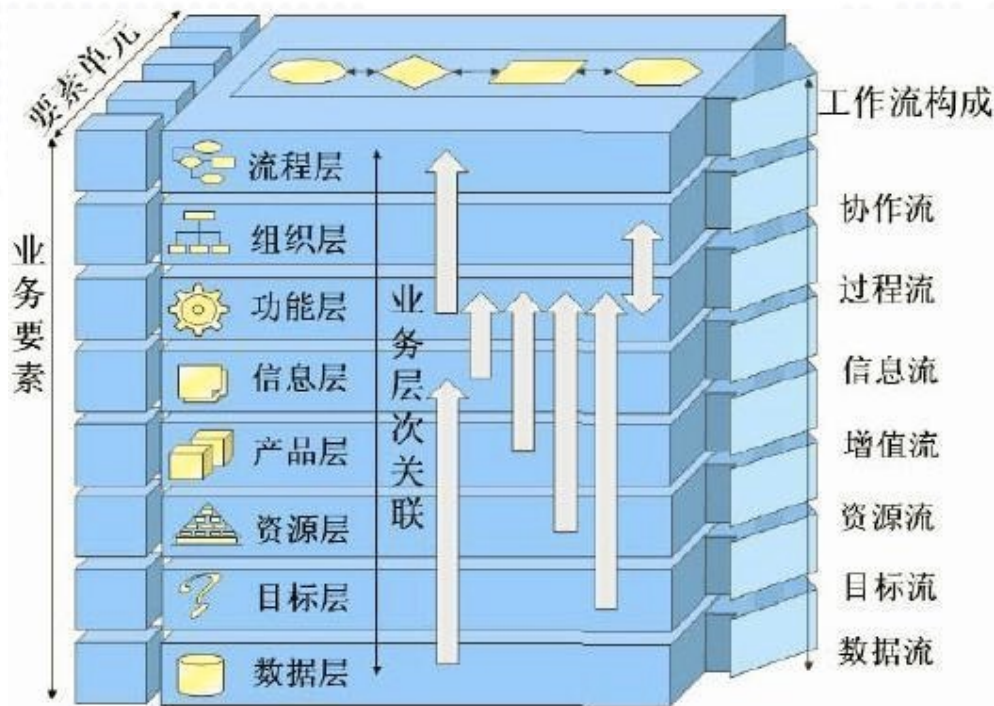
如果触发器、存储过程过多，则会导致运维复杂度升高。

- 单体程序由于技术栈固定，尤其是比较庞大的系统，不能很方便的进行技术升级，或者说对引入新技术或框架等处于封闭状态。每种语言都有自己的特点，单体程序没有办法享受到其它语言带来的便利。
- 相比于基于业务的垂直拆分，基于技术的横向拆分也很重要，使用数据访问层可以很好的隐藏对数据库的直接访问、减少数据库连接数、增加数据使用效率等；横向拆分可以极大的提高各个层级模块的重用性。

领域驱动是一个自上而下的架构设计方法，通过和领域专家建立统一的语言，不断交流，确定关键业务场景，逐步确定边界上下文。领域驱动更强调业务实现效果，认为自下而上的设计可能会导致技术人员不能更好地理解业务方向，偏离业务目标。





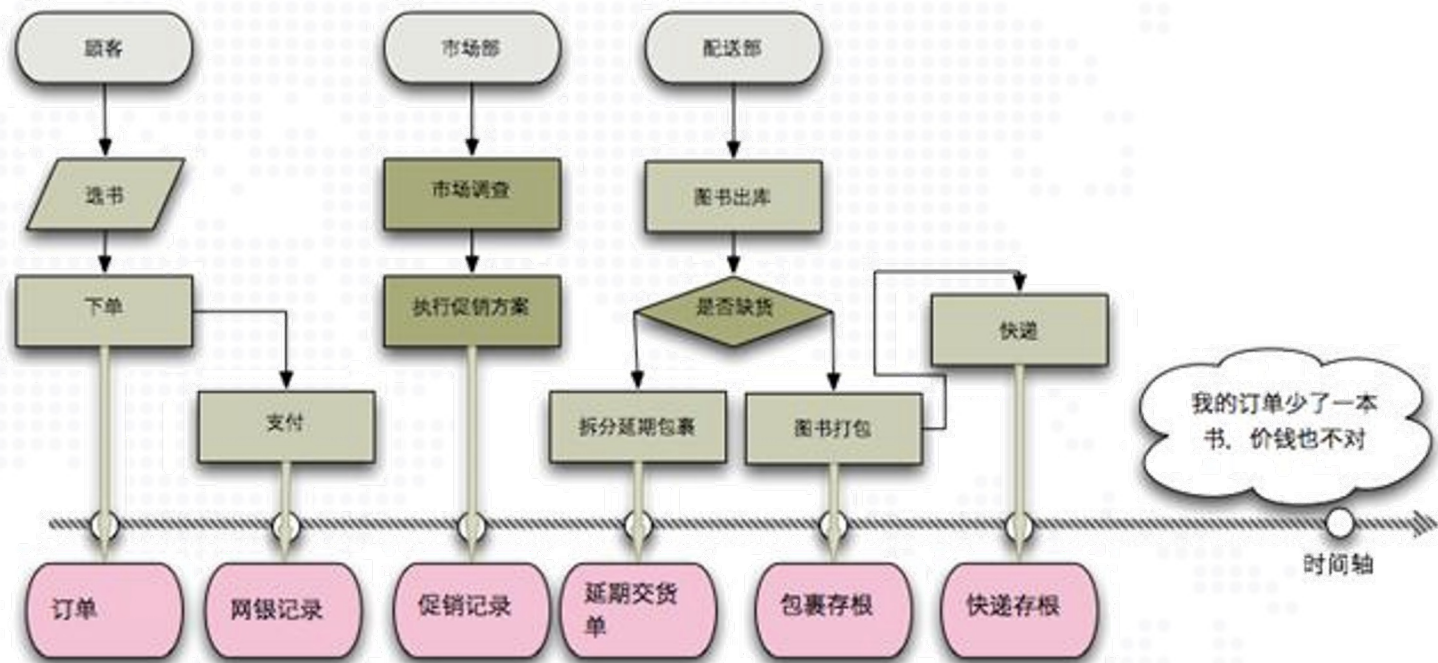


业务模型，组织架构

业务流程

可以基于用户故事/用例等识别出

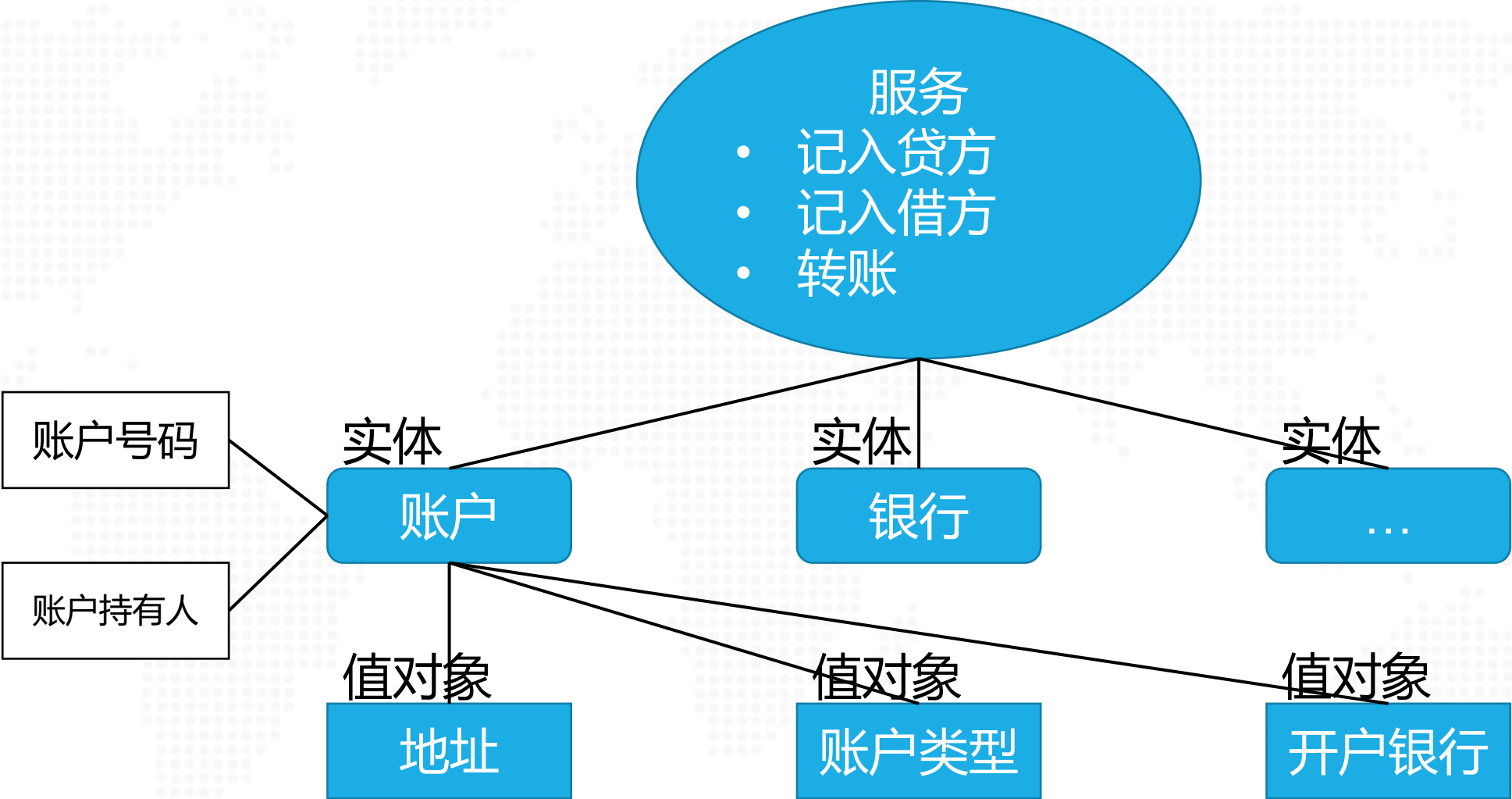
- 角色
- 业务操作
- 对象



领域模型是问题领域不同实体之间关系的蓝图以及其他一些重要的细节，包括：

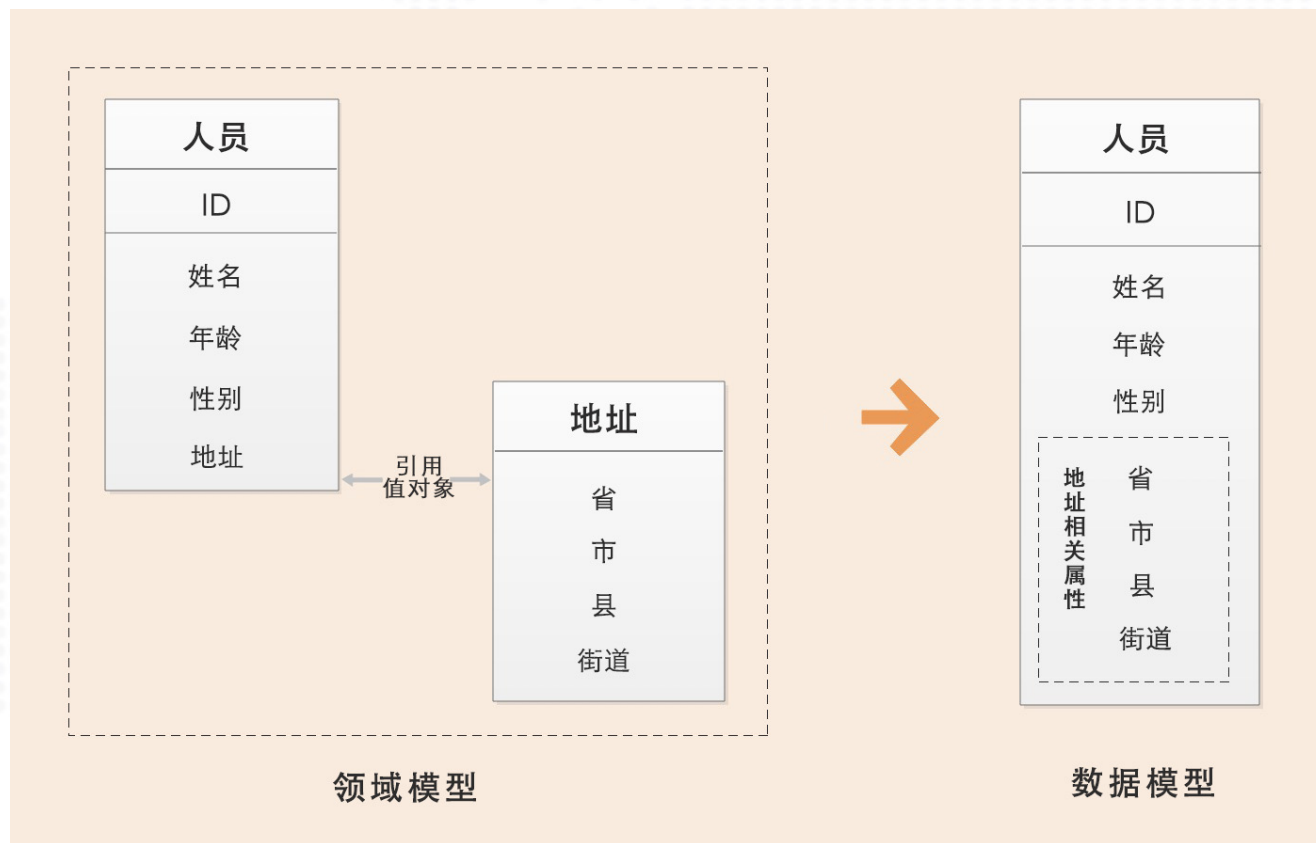
- **领域对象**：例如在银行领域中，会有诸如银行、账号、交易事务等对象
- 对象之间互动展现的**行为**：比如，在银行系统账户的借方中记录一笔款项
- **领域语言**：对个人银行建模时，诸如记入借方、信用、投资组合等术语
- 模型操作内的**上下文**：其中包含了问题领域相关的假设与约束。

元素	特性
实体	有一个ID 在生命周期里专递多种状态 在业务中通常有明确的生命周期
值对象	不可变 能在实体间自由共享
服务	通常对一个业务的用例建模 涵盖多个实体和值对象 对外提供对模型（实体/值对象）的操作



1. 基于业务分析识别领域上下文
2. 在领域上下文语境中分析业务流程及子流程
3. 识别每个业务流程中的「角色」，「操作」和「资源」
4. 基于流程分析定义领域模型
5. 定义领域对象、实体以及值对象

- 由业务流程梳理可划分出如下领域，一个领域通常可以对应为一个微服务。
- 子领域可以视业务复杂性，或者技术要求（性能），单独拆分为一个独立的服务，独立开发和部署



按照领域设计对业务架构进行分解

领域 - 保险

子域 - 产险

核心子域 保单	
核心子域 用户	核心子域 产品

子域 - 寿险

核心子域 保单	
核心子域 用户	核心子域 产品

子域 - 养老险

核心子域 保单	
核心子域 用户	核心子域 产品

领域 - 销售

子域 - 电销

核心子域 用户	核心子域 任务
核心子域 销售人员	核心子域 订单

子域 - 网销

核心子域 用户	核心子域 产品
核心子域 购物车	核心子域 订单

领域 - 理财

核心子域 账户	核心子域 订单
核心子域 用户	核心子域 产品

领域 - 购物

核心子域 商品	核心子域 购物车
核心子域 订单	核心子域 用户

领域 - 基金

核心子域 账户	核心子域 订单
核心子域 用户	核心子域 产品

领域 - 车生活

领域 - 运动

通用支撑域

通用支撑域
账户 (国寿一账通)

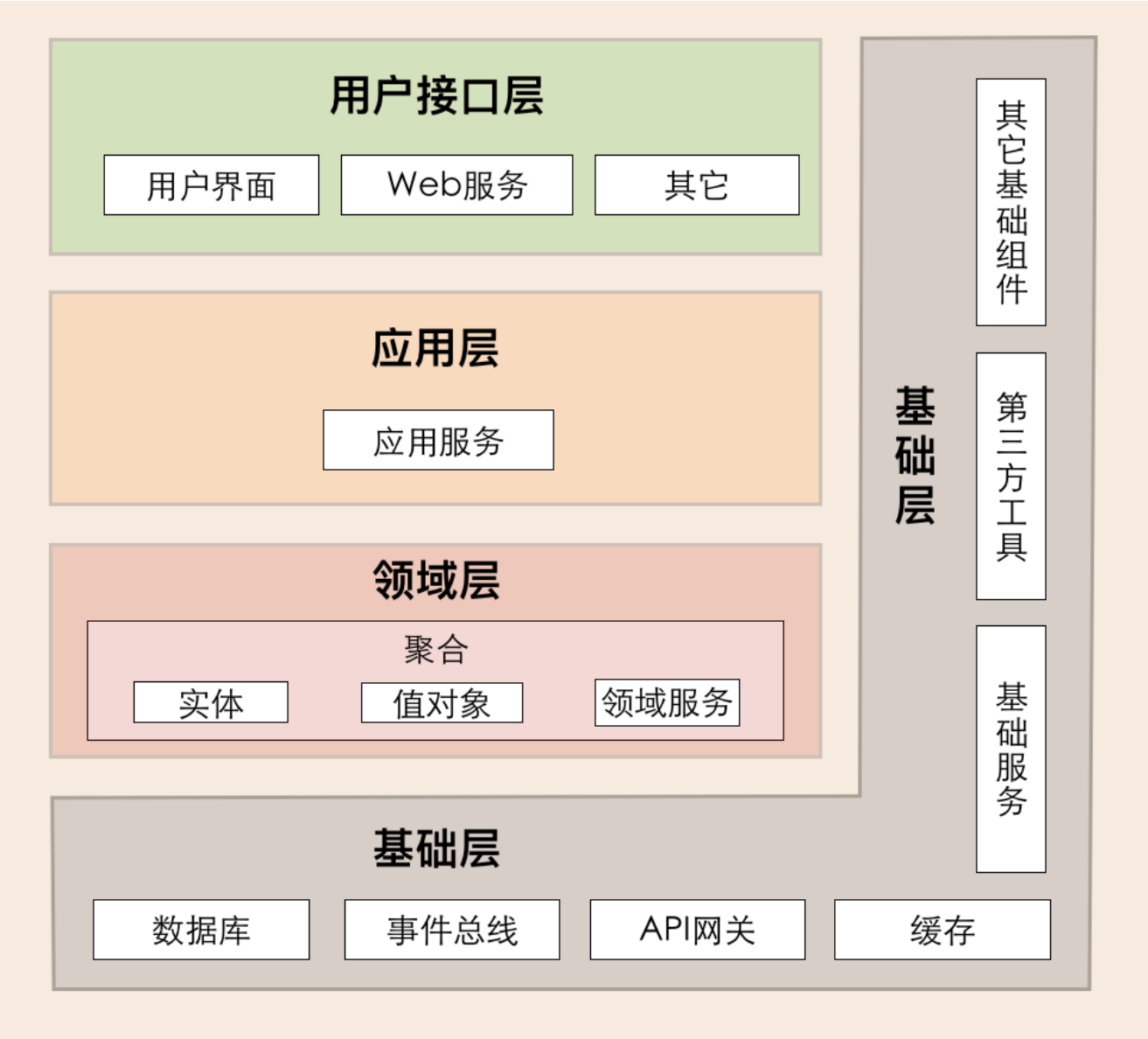
通用支撑域
支付 (国寿一账通)

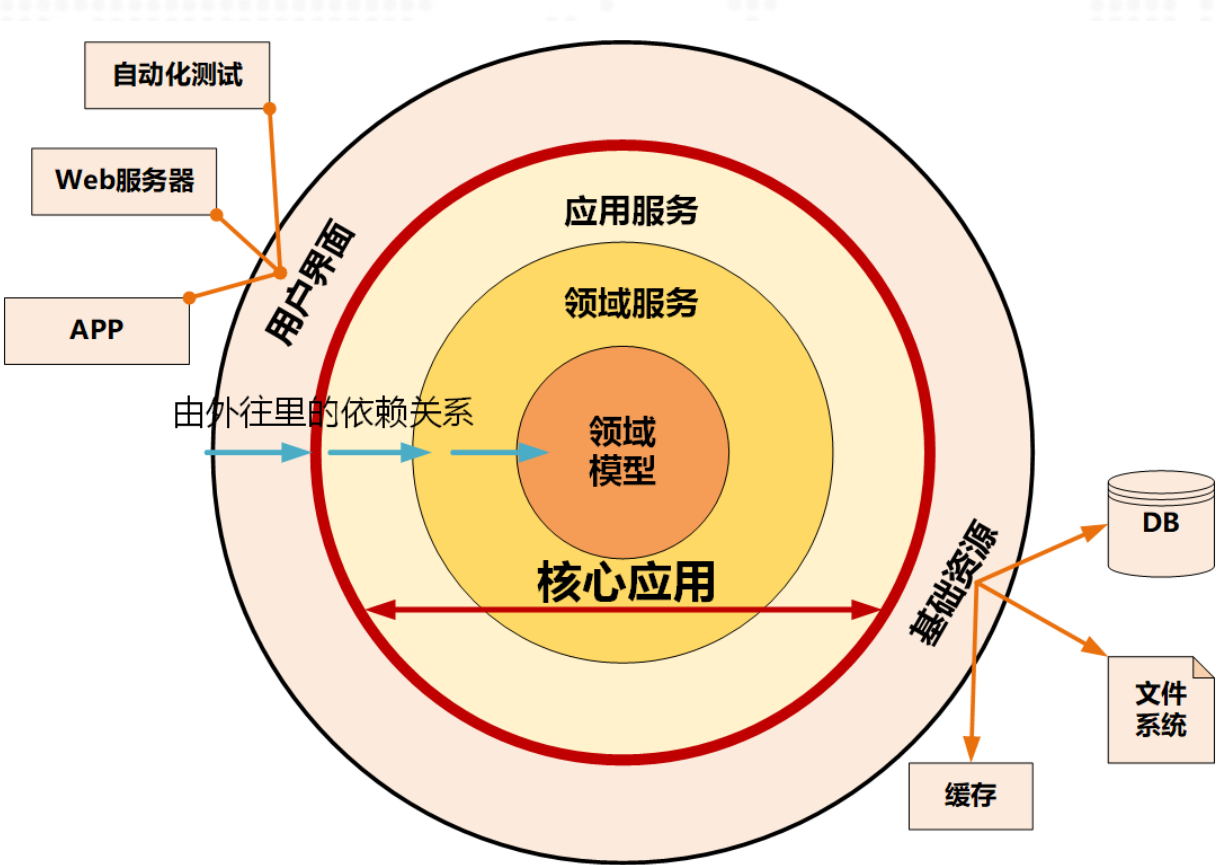
通用支撑域
卡券

通用支撑域
积分

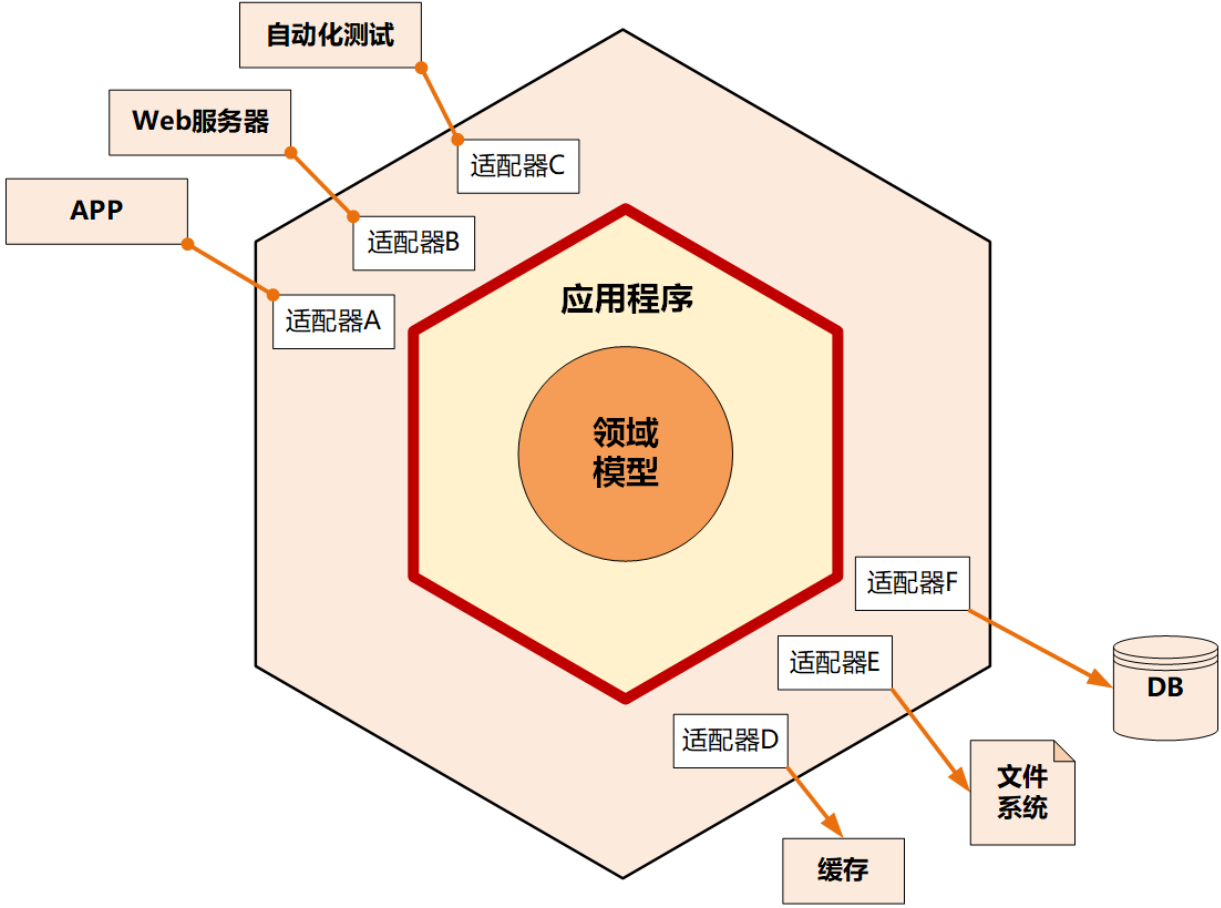
通用支撑域
物流

架构模型 – 分层架构





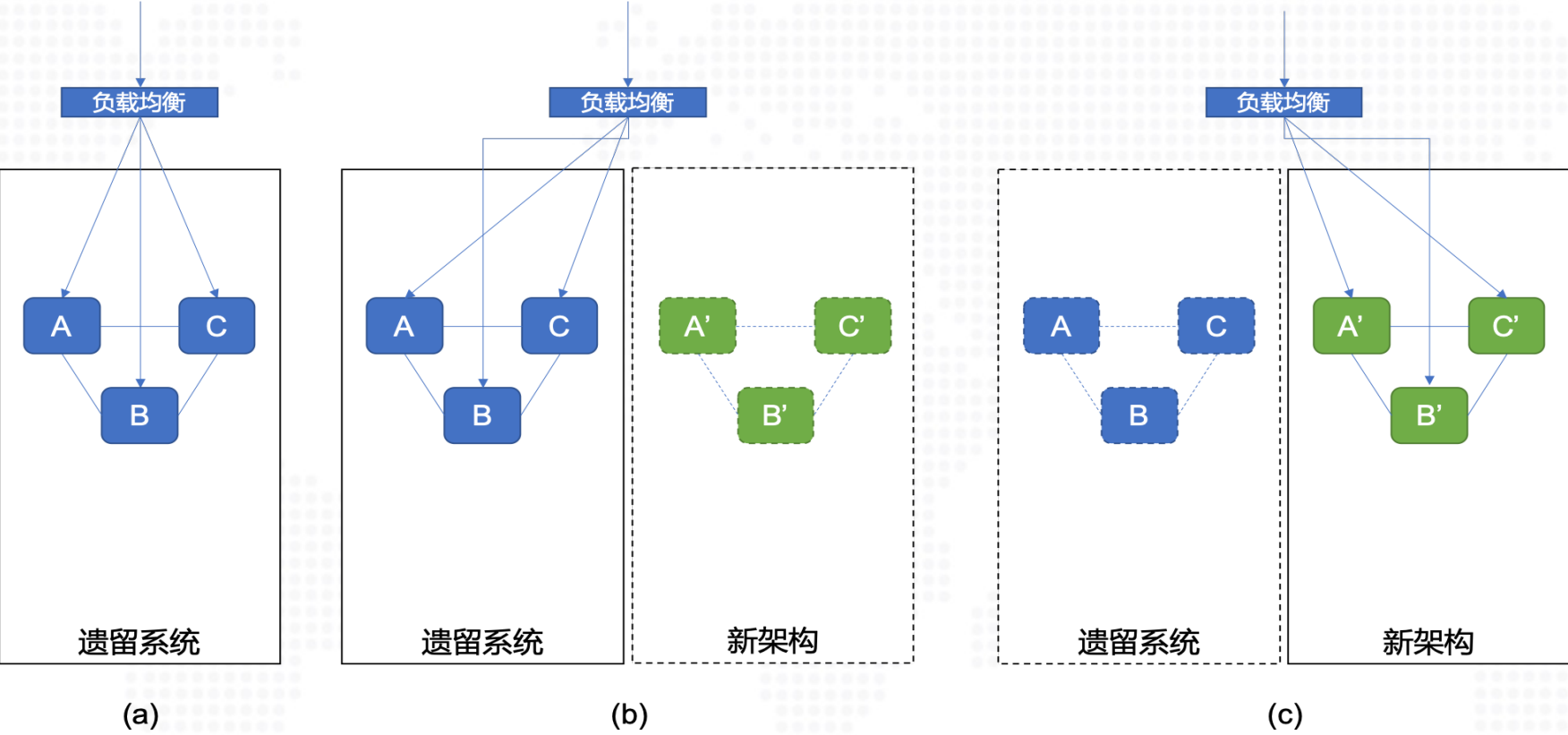
洋葱架构



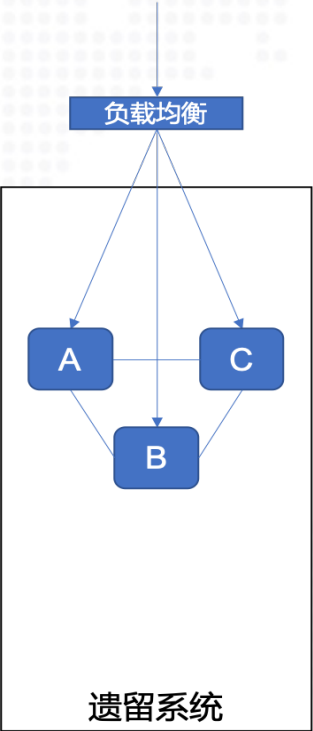
六边形架构

- Melvin Conway (康威) 在1967年发表的文章《HOW DO COMMITTEES INVENT》有一句很有名的话：
- *Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.*
- 这就是我们今天说的康威定律 - **任何组织在设计一套系统时，所交付的设计方案在结构上都与该组织的沟通结构保持一致。**用通俗的说法就是：屁股决定脑袋，组织架构等同系统架构。
- 实践：**做小而美的团队，人多会带来沟通的成本，让效率下降。**

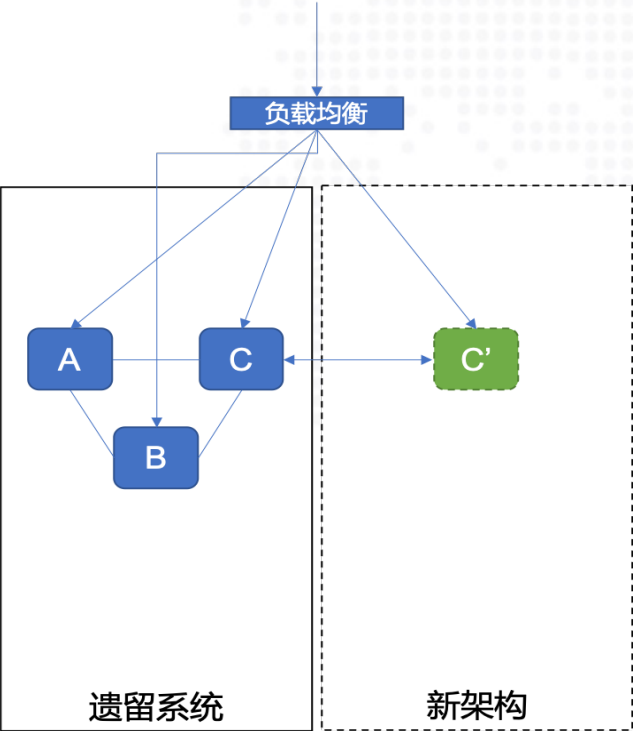
微服务改造策略



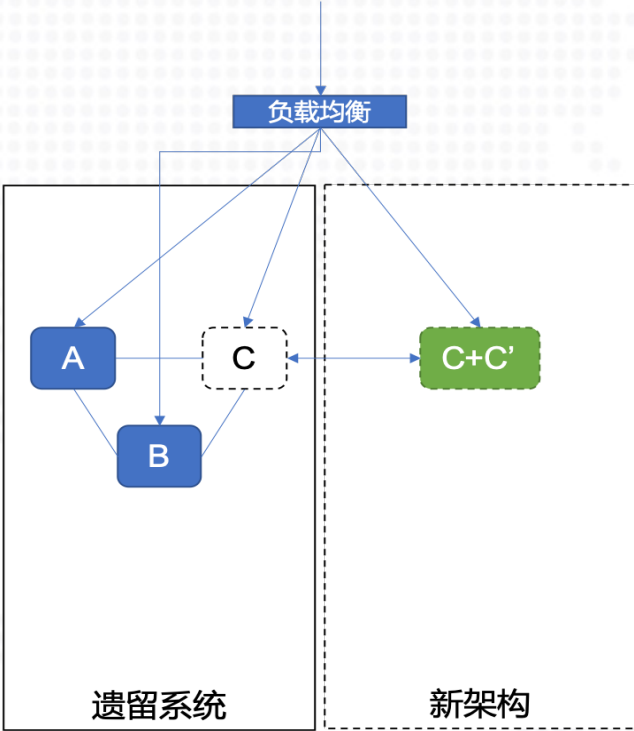
- **业务需求遗漏**：软件的历史版本中，有很多不为人熟知的功能还在使用。
- **市场环境变化**：由于新版本架构无法一蹴而就，当市场需求发生变化时，就会错失市场良机。
- **人力资源消耗大**：必须分出人力，一边维护旧版本的功能或紧急需求，一边要安排充分人力进行架构改造。
- **「闭门造车」**：新版本上线后，无法满足业务需求。



(a)



(b)



(c)

- **优点**

- 不会遗漏原有需求。
- 可以稳定地提供价值，频繁地交付版本，可以更好地监控其改造进展。
- 避免“闭门造车”现象。
- 不破不立，只针对有变更的功能进行改造，降低出问题的风险。

- **缺点**

- 架构改造的时间跨度会很大，也存在不确定性。
- 产生一定的迭代成本。

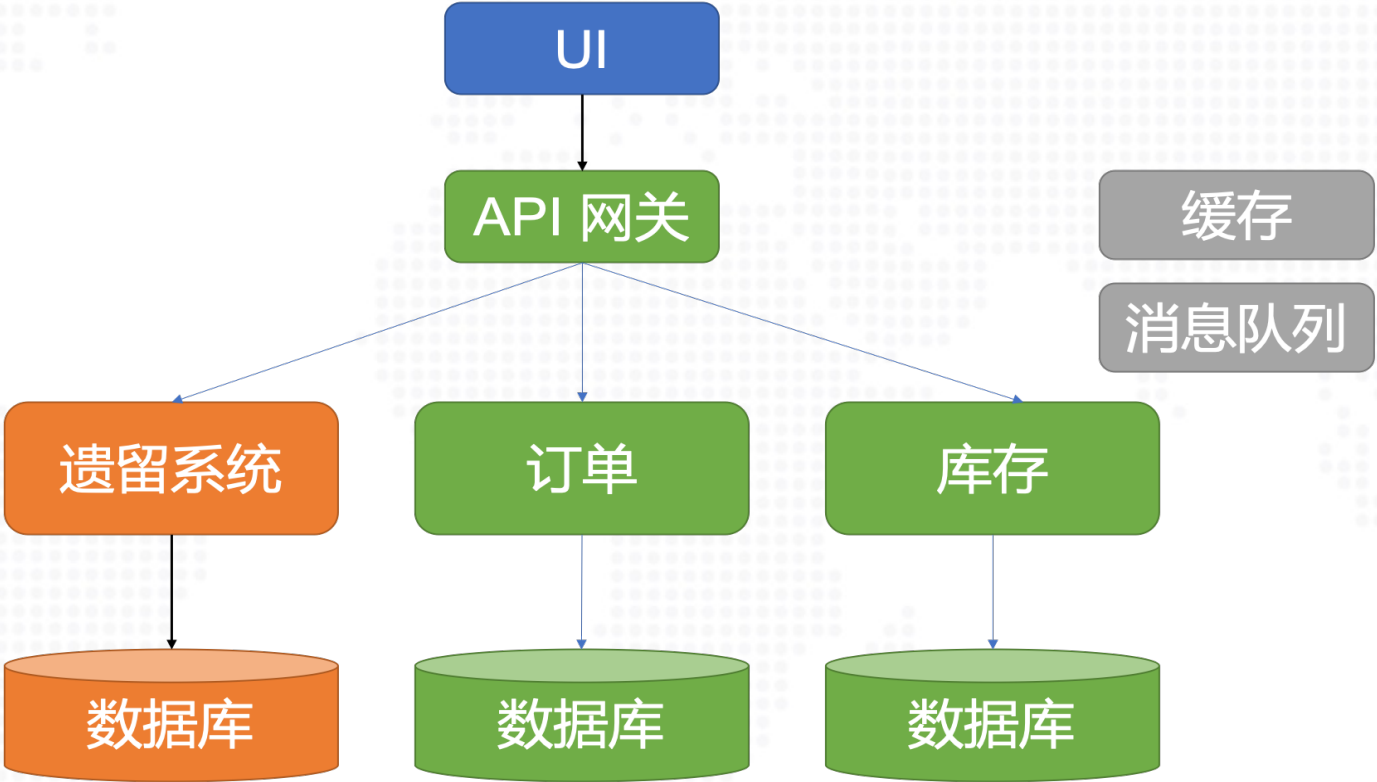
- **比较独立的新业务优先采用微服务架构。**从成本角度考虑，新业务采用新的架构是最合理的，对老业务的影响最小。
- **优先抽象通用服务。**因为通常通用服务的边界比较明显，耦合度低，比较容易分离。
- **优先抽象比较容易识别的、边界比较明显的服务。**如果原有包结构比较清晰，可以基于原有包结构中有明显边界的、比较完整的业务进行划分，这是从成本角度考虑。如果已经基于单体架构开发了一段时间，对业务的理解程度已经非常高了，那么这时往往开发及架构人员能够比较容易地提炼出一些边界比较明显的服务。
- **优先抽象核心服务。**因为微服务的开发及运维成本比较高，并不是所有的地方都要划分的粒度比较小，往往一些比较边缘的运营、管理的系统就不会拆分，另外，随着时间的推移，有一些业务可能会发生改变，因此，应该先抽象出核心服务。
- **优先抽象具有独立属性的服务。**根据功能的变更频率，资源占用，技术栈等属性划分服务。

遗留系统的部分功能与其余部分隔离，以新的架构进行单独改善。在改善的同时，需要保证与其他部分仍能协同工作。这种方式与绞杀者策略类似，但改造只发生在同一个系统内部，而非遗留系统外部。

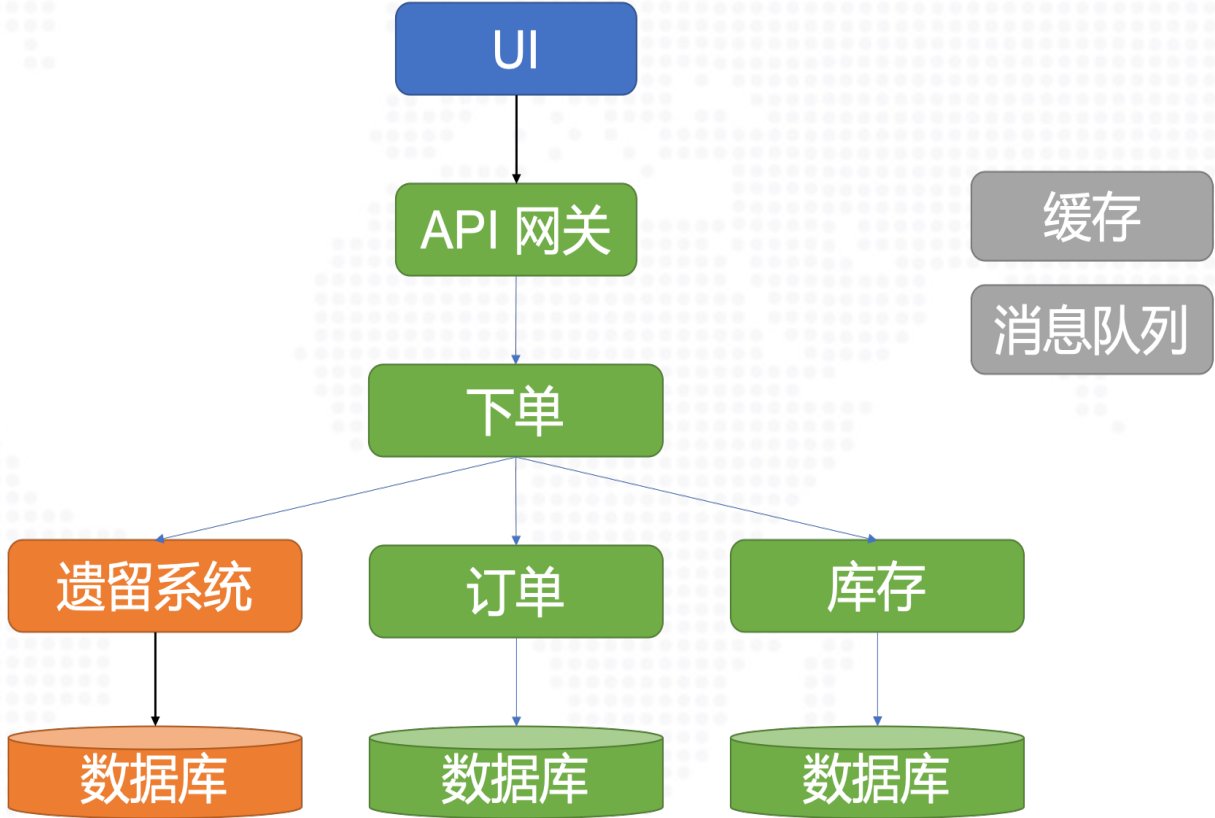
“修缮者策略”本质上就是代码重构，将遗留系统进行模块化改造，模块和模块之间解耦。模块可以很容易被改造为独立运行的服务，模块之间原本的函数调用改为 API 调用即可。

1. 应用无状态化
2. 前后端分离，前后端采用 API 进行交互。这样做的目的可以让前端和后端独立，不紧耦合。
3. 引入网关，这样后面单体应用拆分成微服务之后，前端无感知。
4. 提取公共基础服务，如单点登录，拆分可以遵循逻辑分离和物理分离两种方法。另外随着系统压力的增加，可能会用到消息中间件、分布式缓存等服务。
5. 使用“绞杀者策略”，不断地从老系统抽象服务，垂直划分优先。
6. 当业务越来越复杂的时候，需要引入编排服务，将一些业务逻辑由编排服务通过调用一组原子服务来实现

改造中的架构

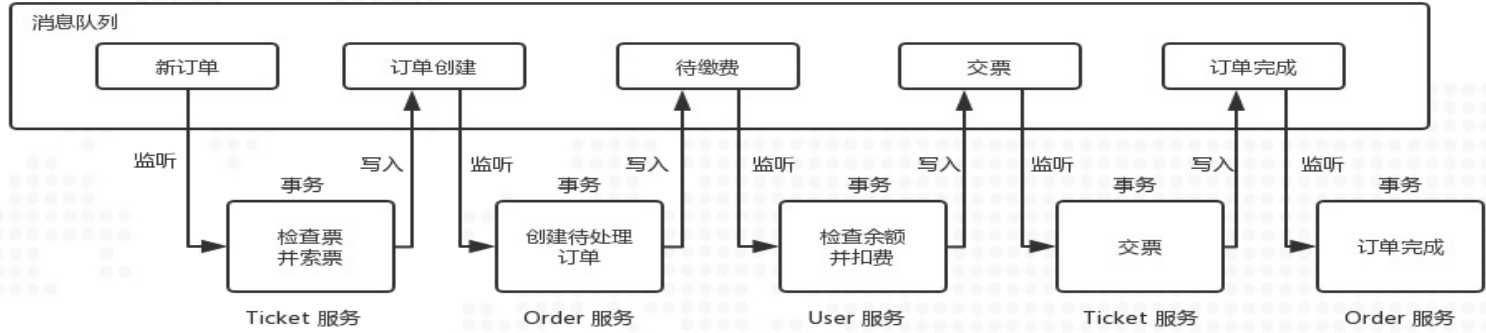


引入编排服务

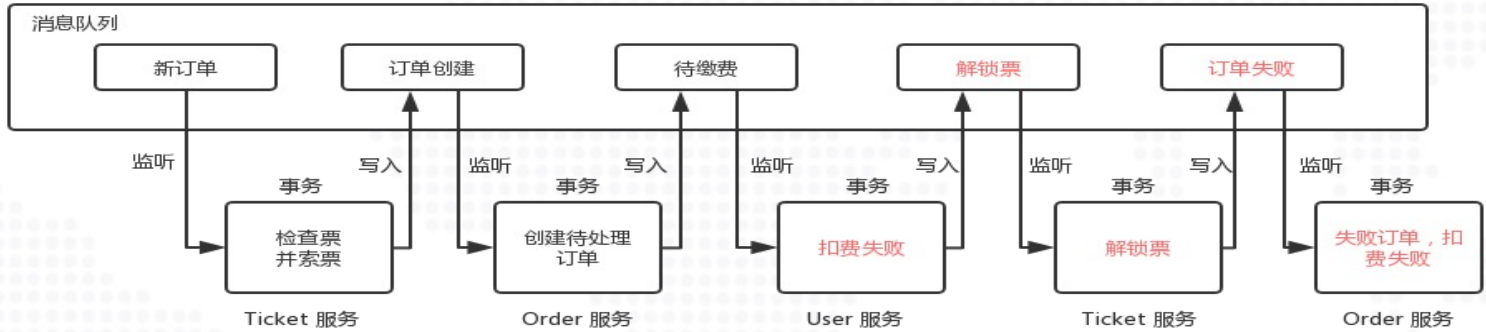


微服务模式

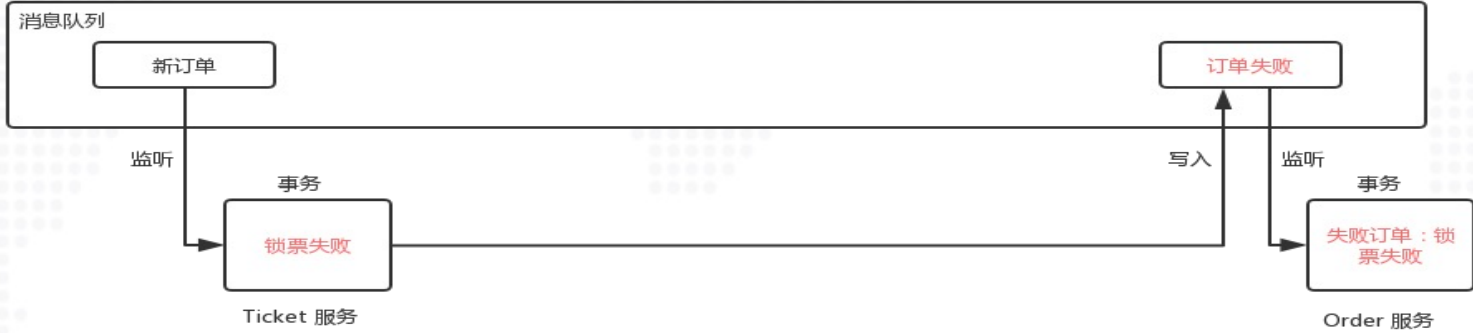
正常购票流程



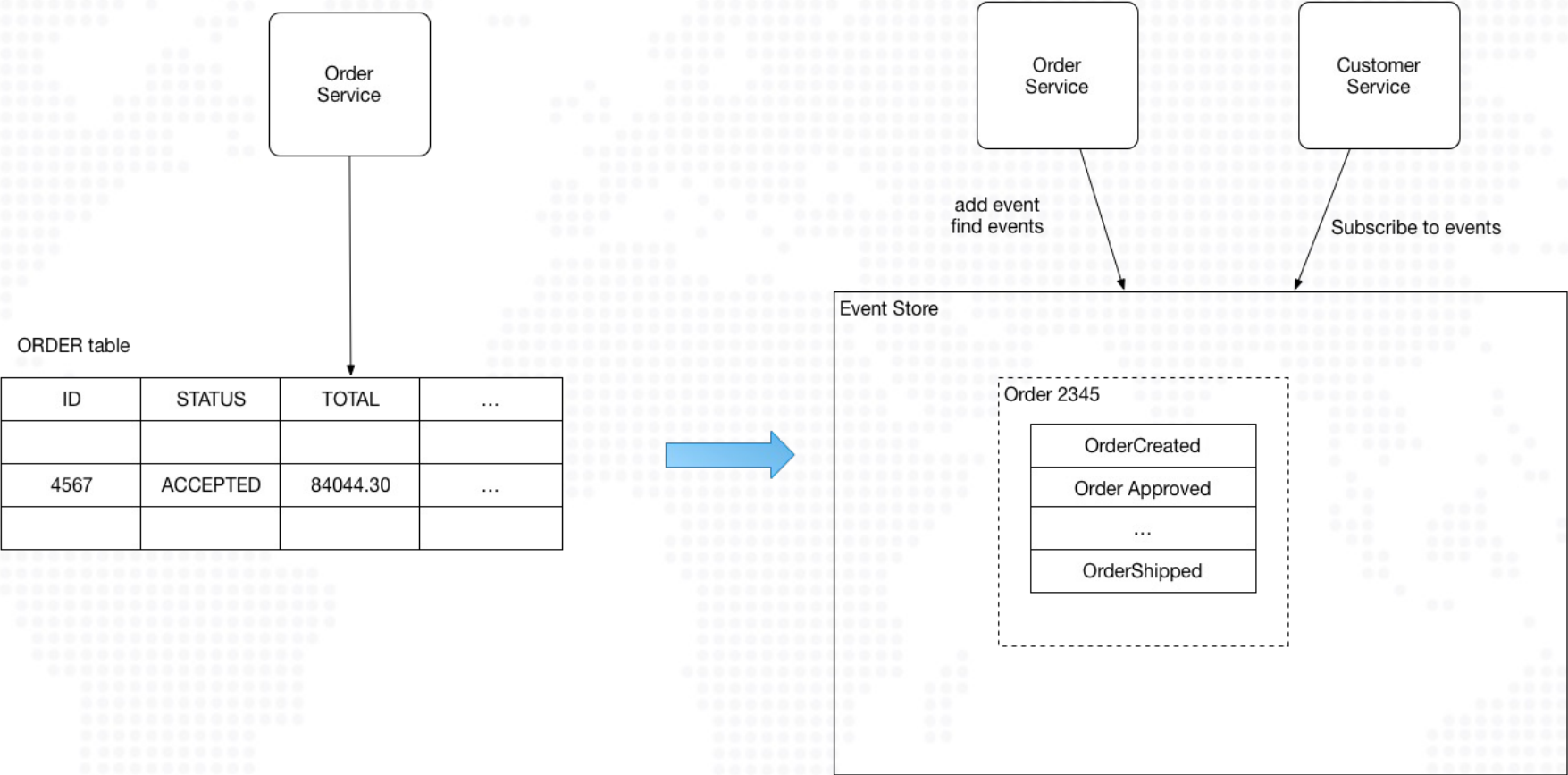
扣费失败流程



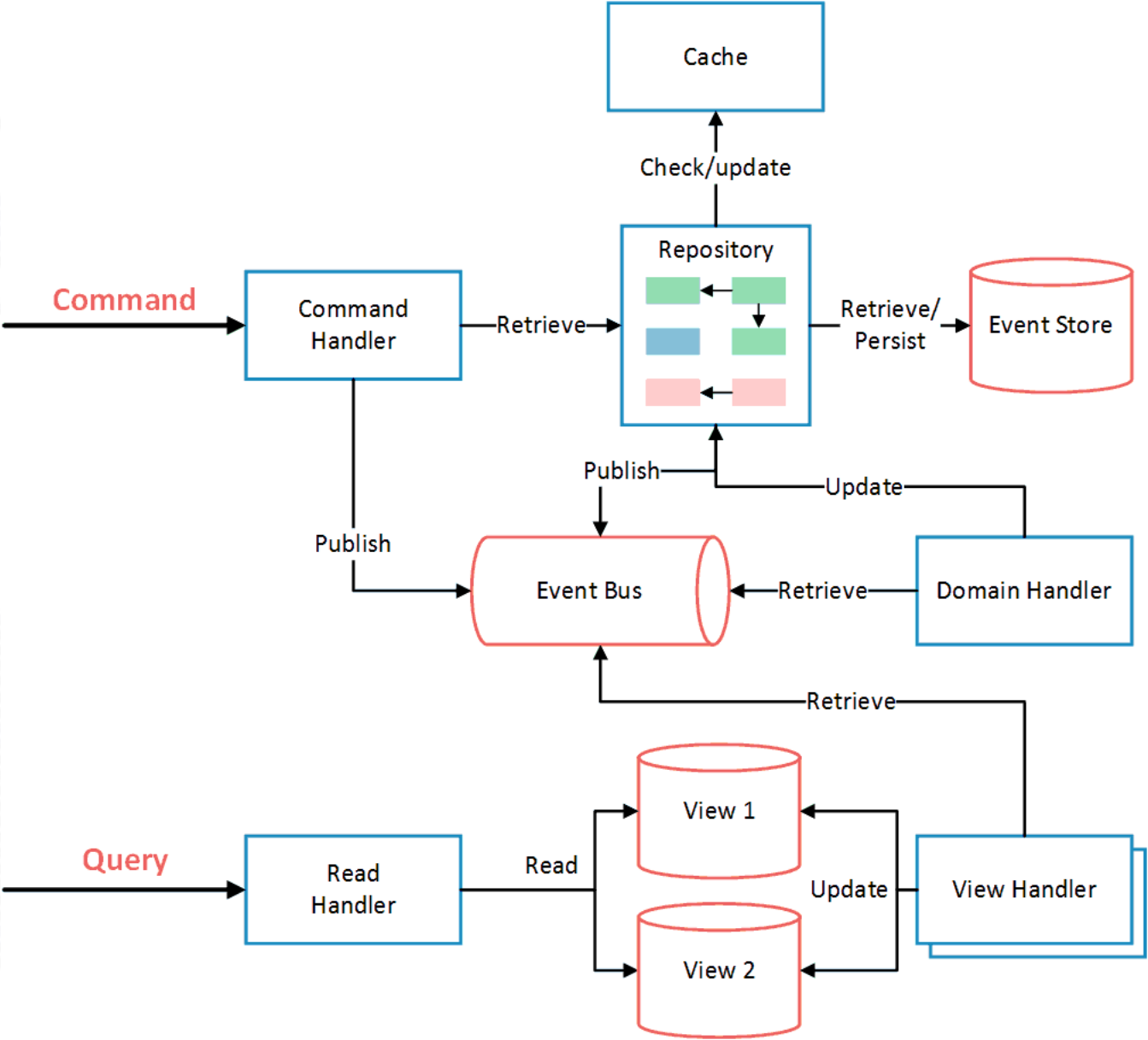
锁票失败流程



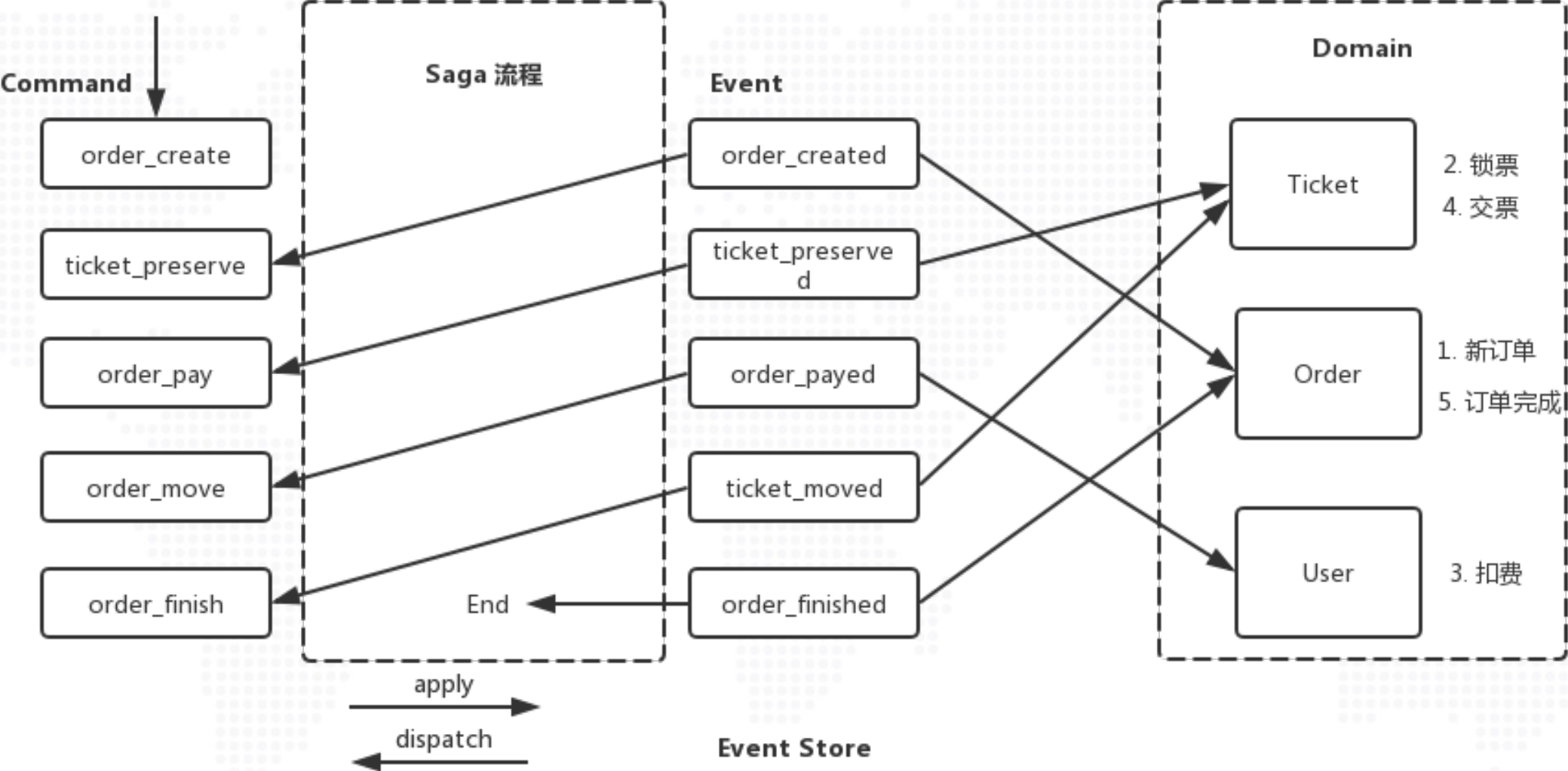
Event Sourcing (事件溯源)

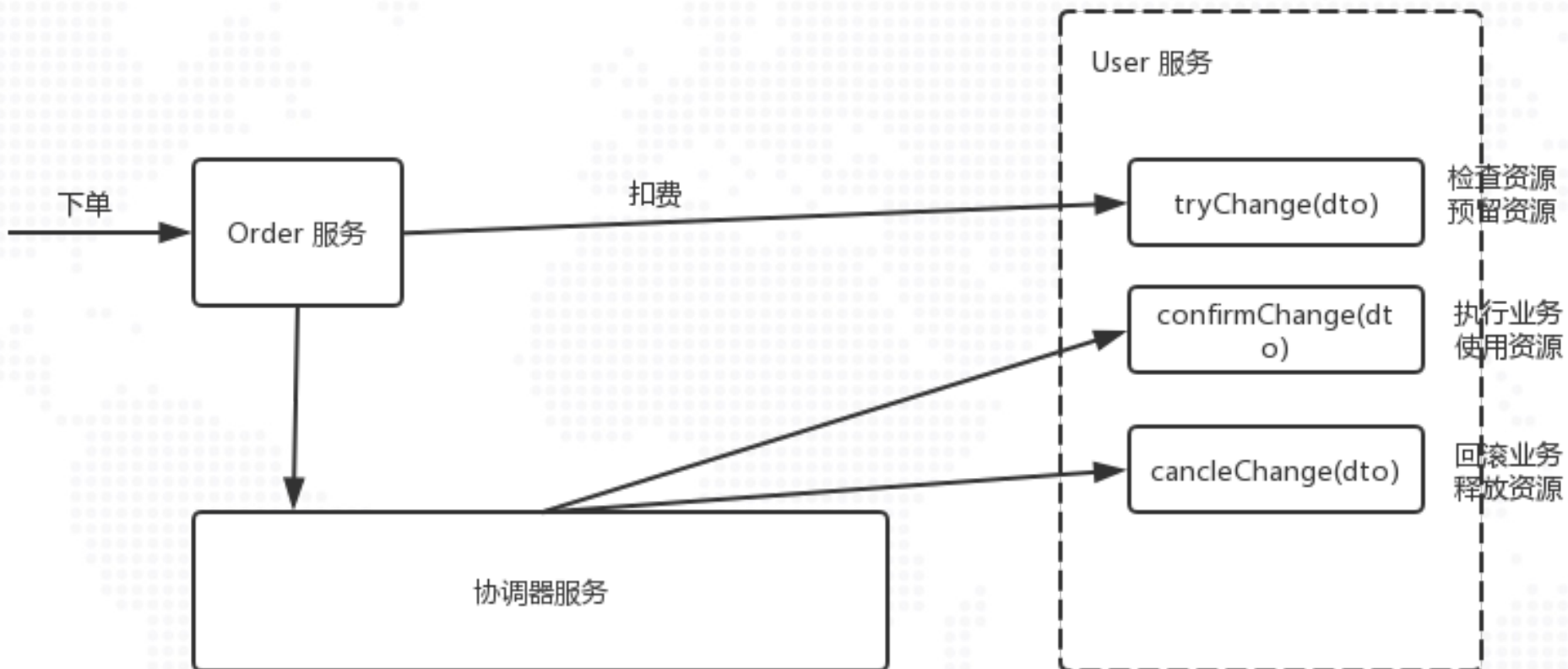


CQRS 模式



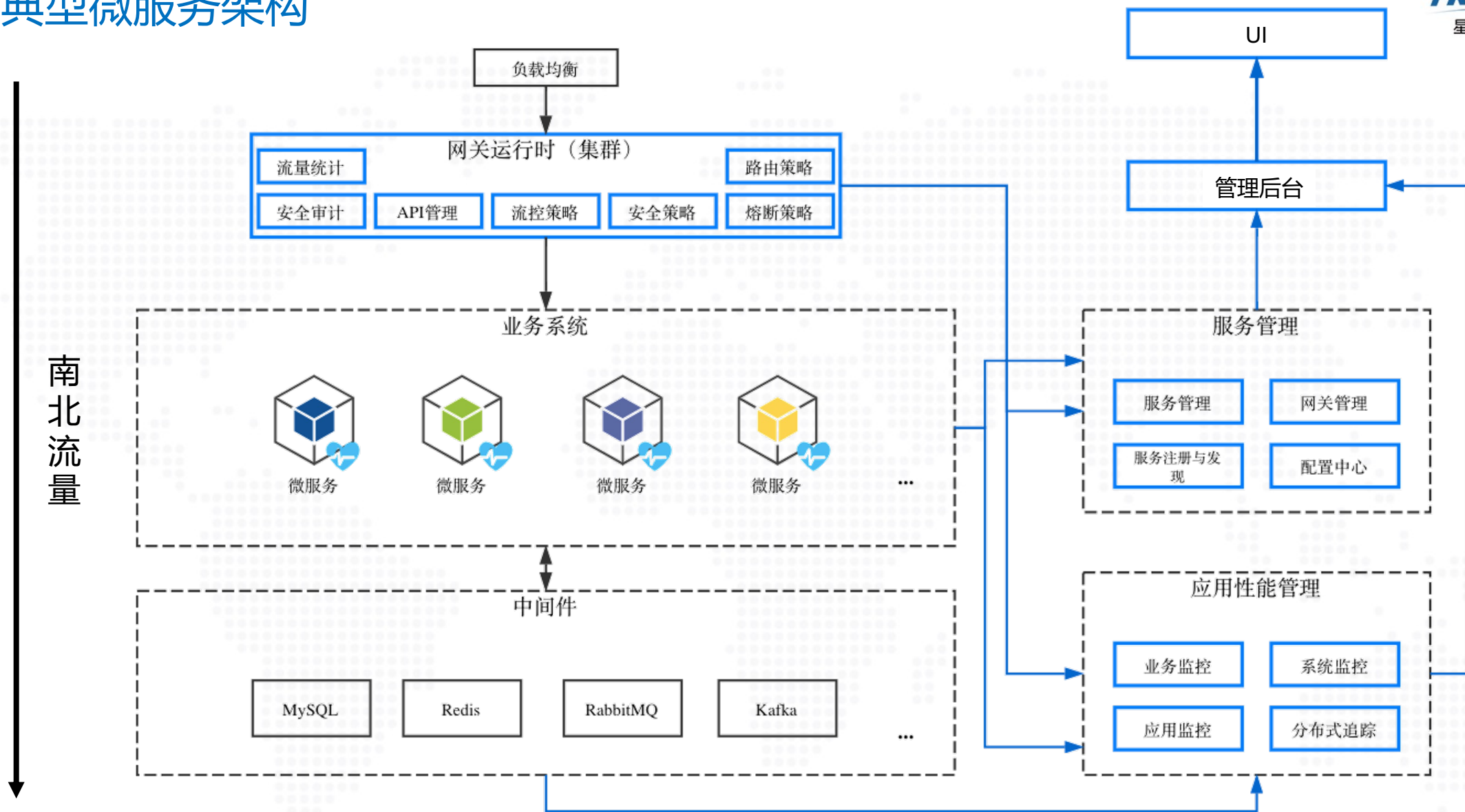
Saga 模式





微服务基础设施 与生态

典型微服务架构



问题

- 应用通常需要在不同的环境下运行，例如测试、预发布，生产环境等等，因此需要与之对应的多套配置文件。

方案

- 将应用程序所有相关的配置信息，包括数据库、网络连接等等保存在外部。在应用启动时，从外部（例如操作系统环境变量）读取这些配置信息。

功能	Apollo	Spring Cloud Config
配置界面	统一界面管理不同环境/集群配置	无，通过Git
配置生效时间	实时	重启，或者使用Refresh机制，或者和Spring Cloud Bus配合
版本管理	支持	部分支持（通过Git来实现）
灰度发布	支持	不支持
授权/审计/审核	界面上直接支持，支持修改和发布权限分离	需要通过Git仓库设置，不支持细粒度权限控制
实例配置监控	可以方便看到当前客户端使用的配置信息	不支持
配置获取性能	快，通过数据库+缓存	慢，需要Git Clone，然后本地文件读取
客户端支持	原生支持Java/.Net，提供API，支持Spring annotation	Spring应用 + annotation支持

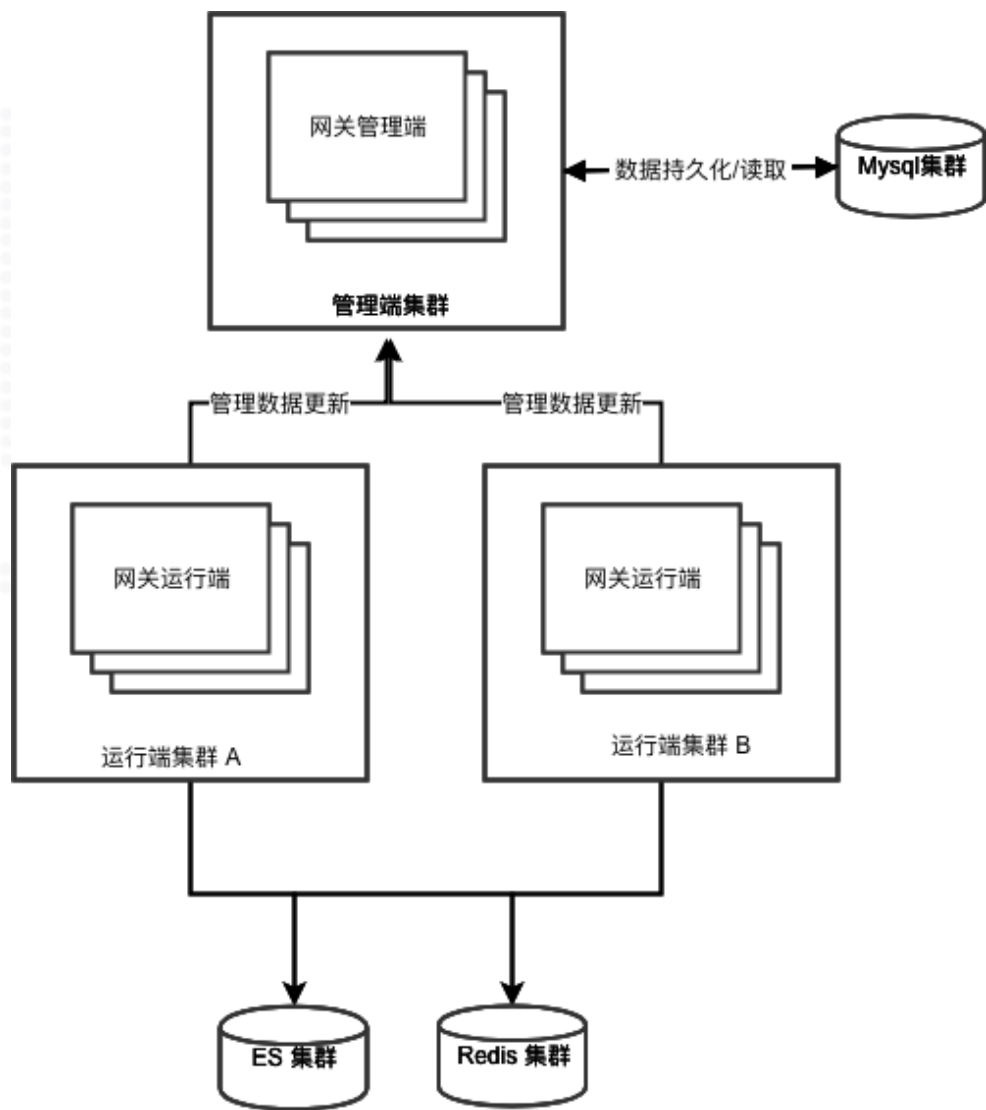
问题

- 微服务架构的应用客户端如何访问各项服务？

需求

- 微服务提供的API粒度通常与客户端需要的有所不同。微服务通常提供的是细粒度API，这意味着客户端需要同多项服务进行交互。
- 不同客户端需要不同的数据。
- 不同客户端的网络性能亦有所区别。
- 服务实例数量与其位置（主机与端口）会发生动态变化。
- 服务的划分方式会随时间的推移而改变，且不应被客户端所感知。

		Zuul	API网关商业产品			
			Mule	TYK	WSO2	Axway
API管理特性	请求分发	开源支持	免费支持	免费支持	免费支持	收费支持
	请求鉴权	需二次开发	免费支持	免费支持	免费支持	收费支持
	参数校验	需二次开发	免费支持	免费支持	免费支持	收费支持
	参数转换	需二次开发	免费支持	免费支持	免费支持	收费支持
	流量控制	需二次开发	免费支持	免费支持	免费支持	收费支持
	集群支持	开源支持	免费支持	收费版支持	免费支持	收费支持
	接口管理系统	需二次开发	收费支持	免费支持	收费版支持	收费支持
	开发者门户	需二次开发	收费支持	免费支持	收费版支持	收费支持
微服务特性	服务注册与发现	开源支持	N/A	免费支持	N/A	N/A



1. 管理端与运行端独立，管理端故障不会影响运行端
2. 管理端集中配置
3. 运行端配置数据热加载
4. 支持多网关运行端集群，满足不同业务需求

用户访问网关的历史数据

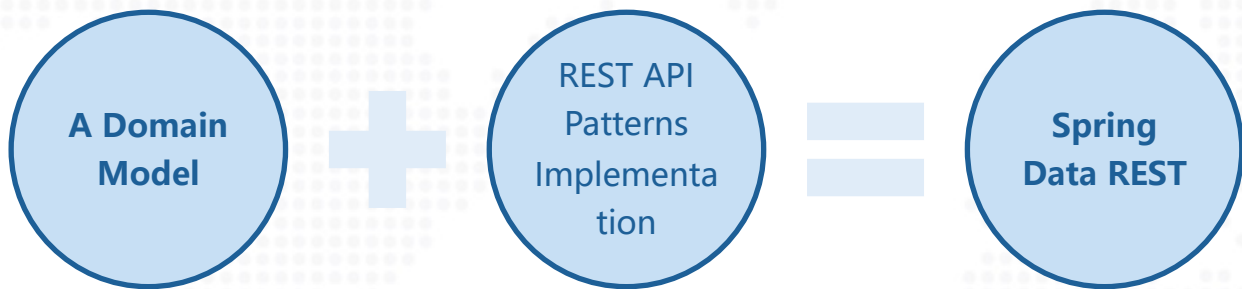


当前网关中高频API，错误数严重和耗时严重的API 显示

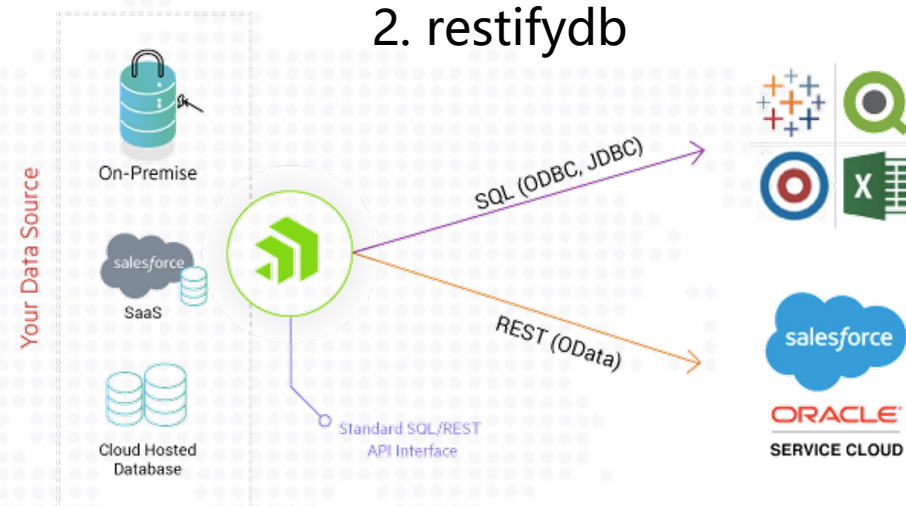
用户访问网关 API 后的状态码占比

数据服务 - 将数据库以 REST API 方式对外暴露

1. Spring Data Rest



2. restifydb



Hypermedia as the Engine of Application State (HATEOAS) is a component of the REST application architecture that distinguishes it from other network application architectures.

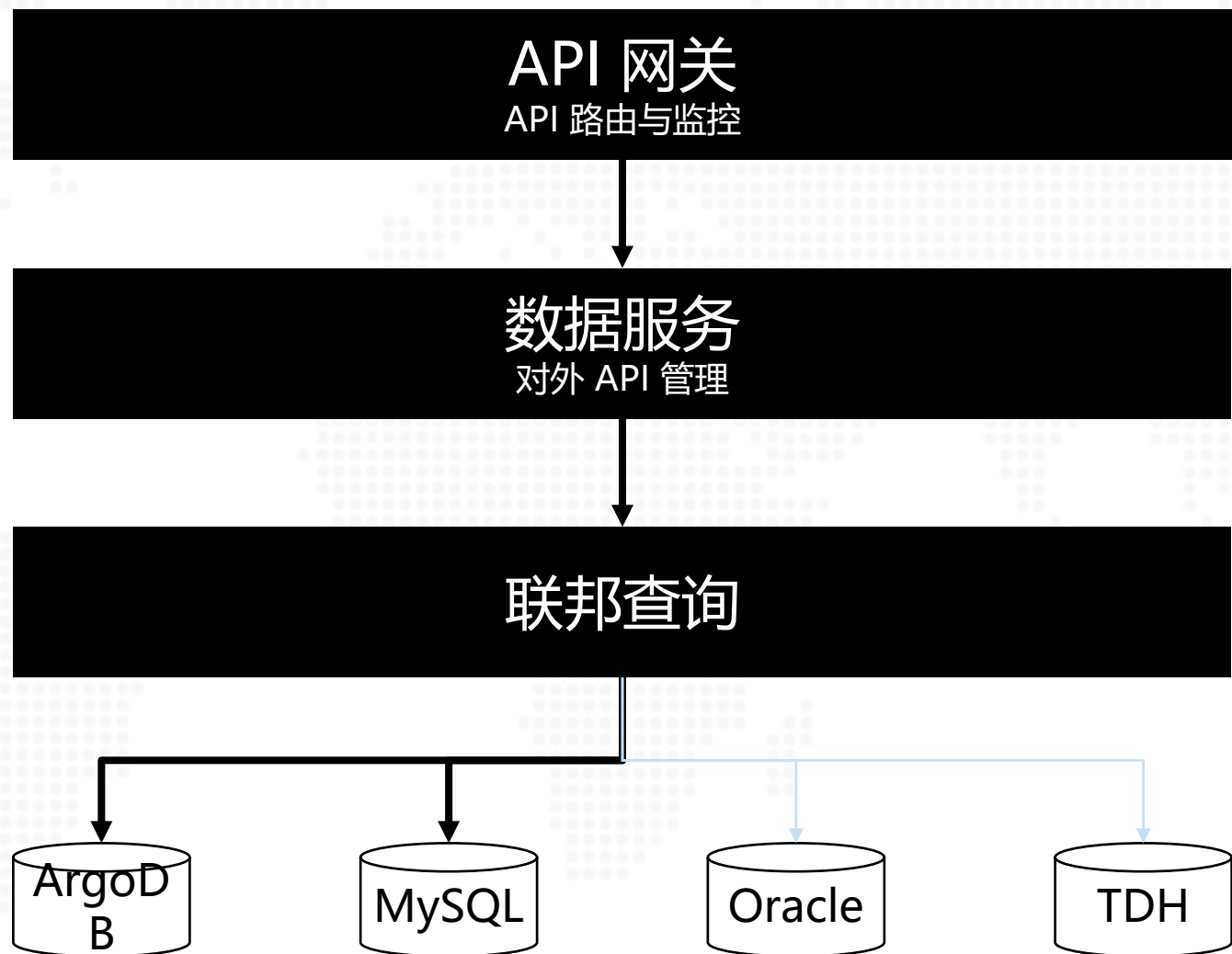
基于 OData (Open Data Protocol) is an ISO/IEC approved, OASIS standard, 自动生成 REST API

参考: <https://www.springcloud.cc/spring-data-rest-zhcn.htm> 参考: <https://restifydb.com/api/documentation>

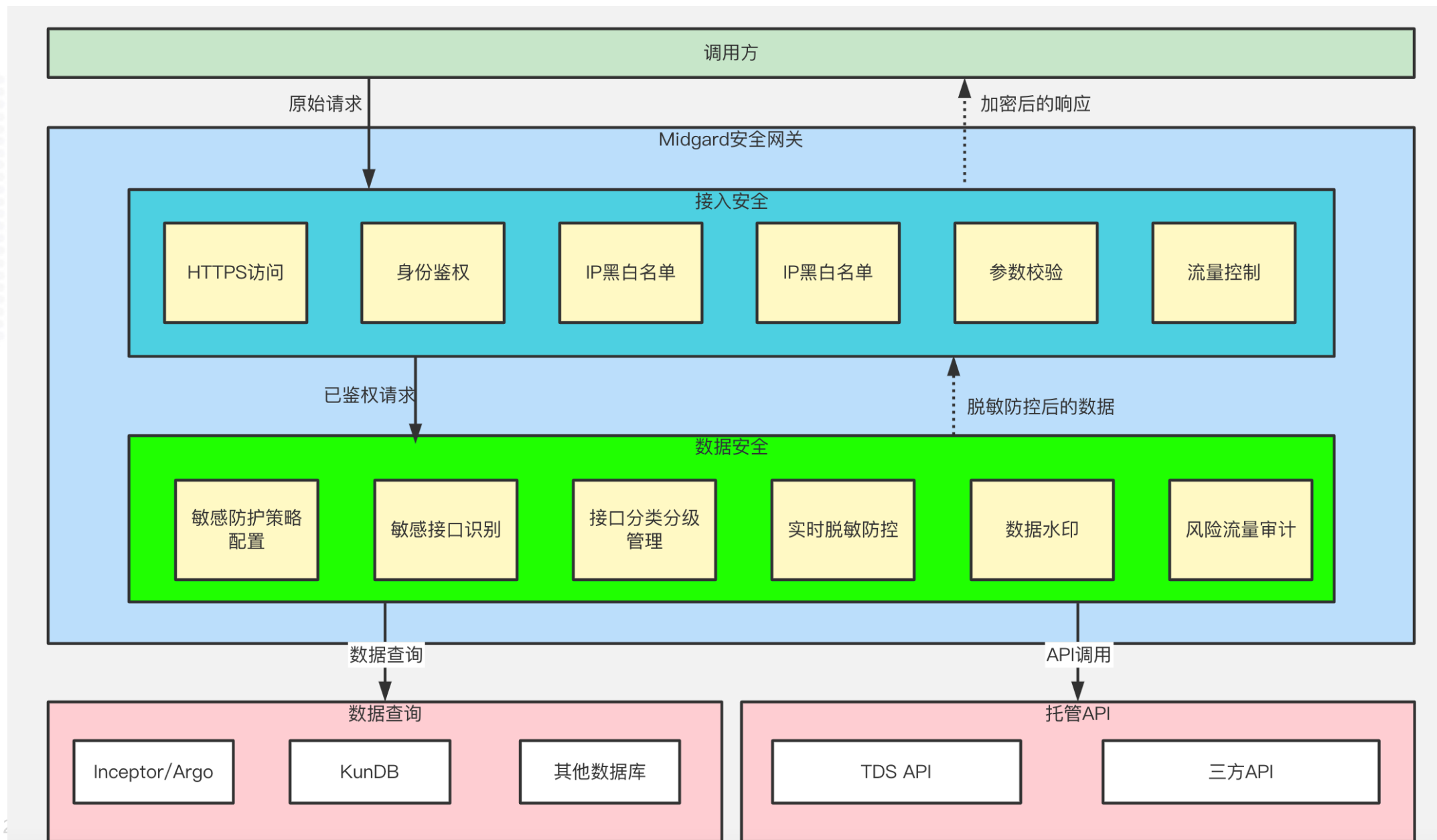
挑战

1. 支持 CRUD 操作等数据库操作 RESTful 化
2. 支持多种数据库
3. 表结构自动发现
4. 自动支持多表查询
5. 权限控制

核心挑战：跨库查询 + 性能



数据安全网关架构

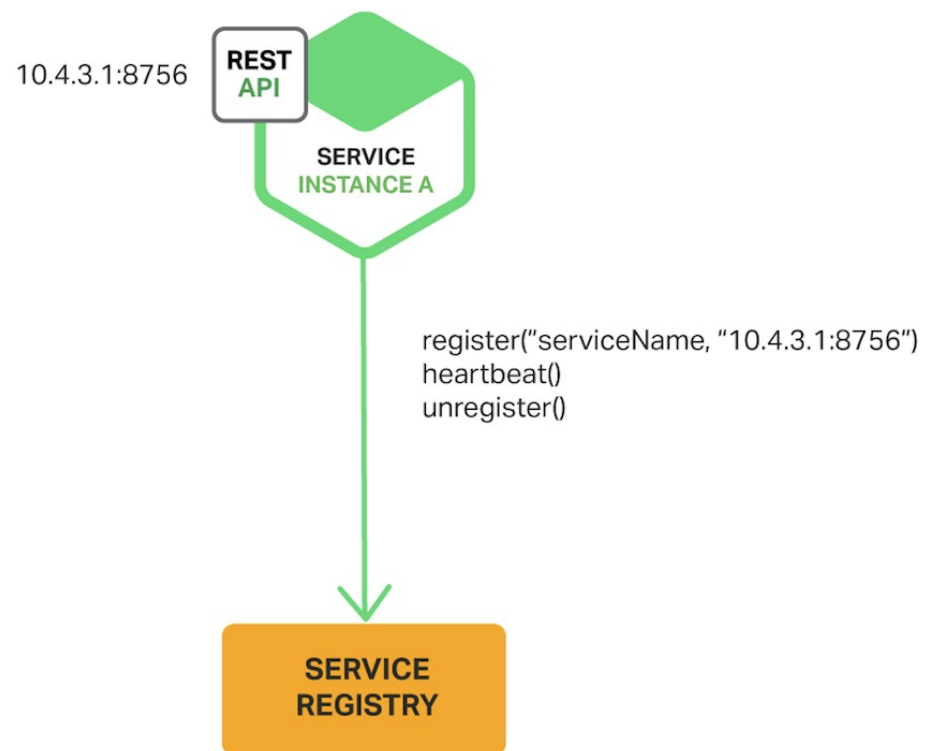


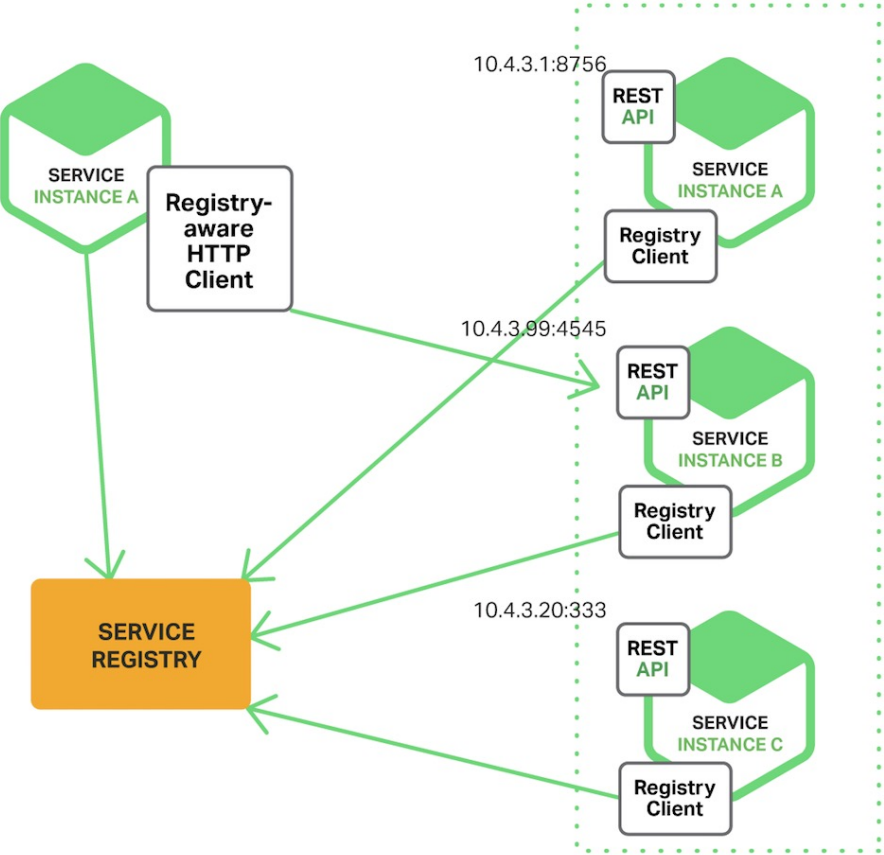
问题

- 服务的消费者如何才能获取服务端实例的访问信息？

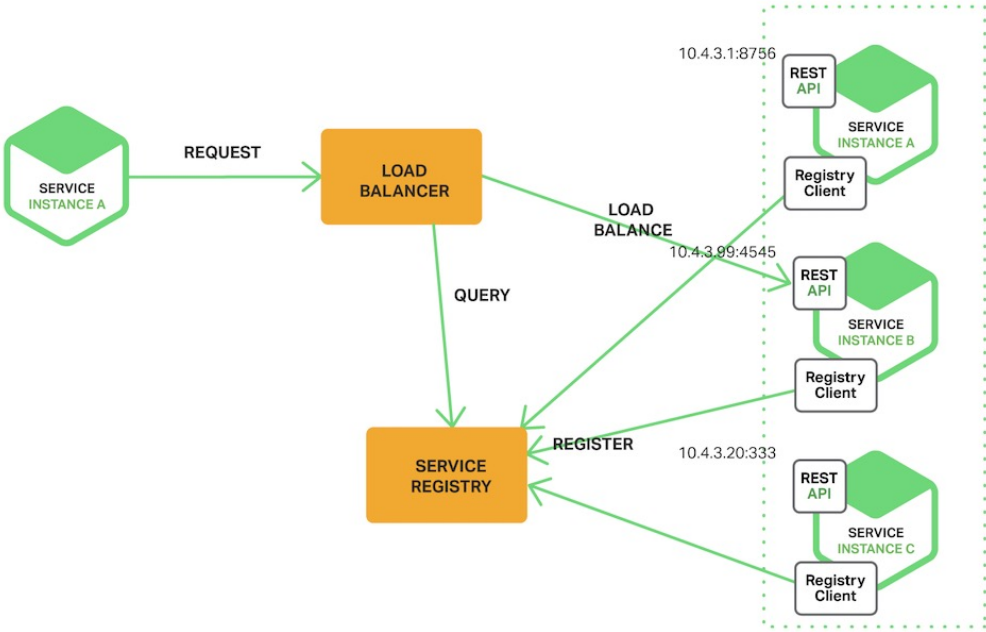
需求

- 每一服务实例都会在特定位置（主机与端口）通过HTTP/REST或者Thrift等方式发布一个远程API。
- 服务端实例的具体数量及位置会发生动态变化。
- 虚拟机与容器通常会被分配动态IP地址。
- 服务实例的数量会发生动态变化。





客户端服务发现



服务端服务发现

企业级「服务注册与发现」需求

- 服务注册

对于提供服务的每一个实例，要能够识别当前环境和存储相关信息。存储的信息本身通常采用键/值对的格式。由于服务发现经常用于分布式系统，所以要求这些信息可伸缩、支持容错。

- 服务发现

服务发现背后最基本的思想是，所有对服务注册阶段存储信息感兴趣的各方，可以找到最起码诸如服务IP地址和端口这样的信息，用于它们之间的相互通讯，这些数据还经常扩展到其它类型的信息。服务发现工具要提供某种形式的 API，用于服务自身的注册以及服务信息的查找。

- 负载均衡

可以制定服务分配规则，例如带某些标签的 client 请求服务时，被分配特定的服务端等。

- 健康检查

可以制定服务失效判断规则，例如，服务不可连接；服务未上报健康状态；服务不返回200；服务所在宿主机内存大于90%等。

- 自动恢复

服务被认定失效后，当其恢复时，应能够感知到服务上线，并及时分配给客户端。

- 服务重载

某些服务依赖静态配置文件，当配置信息变更是，能够及时感知到变更，并主动加载配置，重载服务。

- 去中心节点部署

服务的注册与发现均能够向任一个活着的节点发起。

- 数据的强一致性

分布式部署时，各个节点上注册的信息应该时刻保证一致，即某个服务信息只有复制到全部的活着的节点之后，才会被提供给 client 使用。

- 可视化的管理界面

方便用户使用。

几种常见的服务注册与发现软件比较

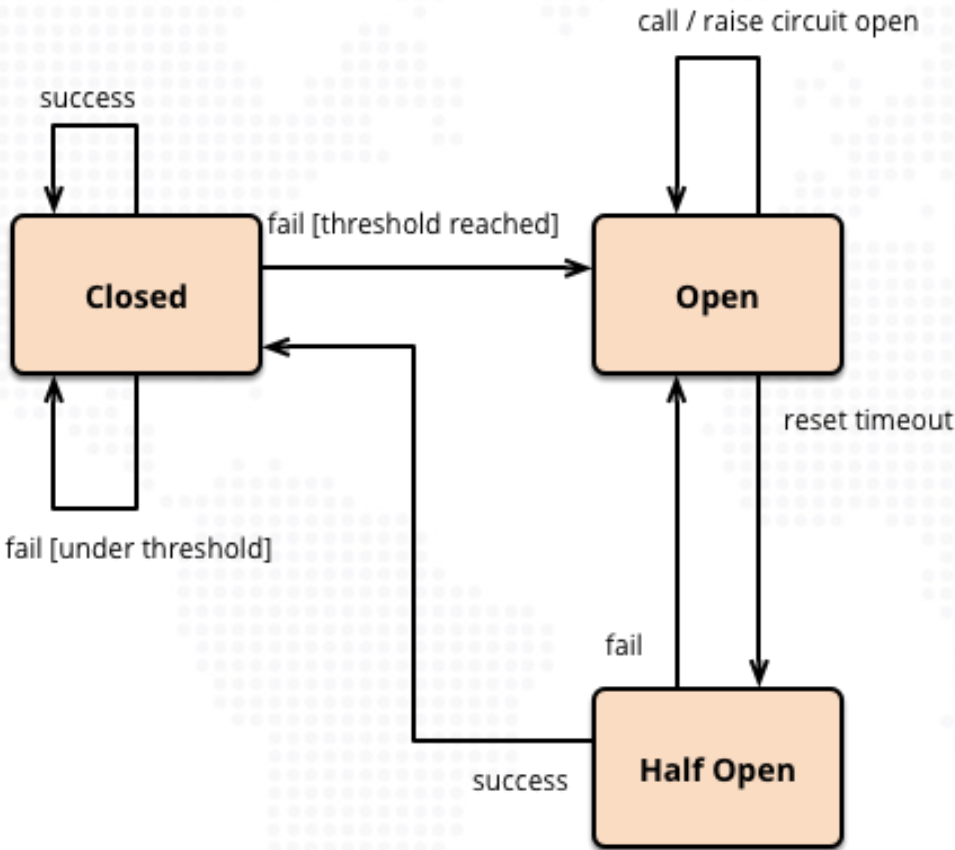
特性	Consul	Zookeeper	Etcd	Eureka
服务健康检查	服务状态, 内存, 硬盘等	(弱)长连接, keepalive	连接心跳	可配支持
多数据中心	支持	—	—	支持
KV存储服务	支持	支持	支持	—
一致性	raft	paxos	raft	—
CAP	CA	CP	CP	AP
使用接口(多语言能力)	支持 http 和 dns	客户端	http/grpc	http (sidecar)
watch支持	全量/支持long polling	支持	支持 long polling	支持 long polling/大部分增量
自身监控	metrics	—	metrics	metrics
安全	ACL/https	ACL	https支持 (弱)	—
Spring Cloud 集成	已支持	已支持	已支持	已支持

问题

- 微服务架构中，服务之间会相互调用，而远程调用可能存在失败的情况（例如超时）。
- 如果大部分服务调用处于等待状态，该服务就会不可用，从而导致调用该服务的其他服务也出现不可用，最后出现雪崩效应

需求

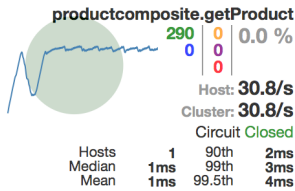
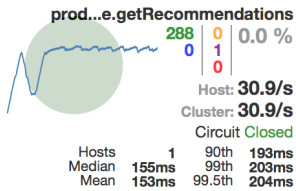
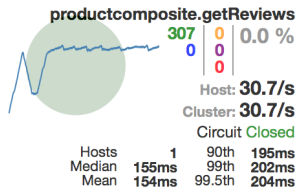
- 隔离失败：当第三方服务不可用时，所有对第三方服务的调用能够被隔离
- 自动重试：对失败的服务调用隔离之后，能够在适当的时机进行重试
- 监控统计：能够对每个接口的失败情况进行统计



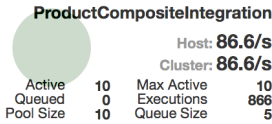
Hystrix Stream: <http://localhost:8989/turbine.stream>



Circuit Sort: [Error then Volume](#) | [Alphabetical](#) | [Volume](#) | [Error](#) | [Mean](#) | [Median](#) | [90](#) | [99](#) | [99.5](#)
[Success](#) | [Short-Circuited](#) | [Timeout](#) | [Rejected](#) | [Failure](#) | [Error %](#)



Thread Pools Sort: [Alphabetical](#) | [Volume](#) |



问题

- 在微服务架构中，各个服务之间的调用关系如蜘蛛网一样复杂，
- 需要追踪各个服务的调用链路，用于性能分析及查找问题

解决方法

- 在每个请求中加入特殊标记
- 每个服务记录日志的时候加入这个特殊标记
- 将这些日志收集在一起进行分析

基于微服务体系之下构建的业务系统存在的问题基本上分为三类：

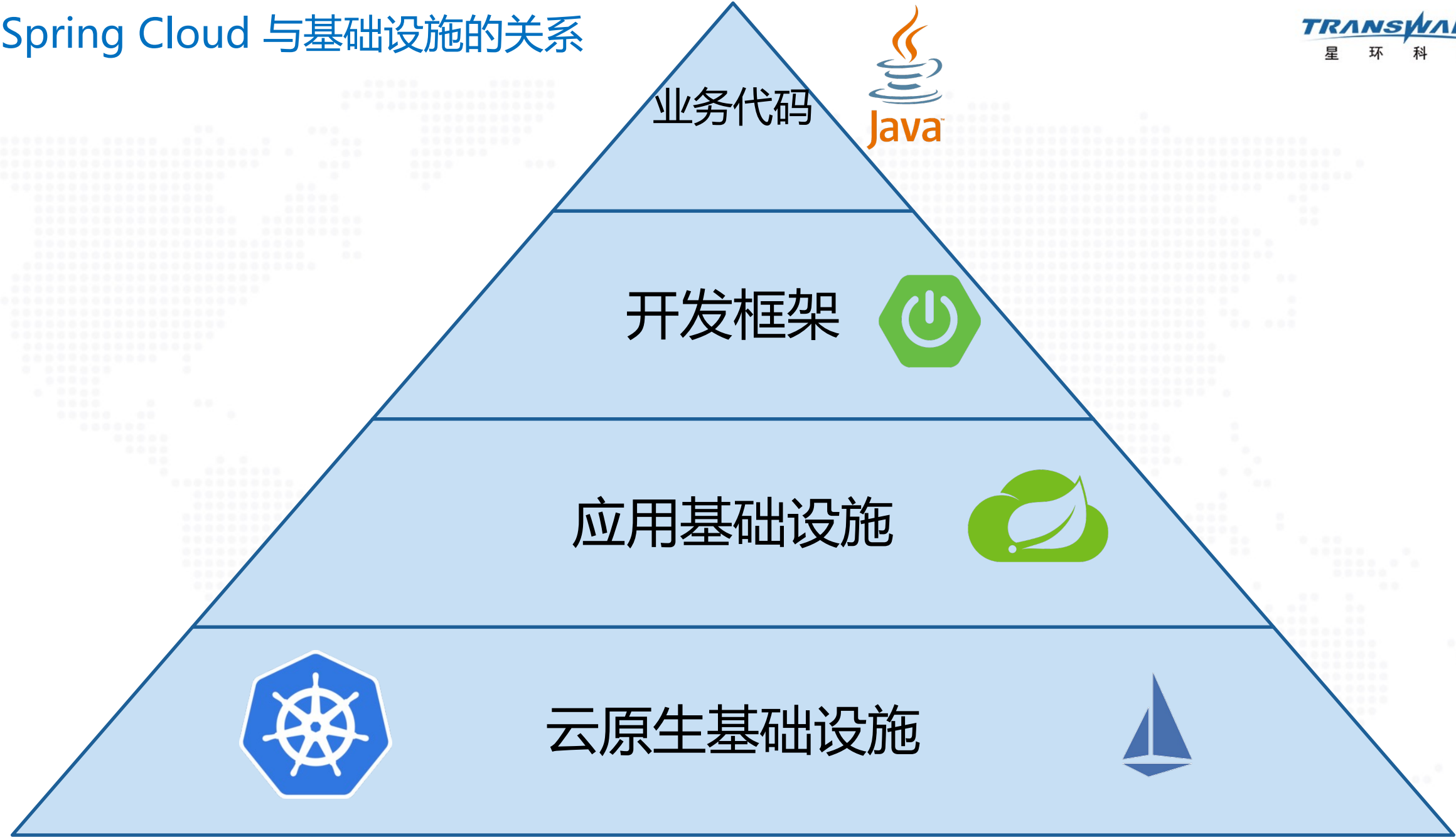
- 故障定位难，一个简单操作，其背后可能是由十几个微服务共同完成的，这些微服务也由不同的团队去负责。一旦出现问题，最坏情况下也许需要这十几个团队一起来解决问题。
- 链路梳理难，应用没有形成应用拓扑，不知道自己的服务下游会影响其他哪些人。
- 资源浪费多，容量预估难。对于一些服务，其消耗的 CPU 和 Memory 可能连 10% 不到，远远没有充分利用物理机。这其实和容量预估关联，过大或者过小估算峰值的机器容量，都是浪费。

分布式追踪主要的目的就是解决上面所说的这四个问题，主要的手段是通过收集、存储、分析、分布式系统中的调用事件数据，协助开发运营人员进行故障诊断、容量预估、性能瓶颈定位以及调用链路梳理。以下是链路监控组件的需求：

- 代码的侵入性
- 探针的性能消耗
- 全面的调用链路数据分析
- 可扩展性

	大众点评 CAT	Zipkin	Pinpoint	SkyWalking
调用链可视化	有	有	有	有
聚合报表	丰富	少	中	中
ServerMap	简单依赖图	简单	好	星 拱 科 技
埋点方式	侵入	侵入	非侵入（字节码增强）	非侵入（字节码增强）
HeartBeat支持	有	无	有	有（JVM）
Metrics支持	简单	无	有	有
告警支持	有	无	有	有
多语言支持	Java/.Net	丰富	只有Java	Java/.Net/nodejs
存储机制	Mysql（报表），本地文件/HDFS(调用链)	可选 In Memory, Mysql, Elasticsearch（生产），Cassandra（生产）	Hbase	Elasticsearch，H2，Mysql
界面中文友好	好	无	无	无
社区支持	内置文档，作者在国内	文档较丰富，中文社区一般	文档一般，暂无中文社区	文档丰富，中文社区活跃
国内案例	携程，点评，陆金所，拍拍贷	京东、阿里定制不开源	唯品会改造定制	华为，当当，小米，科大讯飞，DaoCloud
源头祖先	eBay CAL	Google Dapper	Google Dapper	Google Dapper
依赖	Java 6， 7， 8 Maven 3.2.3+ mysql 5.6 Linux 2.6以及之上（2.6内核才可以支持epoll）	Java 6， 7， 8 Maven 3.2+ rabbitMQ	Java 6， 7， 8 maven 3+ Hbase0.94+	Java 6， 7， 8 maven 3.0+ nodejs zookeeper elasticsearch
实现方式	代码埋点（拦截器，注解，过滤器等）	拦截请求，发送（HTTP，mq）数据至zipkin服务	java探针，字节码增强	java探针，字节码增强
通信方式	自定义RPC	Http，MQ	thrift	gRPC
MQ监控	不支持	不支持	不支持	支持（RocketMQ，Kafka）
trace查询	不支持	支持	不支持	支持
star数量	5.7 k	9.2 k	6.6 k	4.2 k
优点	功能完善	1、spring-cloud-sleuth 可以很好的集成 Zipkin， 代码无侵入，集成非常简单，社区活跃。 2、对外提供有 Query 接口，容易二次开发	1、完全无侵入， 仅需修改启动方式 2、界面完善，功能细致	1、完全无侵入 2、界面完善，支持应用拓扑图及单个调用链查询 3、属于 Apache 社区项目
缺点	1、代码侵入性较强，需要埋点 2、文档比较混乱，文档与发布版本的符合性较低，需要依赖点评私服（或者需要把他私服上的jar手动下载下来，然后上传到我们的私服上去）。 3、内部已经有新版本，不兼容目前的开源版本	1、默认使用的是http请求向zipkin上报信息，耗性能。 2、跟 Sleuth 结合可以使用 RabbitMQ 的方式异步来做，增加了复杂度，需要引入 RabbitMQ。 3、数据分析比较简单	1、代码质量一般 2、不支持查询单个调用链， 对外表现的是整个应用的调用 3、二次开发难度较高	依赖较多





微服务治理

应用架构遇到的挑战

业务增长

企业业务随着企业的发展呈现不断增长的必然趋势

代码量增加

因此需要补充相应的开发人员，代码量同时不断的递增

维护负担

系统变得臃肿，维护急剧增高，需要投入大量人员了解系统，增加运维负担

扩展艰难

一个微小的扩展可能导致整个系统挂掉，因此导致性能提升困难

性能降低

测试难度大，上线风险高，用户使用效果降低，影响企业发展

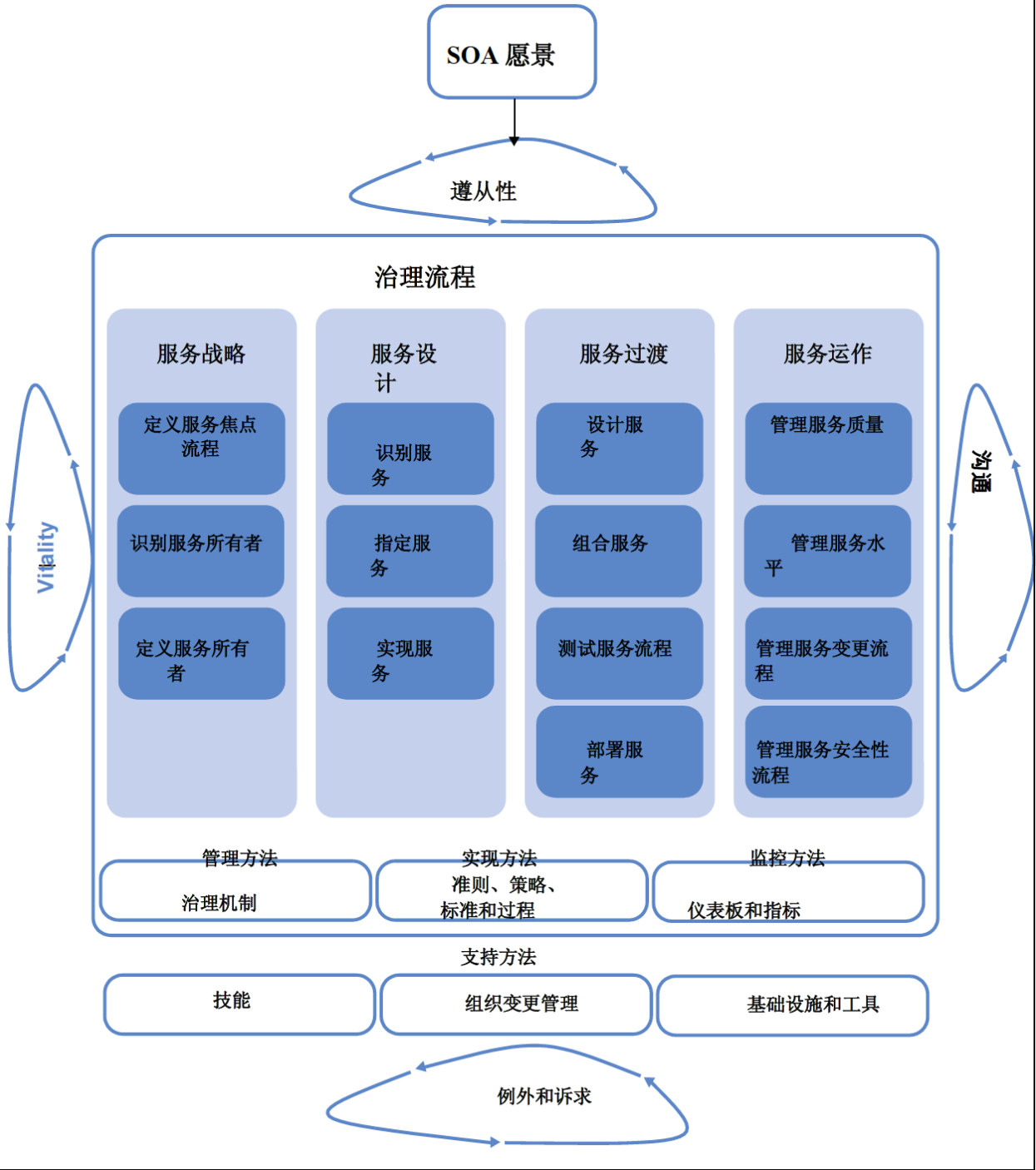
转型挑战

微服务架构落地的挑战

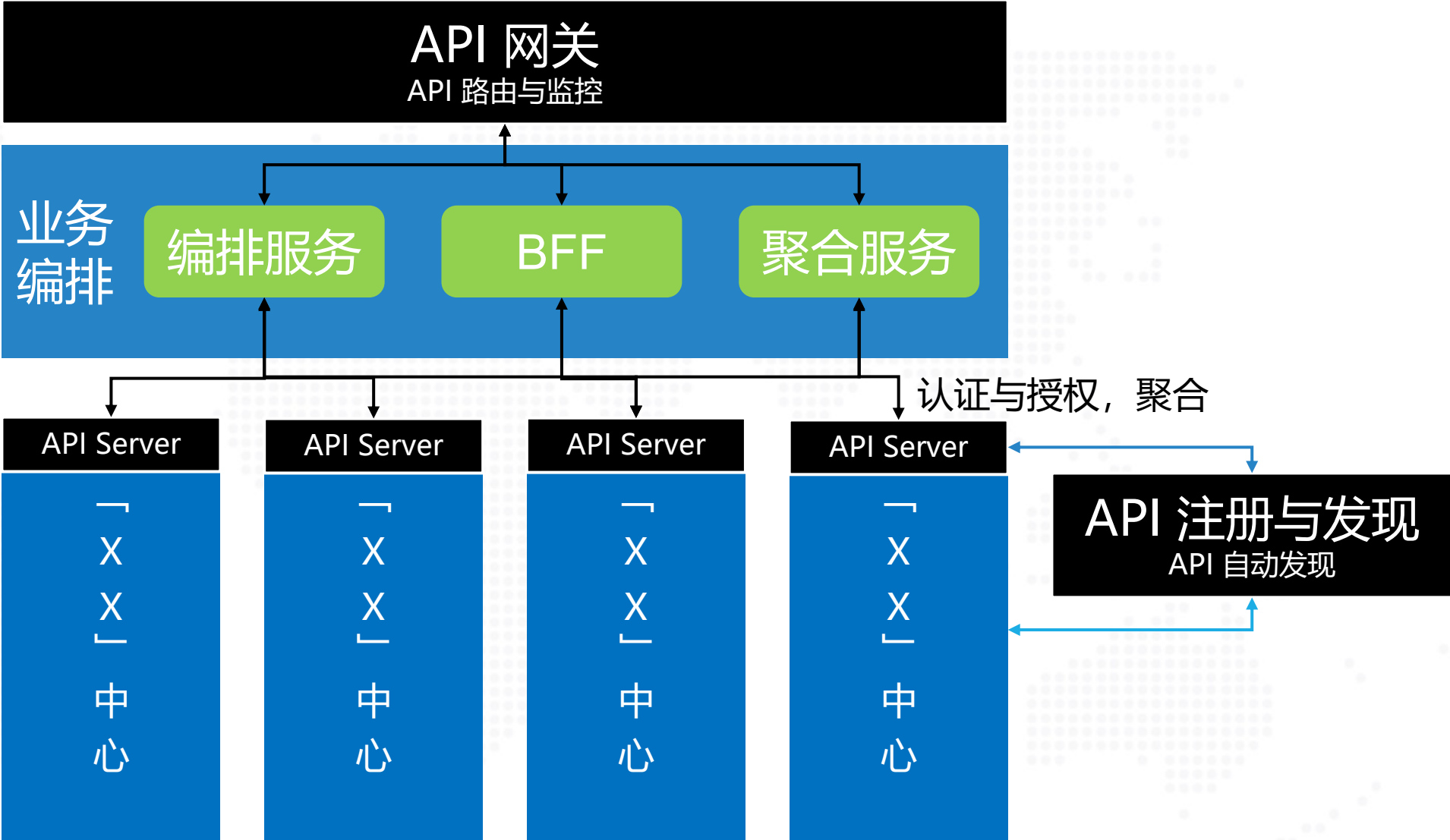


SOA 治理

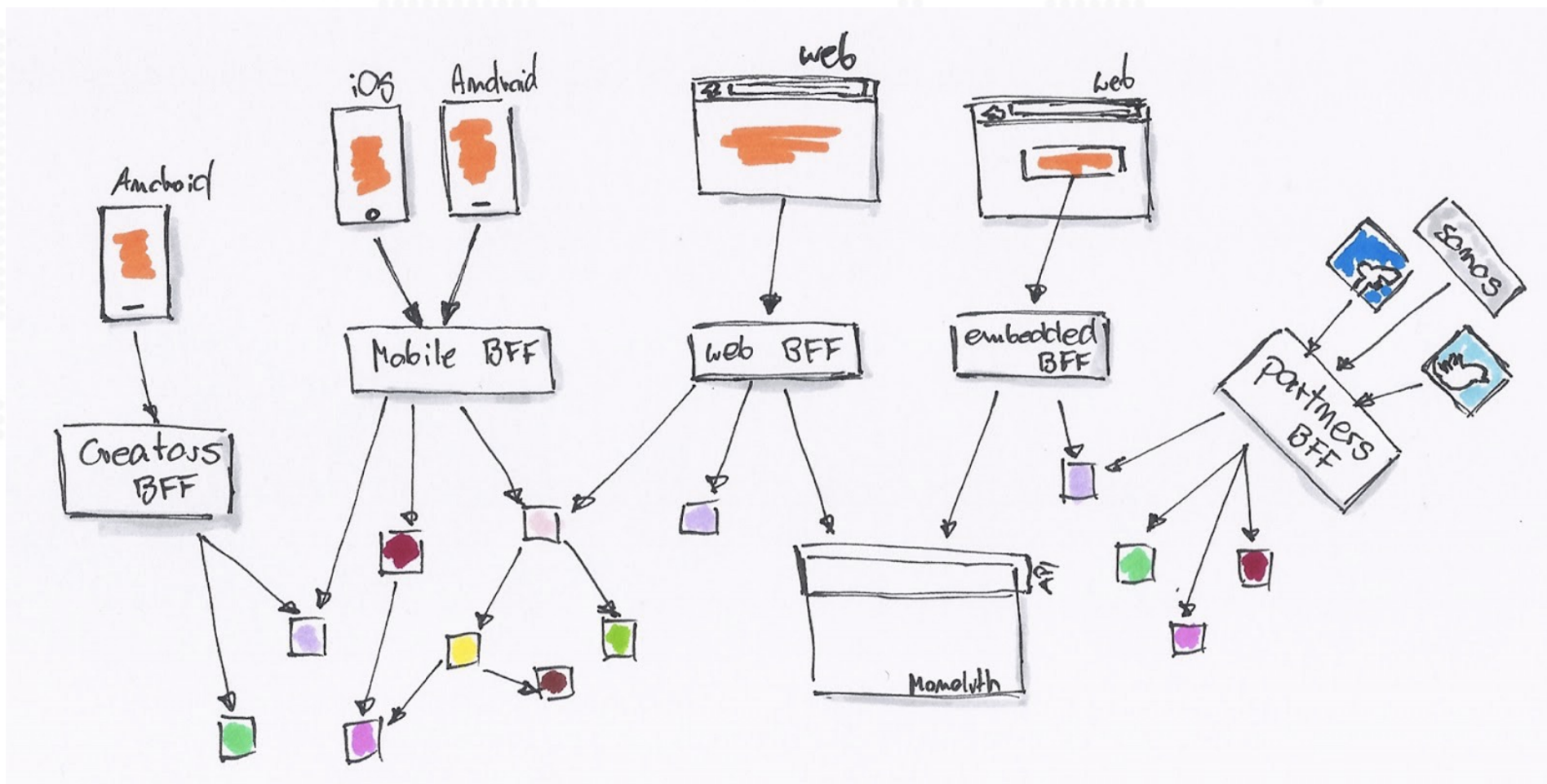
- 20111648-T-469 信息技术 SOA治理
- GB/T 32419.1-2015 信息技术 SOA 技术实现规范 第1 部分：服务描述
- 20100395-T-469 信息技术 SOA 技术实现规范 第1 部分：服务描述
- GB/T 32428-2015 信息技术 SOA 服务质量模型及测评规范
- GB/T 32430-2015 信息技术 SOA 应用的服务分析与设计
- GB/T 32431-2015 信息技术 SOA 服务交付保障规范
- 20111647-T-469 信息技术 SOA 服务交付保障规范
- 20091376-T-469 信息技术 SOA 服务质量模型及测评规范
- 20111638-T-469 信息技术 SOA 应用的服务分析与设计
- GB/T 32923-2016 信息技术 安全技术 信息安全治理



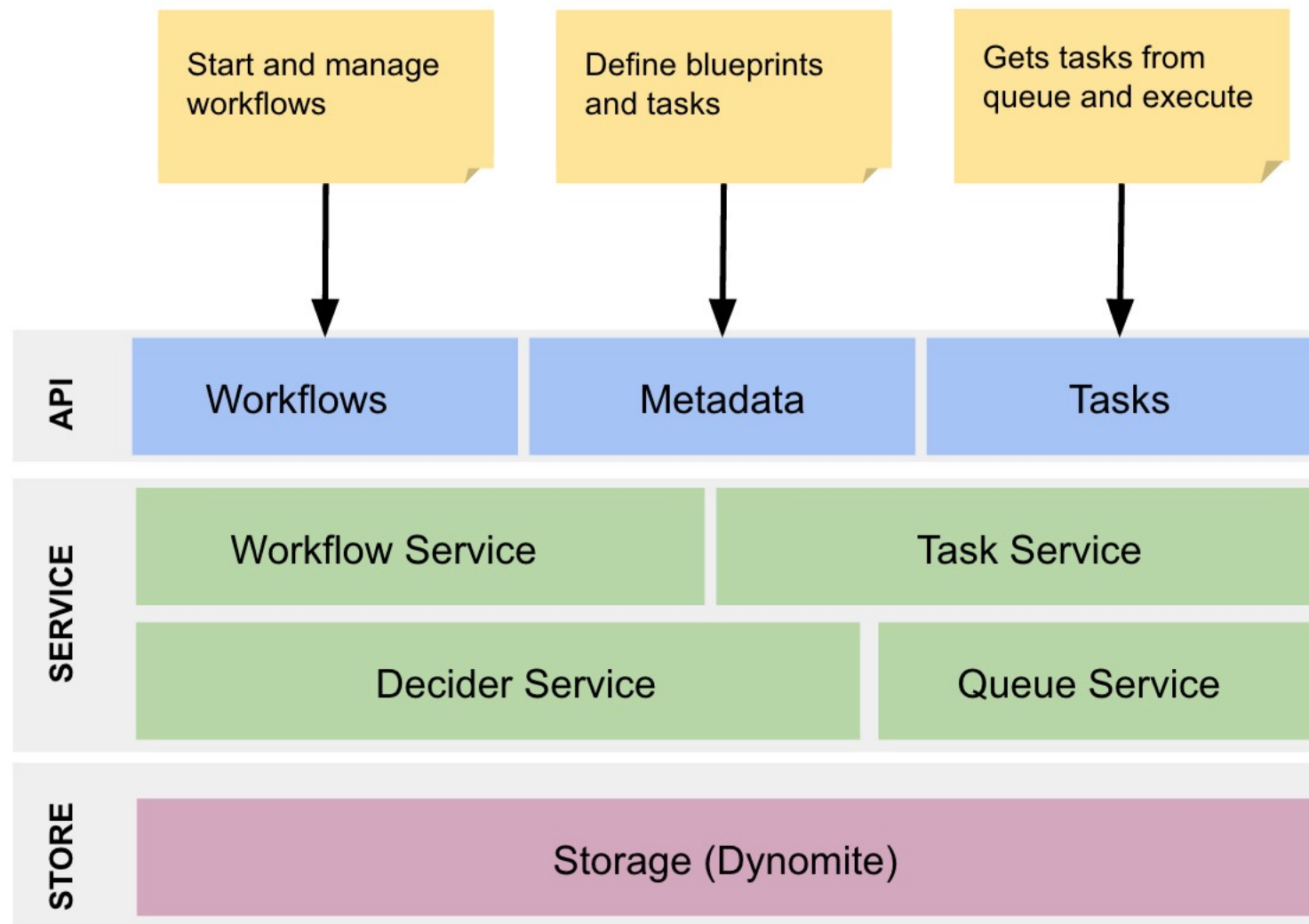


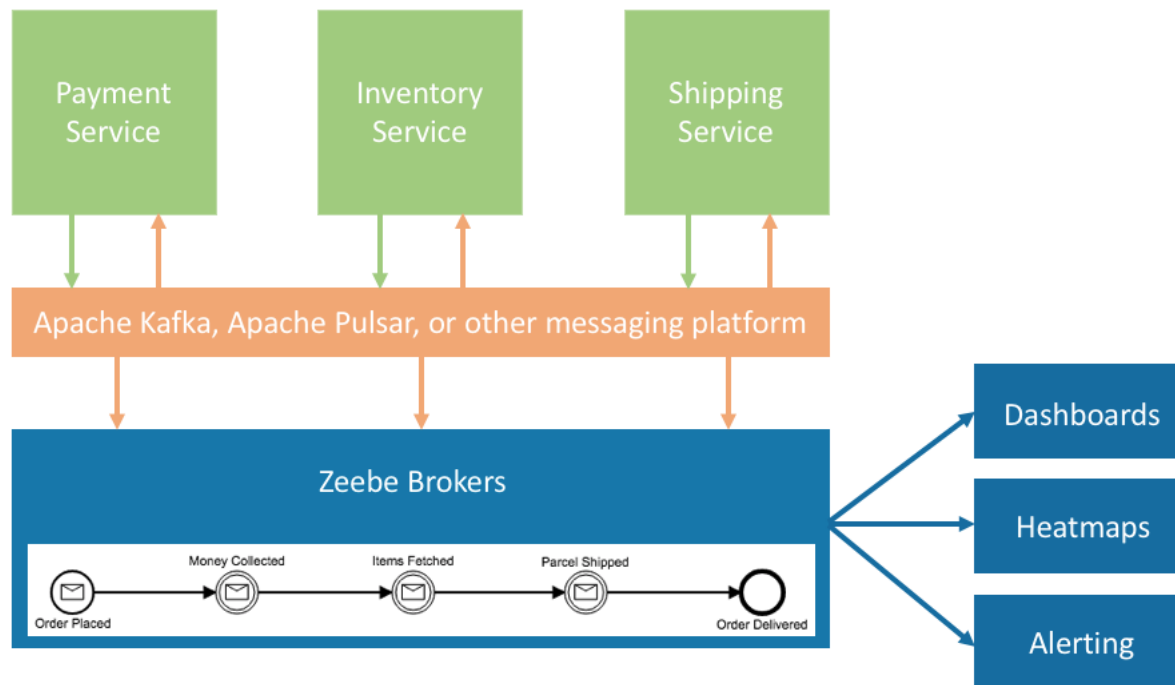


参考架构 – BFF(Backend For Frontend)



微服务编排 – Netflix Conductor





The three pillars of microservices orchestration



Define

Define workflows graphically or via API in BPMN 2.0



Orchestrate

Execute cross-microservice workflows reliably and at scale



Monitor

Get visibility into your distributed microservices architecture

RPC VS REST

	RPC	REST
耦合性	强耦合	松散耦合
消息协议	二进制thrift, protobuf, avro	文本XML, JSON
通讯协议	TCP	HTTP/HTTP2
性能	高	一般低于RPC
接口契约IDL	Thrift, protobuf idl	Swagger
客户端	强类型客户端, 一般自动生成, 可支持多语言客户端	一般HTTP client可访问, 也可以自动生成强类型客户端, 可支持多语言客户端
案例	Dubbo, motan, Tars, grpc, thrift	Spring MVC/Boot, Jax-rs, dropwizard
开发者友好	客户端比较方便, 但是二进制消息不可读	文本消息开发者可读, 浏览器就可以访问
对外开放	对外一般需要转换成REST/文本协议	直接可以对外开放



四个黄金指标 By Google

延迟：服务请求所需耗时

- 例如HTTP请求平均延迟

流量/吞吐：衡量服务容量需求

- 例如每秒处理HTTP请求数

错误：衡量错误发生的情况

- 例如HTTP 500错误数

饱和度：衡量资源使用情况

- 例如CPU/内存/磁盘使用量

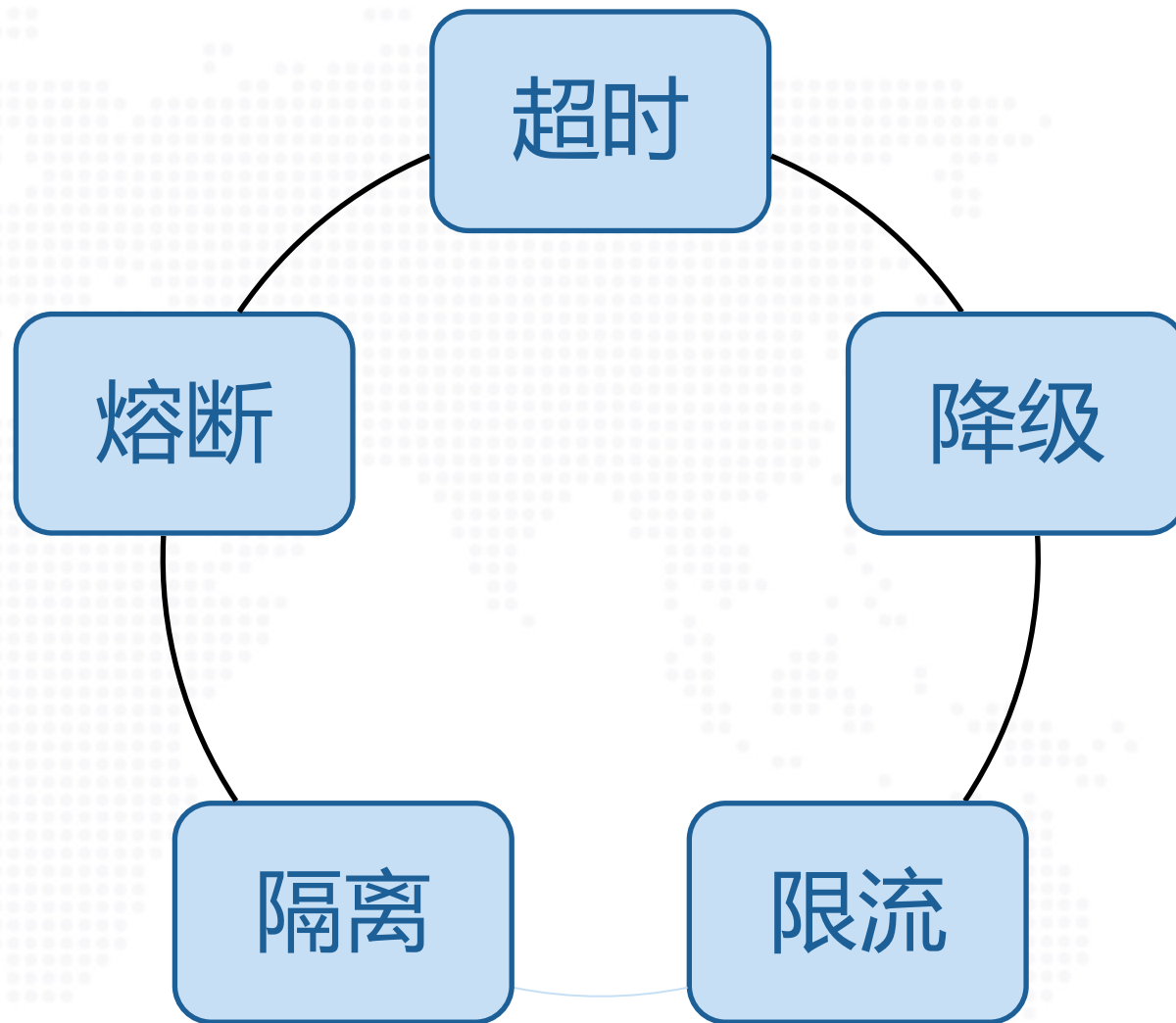


■ 问题

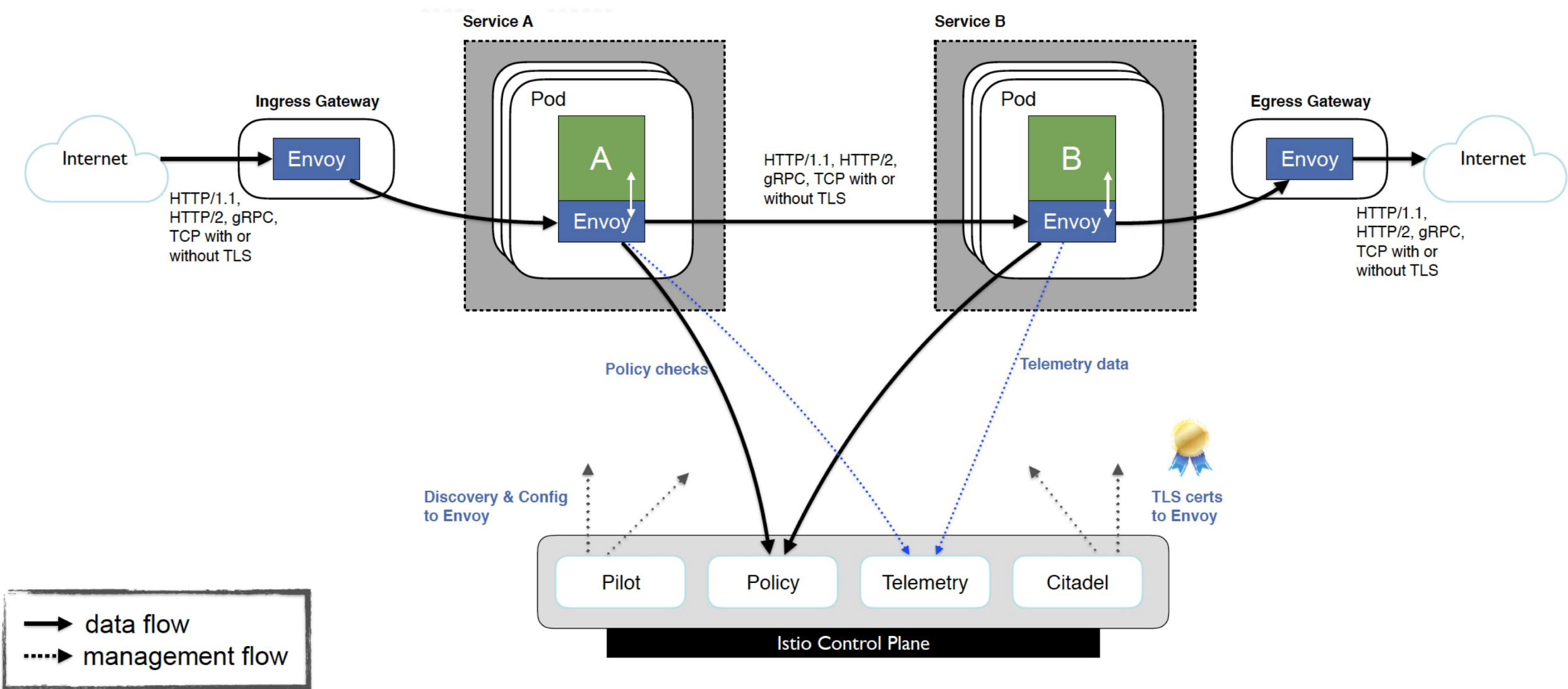
假定一个**单块**服务的可用性是**99.99%**，如果我们有~**30个微服务**，每个的可用性都是99.99%，那么总体可用性是多少呢？

■ 总体可用性

99.7% uptime!
每月**2小时**宕机时间
实际情况往往更糟糕

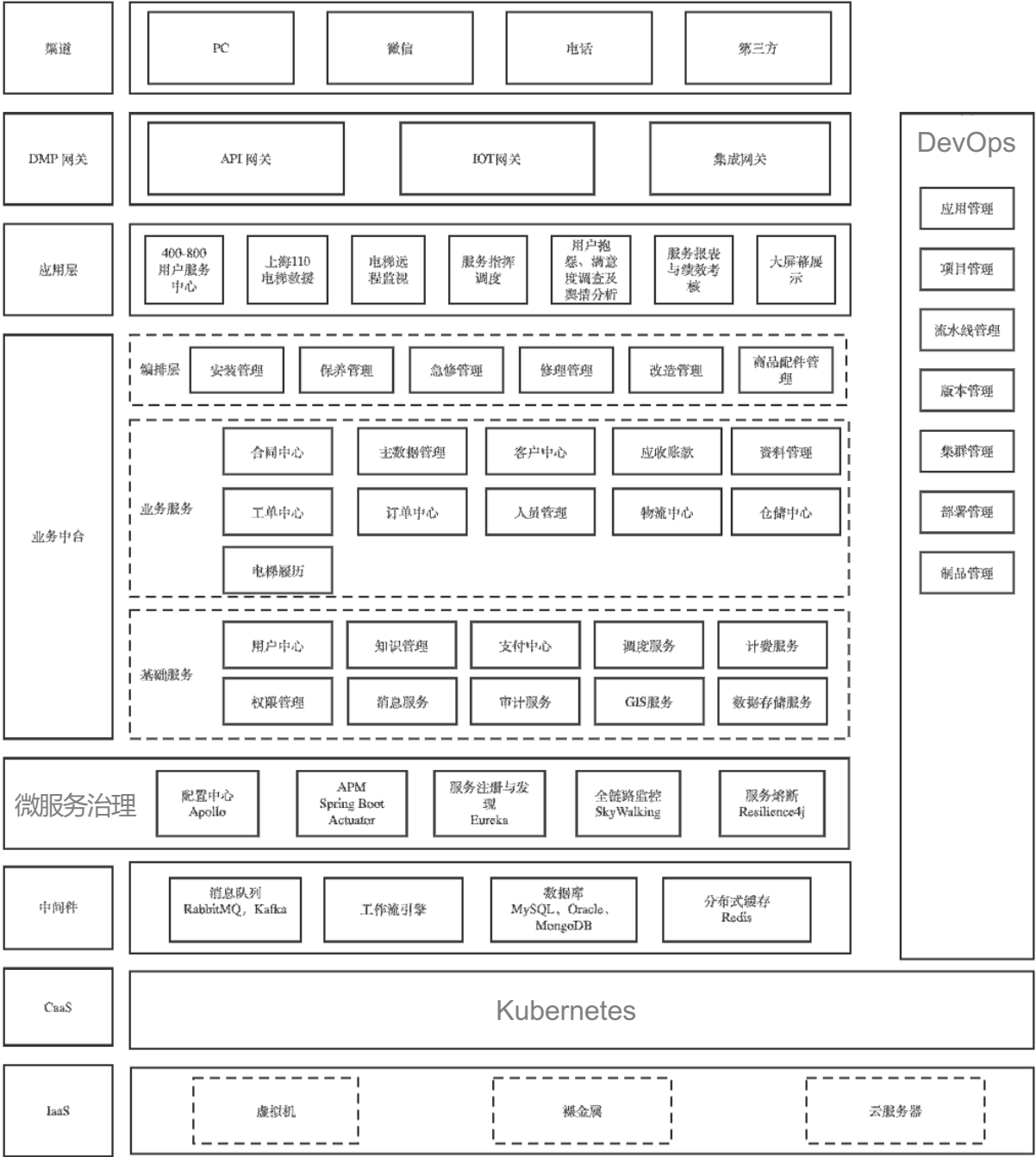


- 凡是依赖都可能会失败
- 凡是资源都有限制：CPU/Memory/Threads/Queue
- 网络并不可靠
- 延迟是应用稳定性杀手



微服务实践案例

某制造业客户



DevOps

- 应用管理
- 项目管理
- 流水线管理
- 版本管理
- 集群管理
- 部署管理
- 制品管理

数据中台

- 大数据 存储
- 大数据 实时处理
- 大数据 批处理
- 数据服务
- 数据可视化
- 数据集市
- 数仓规划



谢谢！

领航大数据与人工智能基础软件新纪元

星环信息科技（上海）有限公司

www.transwarp.io

07 - 2020

